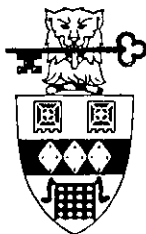


VECTOR

- * APL-ication Details
- * Review of Dyalog APL on IBM 6150
- * Windows and Pop-ups in APL Design
- * Interfacing APL with Assembler
- * APL Software Library Catalogue



*The Journal of the
British APL Association*

A Specialist Group of the British Computer Society

Vol.4 No.2 October 1987

Contributions

All contributions to VECTOR should be sent to the Editor at the address given on the inside back cover. Letters and articles are welcomed on any topic of interest to the APL community. These do not need to be limited to APL themes nor must they be supportive of the language. Articles should be submitted in duplicate and accompanied by as much visual material as possible, including a photograph of the author. Unless otherwise specified each item will be considered for publication as a personal statement by its author, who accepts legal responsibility that its publication is not restricted by copyright. Authors are requested wherever possible to supply copy in machine-readable form ideally text files on a 5¼" IBM-PC compatible diskette. For other standards, please contact the Editor beforehand. Program listings should indicate the computer system on which they have been run. APL symbols should be displayed on a separate line and not embedded in narrative. Except where indicated, items published in VECTOR may be freely reprinted with appropriate acknowledgement.

Membership Rates 1986-87

Category	Fee p.a.		VECTOR copies	Passes
	£	\$		
Nonvoting student membership	5		1	1
UK Private membership	10		1	1
Overseas private membership	18	27	1	1
Supplement for airmail (not needed for Europe)	8	12		
Corporate membership (UK)	85		10	5
Corporate membership (Overseas)	140	210		
Sustaining membership	360		neg	5

The membership year runs from 1st May to 30th April. Applications for membership should be made on the form at the end of the journal. Passes are required for entry to some Association events and for voting at Annual General Meetings. Applications for student membership will be accepted on a recommendation from a course supervisor. Overseas membership rates cover VECTOR surface postage and may be paid in £UK or \$US.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive multiple copies of VECTOR and are offered group attendance of Association meetings. Partaking individuals need not be identified but a contact person should be nominated for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in the editorial section of the journal and opportunities to inform APL users of their products via seminars and articles.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready A5 portrait format with a 20 mm blank border. Illustrations should be black-and-white photographs or line drawings. Rates are £250 per page. A6 and A7 sizes are offered subject to layout constraints.

Deadlines for advertisement bookings and receipt of camera-ready copy are given beneath the Quick-Reference Diary.

Advertisements should be booked with and sent to Cathy Dargue, whose address is given beneath the Index of Advertisers.

CONTENTS

EDITORIAL:	Adrian Smith	Page 3
APL NEWS		
Quick Reference Diary		7
General Correspondence		9
British APL Association News		
APL-ication		19
Membership Projects Scheme	David Eastwood	23
News from Sustaining Members	Cathy Dargue	25
The Education VECTOR	Norman Thomson	31
Report on Winchester Summer Course	Pauline Hawkes	32
REVIEWS		
APL Booklist		36
APL Product Guide	Cathy Dargue	37
Bergquist - Advanced Techniques	Dick Bowman	49
The NEC-V20 processor	Adrian Smith	51
Dyalog APL on the IBM 6150	Adrian Smith	53
RECENT MEETINGS		
Windows and Pop-up Menus	Adrian Smith	62
GENERAL ARTICLES		
Finished Goods Inventory Control	Barrie Webster	70
MetaAPL: an APL Preprocessor	G Tickner & N Oswald	72
Expcheck: Intelligent Diagnostics	Mike Williams	83
TECHNICAL SECTION		
Caption Competition Result	David Ziemann	91
Hacker's Corner	Adrian Smith	92
Technical Correspondence		93
Workspace Management	Peter Branson	99
Go Pack your Knapsack	Norman Thomson	103
Interfacing to PC Assembler (I)	P Wortham	107
Public Domain Software Library		
Call for Software	David Ziemann	113
Full Catalogue	David Ziemann	115
Software Submission Details	David Ziemann	124
Software Order Form	David Ziemann	126
Index to Advertisers		127

NEW MAINFRAME SOFTWARE FOR IBM'S® APL

Now available in the UK, two new offerings from STSC that enhance IBM's mainframe APL implementations

If you're staying with VS APL ...

COMPILER

The first commercial compiler for APL compiles functions individually. Results in significantly faster execution. Interpreted functions can call compiled functions and vice versa.

If you're migrating to APL2 ...

SHAREFILE/AP

STSC's popular APL component file system is now available under APL2. Multi-user, nested array storage, libraries, access matrices. Multiple file system support. International language translations.

For full information, contact the APL*PLUS™ Product Group, Cocking & Drury on 01-493 6172.

Trademarks/Owners: IBM/International Business Machines Corporation · APL*PLUS/STSC, Inc.



COCKING & DRURY LTD.
THE APL PROFESSIONALS

16 BERKELEY STREET · LONDON · W1X 5AE
Tel: (01)493 6172 · Tlx: 23152 MONREF G

Editorial

by Adrian Smith

A VECTOR is variously defined as 'a quantity with both length and direction'; 'the course of an aircraft or missile'; 'a carrier of disease or infection'. So far we have managed a fair length (13 issues and still going), and I hope we are beginning to set some kind of course. What we have not yet done is to carry the disease to those not already infected. VECTOR is still circulated largely within the ranks of the BAA, and if anything our print runs are coming down.

I feel that as Editor, I am under pressure to move the journal in two diametrically opposite ways; do we go 'Quote-quad' and try for technical excellence and APL 'whizzbangs', or do we go 'Education' and 'APLication' to spread the word beyond the inner circle of APL cognoscenti? Are we principally here as a service to the existing membership, or are we a vehicle for converting the masses?

In general the membership survey rated technical articles, software reviews, and application articles as the best items we do. The notes on meetings are seen to be a problem; the most interesting talks are invariably the ones where there is no written paper, or the material diverges so far from it that the paper would be of little relevance to readers.

Here is what we (the working group) intend to do. I cannot pretend that we have all the answers, but at least if we set out our stall in public we can hope for some feedback from our current readership!

Format and Layout

We do not see the need for any major changes here. In particular we propose to ...

- stay A5, and retain the current cover design.
- ensure APL code is big enough to read easily!
- try to find an APL typeface that sits naturally with our normal text face, and standardise on it for all APL code.

Contents

We would like to move towards more regular 'slots' as a way of giving the journal a clearer structure that readers will be able to anticipate and become accustomed to.

- Product Guide and Public Domain Software Library to continue every issue in its present form. Addresses of overseas associations to be carried every issue, in compressed format.
- a 'Software Workshop', typically covering a related set of primitives, or programming technique in around 2 pages. Examples could be: quad-FX, encode and decode, PC windows, Laser printers and how to drive them. A technically proficient member of the group would either write these himself, or vet and select from submitted material.
- part of the above could be a 'Dr Solomon's' column of helpful hints to bemused readers.
- some regular columnists, selected for their ability to write English rather than APL.
- regular reviews of both hardware and software. We shall also keep up the Press Review.
- 'APL Programmers at Work'. A series of interviews, slanted to reveal how we really do the job.
- other languages ... BASIC, Pascal, C, Macro, HP plotters etc.

There will also be the occasional 'theme' issue, e.g. Education. However we think it would be a mistake to try for a theme every issue ... this would be giving far too many hostages to fortune as regards getting enough of the right copy! We will commission articles well in advance from known experts in the field; a bundle of VECTORS should be bribe enough!

Summary

We are aiming for a bit more structure, a guaranteed technical slot of high quality, and a bit more readability. The format and general tone will continue to be much the same as now.

Addendum

On the sore subject of timing! This VECTOR is roughly 4 months late, and has suffered the additional embarrassment of being overtaken by 4.3! Firstly, I would like to

apologise to frustrated readers, and to even more frustrated advertisers. In mitigation, I would say that we now have a much improved production method, typically reducing the lead time on material from an absolute minimum of 12 weeks down to as little as a week.

VECTOR 4.3 was begun in anger just before Christmas (Dec 17th I think). Two of us worked on it fairly solidly for one day, and one morning. It rested untouched over the bulk of the holiday, and was finally assembled (along with Dave Weatherby's review and Graham Parkhouse's article which arrived around Dec 29th) for dispatch to the printers on Jan 6th. Copies were available for distribution by January 15th, and a parcel did make it to Sydney in time for APL88.

In other words I believe we now have a Journal that we can sustain without a killing amount of work, or absurdly long-range deadlines on copy. VECTOR 4.4 will be out on time, and VECTOR 5.1 is roughed out. As long as I am running this beast, and as long as I continue to get the support and help of the VECTOR Working Group, it will not be late again.

Submitting Material to VECTOR

I occasionally dare to open 4.1 round about page 123; it is awful!! Please, please ignore what we said there ... if you have something to say then say it. I don't care if it is on scratty bits of paper, or in the most stylish of typesetting. If it is worth saying and is fit to print then we will jolly well print it!

The ideal is plain ASCII text on PC disk (5 inch or 3 inch are fine). Wordstar documents are OK, and if you have MS Word that would be wonderful! APL code is best printed separately ... FX80 quality is actually quite good (but do put a new ribbon in), otherwise daisywheel or laser are fine if you can do it.

If you don't have a PC, but can produce really good copy from a mainframe system (see Norman Thomson's 'Knapsack' in this issue) then inch margins at each side, 2 inches at the top and 1.2 at the bottom would be ideal. Otherwise we will do a bit of 'shrink to fit' on the copier, which degrades the image a bit (and takes time to do), but is infinitely better than the alternative of resetting the whole article. At least you know that the only typos are your own! Use a biggish font (remember that the result will be halved from A4 to A5 at the final printing stage) and generous line spacing. Do a trial run and reduce it down ... then hold the result up against an old copy of VECTOR to see the effect. The closer you can get the happier we will be!

Outline of Forthcoming Issues

VECTOR 4.4 is basically a conference special. Several VECTOR stalwarts made the trip to Sydney, and I have every confidence that they are even now beaver away at their respective keyboards.

VECTOR 5.1 is a special issue on 'interfaces'. We intend to cover the links between APL systems and (for example) Oracle databases on Vax, C programs on the PC, HP plotters, electronic balances, and mainframe databases such as DB2. It is a wide open field - if there is something you know about which fits under this umbrella, then give me a call and book your slot now!

BACK NUMBERS OF VECTOR

Back numbers of VECTOR are available from the BCS. If you don't have them all, now is the time to complete your collection. Apart from the technical contents, every issue includes book and product reviews, letters, news and a competition. Send in your order before they run out. These will one day be unobtainable collectors' items, like the early issues of Quote Quad.

The prices inclusive of postage and packing are as follows:

	UK	Prices in Pounds Sterling	
		Surface (inc. Europe)	Airmail (outside Europe)
Single issues	3	3.75	5.75
Volume 1	10	14.00	22.00
Volume 2	10	14.00	22.00

Please send sterling cheques or money orders (payable to The British APL Association) to the Treasurer:

Mel Chapman, 12 Garden Street, Stafford ST17 4BT.

Don't forget to include your name and address and to be clear which VECTORS you want.

Quick Reference Diary

Date	Venue	Event
1988		
18 March	London	BAA Meeting
20 May	London	BAA AGM and Meeting
16 September	London	BAA Meeting
28-30 September	Canterbury	APL-Ication the Practical State of the art APL Conference
21 October	London	BAA Meeting
18 November	London	BAA Meeting

All British APL Association meetings are to be held at the Royal Over-Seas League, Park Place, near Green Park Tube station. Meetings begin at 2.00pm and close at 5.30pm.

Dates for future issues of VECTOR

	Vol.4 No.4	Vol.5 No.1	Vol.5 No.2
Copy date	20th Feb 88	1st May 88	1st Aug 88
Ad booking	7th March 88	16th May 88	22 Aug 88
Ad Copy	21st March 88	25th May 88	31st August 88
Distribution	April 88	July 88	October 88

MicroAPL for ...

APL on the widest range of micros

PCs

MultiAPL - APL.68000 under PCDOS

Sample configurations:

68000/1MB	£ 995
68000/1MB/Maths processor	£1295
68020/1MB/Maths processor	£2995
68020/4MB/Maths processor	£5495

Prices include: Plug-in card, APL.68000, Keyboard stickers, Manual, STSC emulation software

STSC's APL*PLUS

£450

Single user micros

APL.68000 for the Apple Macintosh

APL.68000 for the Atari ST

APL.68000 for the Commodore Amiga

All versions £86.91

Prices include: APL.68000, Keyboard stickers, Manual, Reference card

Multi-User systems

APL.68000 runs under the following operating systems:

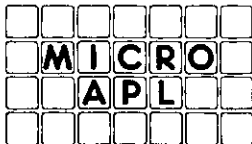
Unix, CP/M68K, Wicat MCS, Mirage

on computers which include:

Sun, NCR, Wicat, HP200/300, Spectrum, Aurora, Stride

Prices range from £1200 to £2000

All prices quoted exclude VAT



MicroAPL Ltd.
South Bank Technopark
LONDON, SE1 6LN
Telephone: 01 922 8866

Access and Barclaycard
Orders accepted

General Correspondence

The VECTOR working group welcomes correspondence on any topic affecting the APL community. All letters should be addressed to the Editor, and should indicate whether they are for the general or technical section. (Letters containing APL code will normally be classed as 'technical'.) The Editor reserves the right to edit any letter unless the writer states that it is to be published in full or not at all.

Experience of 4GLs

From: Dick Bowman

29th May 1987

The VECTOR 3.4 Technical Editorial mentioned 4th Generation Languages, and asked for experience/opinion (well, maybe you didn't, but here are a few words anyway).

Firstly, several years ago I did get a little exposure to what I'm told is a reasonably successful example, called UFO; it wasn't a very happy experience for many reasons, not all of them technical. Nevertheless I can recall one or two things about it which I believe are relevant and may be of general applicability.

- a) the panel design process (called for some obscure reason 'screen painting') was slightly simpler and faster than the 2 FSC124/6 <DESIGN> process which most IBM-based APLers will be familiar with. But not so much so that I ever felt I wanted to write an emulation to use in my APL work.
- b) the system provided absolutely no feedback about whether or not it was working (being used) correctly - if it worked everything was fine, but if not there never was any information about why (my favourite was a message along the lines that 'the information being displayed may not be the information requested' - I really loved the word 'may', it did so much for my confidence).

Now maybe I'm not totally fair on the product, and the UFO vendors could have reasonable cause to get mad at me; but the point is that I've never felt that this product had any advantage to me over using APL and all the utility workspaces which sit around in any well-endowed system.

But I think that beyond particular instances there's a more general topic about APL and where we're going with it that we should be thinking about (and I'm quite happy for the APL community to do its thinking in public).

Back when I started with APL (and it's longer ago than it ever seems to have been) there seemed to be two sports which the community engaged in: IBM-baiting and Cobol-knocking. The first seems still to be with us, which is increasingly unfortunate, but the

second has evolved over the years (were we ever seriously in competition with Cobol?), and now seems rather belatedly to have turned into worrying about APL usage being eroded away by fourth-generation languages.

I predict that you could get a number of contributions like mine above, from APLers who took a look and didn't like what they saw; what you won't get (and what you probably need rather more) is contributions from people who did like what they saw and moved away from APL of their own volition (let's not get distracted by people who had to move because their managers got hyped/threatened/bribed into adopting non-APL solutions).

But that's all beside the point - which is that it's TOO LATE to get bothered about this sort of thing - if we were going to do anything to make sure that APL was regarded as the major tool for solving the sort of problems which fourth generation languages are being applied to then we should have done it four or five years ago. What were we doing then? Mostly snarling at anyone who had the audacity to use ADRS ('real men don't use ADRS').

If we go around on the defensive all the time then we'll lose out (did the Vietcong win by building nuclear bunkers?) - in any case since everybody knows that APLers are offensive why not act like they expect?

What we don't do is to build APL-based fourth-generation packages (the opposition has a size advantage, their fifty assembler coders are going to beat our one APLer every day).

What we ought to be doing is looking forward, and finding niches in the application world which aren't filled (yet?); our strength lies in the processing power of APL and in the speed with which we can utilise it; I for one am not going to get upset if I never again have to write one of the form-filling/file updating applications which I used to do in APL but which can now be handled economically by someone else using a fourth-generation language - the world is full of interesting and difficult jobs which we can handle and these guys can't.

I suggest that the threat to APL is a more subtle one than that of taking away some of our existing work - what the package suppliers want to do is take the DP department onto a roller-coaster of their own proprietary products regardless of the eventual cost to the customer. There are one or two questions which the APLer ought to be asking in situations which bring them close to things like fourth-generation languages:

- a) Are all foreseen requirements handled by the package as it presently stands - or are future add-ons likely?
- b) Can the data be processed with existing 'standard' languages?

- c) Is there a requirement for portability/comparability across a variety of hardware/software environments - how is this met?
- d) How long is this solution supposed to last?

I think that APL can give pretty good answers to these sorts of questions; the impression I've received has been that what you get with a fourth-generation language is a very good response to a very limited class of problem. Now some people only have limited problems and good for them - what APL needs to look at is the more ambitious problem where the fourth-generation answer only goes part way; we can eat them for breakfast here if we try (and are allowed to).

Dick Bowman
CEGB
85 Park Street
LONDON SE1

Comments on APL

From: Richard L. Connor

31st May 1987

I'm taking advantage of the opportunity to renew my subscription to offer a few comments on items I noticed in the 1987 April issue of VECTOR.

Regarding Norman Thomson's postscript to his letter on page 9, the issue of the name of Johns Hopkins University's Applied Physics Laboratory is discussed in Eugene McDonnell's introduction to 'A Source Book in APL' (APL Press, 1981). Mr. McDonnell also mentions that here in the San Francisco Bay Area, "APL" is known to most people as "American President Lines", a Pacific Basin shipping company.

Two articles discussing approaches to compiled APL appear in the IBM Journal of Research and Development, Vol 30 (November 1986) on pages 583 and 594, respectively. One may obtain a copy of that issue from IBM, Order number G322-0150.

For what it's worth, I feel that APL compilers have a limited place for a while. I find the hurdles facing the handling of shared variables and of loose typing in general rather formidable.

I too am concerned about the thin appreciation of APL. I believe part of the problem is that we APLers and others take the full name too seriously. This leads to publicity in the wrong places. We should be emphasizing APL less or not at all in computer science curricula. One of APL's great virtues is that its effective use doesn't require technical

knowledge of computers and software. It is an excellent language for the end user. Hence, I think our educational efforts should reach the potential end users: those who are studying statistics, engineering, economics, almost any discipline requiring numeracy.

History can perhaps provide lessons

APL appeared in the rather early days of that hardy survivor FORTRAN. One of the reasons that FORTRAN caught on and remains valuable is that it too is user-friendly. I've trained countless engineers, geologists and chemists to use FORTRAN, and I haven't had to talk much about computers. They became enthusiastic because they could go right to work with it. I venture to say that FORTRAN wouldn't have enjoyed its continuing success had most of our efforts gone into teaching it to professional programmers. Professional programmers, quite naturally in order to keep their jobs, prefer "programming" languages.

One of the factors behind the growth of the desktop computer industry is that many of the successful languages - Lotus 1-2-3, WordPerfect, etc., also require no professional programmer.

There will and should be a place for the APL "programmer" to create hardware- and software-oriented auxiliary processors, for example; but the real future and most of the added value lies on the APL user.

The preceding remarks about our audience may have some relevance to Mr. A.N. Wiggins' complaint about the marginal costs of distributing APL programs. If we sell the user properly on APL, he'll look upon it as a worthwhile investment in itself, and there should be a growing demand for workspaces from small developers like Mr. Wiggins. I know that Mr. Wiggins will find this observation cold comfort; but we must start somewhere, or else we'll see the continued decline of one of the finest aids to productivity ever hooked to computers.

I also take this opportunity to congratulate you and the Committee on your fine publication.

Richard L Connor
1243 Pine Street
San Francisco
CA 94109

APL Hand Calculator

From: E J Sargeant

30 May 1987

Re the editorial of VECTOR Vol 3 No 4: it strikes me that what is required, in order to promote the language, is a cheap hand/pocket calculator. This should provide the APL notation, except the programmatic ones del; goto etc., indexing keys and indication to step a one-line display through tables and alphabetic keys for variable assignment. 36 multi-function keys should do it OK, which is comparable with current 'scientific calculators'.

The problem, it seems to me, being that whilst APL was devised as a maths notation, before being accepted as such, it became the big blue programming language of the elite.

E J Sargeant
33 Breedon Street,
Long Eaton
Nr. Nottingham

The Failure of the APL Community to Market Itself

From: Barrie Webster

30th July 1987

I have been involved with APL since 1973. In the first instance I used the STSC time-sharing service and after spending six years with APL*PLUS LIMITED, Gareth Brentnall and I formed Mercia Software at the beginning of 1985.

In the whole of my professional data processing career I have never come across a community so dedicated to contemplating its own navel.

I have observed over the years that many of the people involved in promoting APL vary from the technical nutcase to the megalomaniac. There is no doubt in my mind that the major reason for the lack of growth in APL is that the product has never been sold. On the one hand we have the technical types who consider APL a thing of beauty, an end in its own right, which should certainly not be debased by allowing it to be used commercially. Whenever these individuals find the time to speak or to write then it is to chunter on about the overwhelming pleasure to be obtained by transposing the reshape of the dyadic form of the whatsit operator. I suppose at the end of the day if that's what turns you on fair enough, but I am quite certain that this is not what industry, business and commerce want to hear.

At the other extreme there are those who claim to be commercially oriented. Here I have noticed two phenomena. First of all there is the desire to concentrate on excluding all others from the market place. The APL market today is tiny in terms of the total software market place. For a vendor to concern himself primarily with the exclusion of competition from the existing market place is narrow-mindedness the like of which is in my experience unparalleled. Secondly the majority of products tend to be APL interpreters or APL utilities. Such products will be of interest to the APL community but not to business people. Why do those who talk endlessly about APL as a powerful and productive development tool not use it to develop application software? Do they not see that the Finance Director of a company looks for financial applications software, and the Production Director looks for Materials Management software? Is it not clear that these are the people that matter, and they have no interest in APL itself but are only interested in application tools?

I see a constant flow of self-congratulation. Success is achieved outside the APL community not within. For example the APL 86 event was to my mind a failure, it was the APL community talking to the APL community. Why was U.K. business not persuaded to attend? The reason I believe is because they were actively not wanted.

It was an ideal forum for a company to send non-APL people to an exhibition where vendors would show the benefits to be achieved with the use of APL systems. These people who would have visited the exhibition for a day were effectively barred by only providing registrations for the full week.

We are a very small company and consequently our impact is small. What you will see however is most of our limited advertising being spent on promoting products like LOGOL, a finished goods control system, in "trade" publications like BPICS and the O.R. Newsletter.

The last issue of VECTOR published a survey where surprise was expressed at the statistics showing that APL*PLUS/PC accounted for as much as 28% of implementations used. The reason is simple; in early 1983 when the IBM PC was launched APL*PLUS LIMITED set in motion a plan to actively market APL*PLUS/PC. This involved advertising in all sorts of media including "horizontal" magazines like PC USER. We organised mailshots and took the product to trade shows. In short we actively sold the product rather than waiting for the prospects to come to us pleading to be allowed to buy the product.

Back in the early seventies APL was going to be the number one language. It has not happened, the facts show that it claims a tiny proportion of the market. Stop kidding yourselves it is growing and the problem is with the shortage of APLers. That is rubbish, demand drives production in almost every environment, it is not the other way around.

To address the problem of penetration of APL in the market place the community needs to totally re-think its strategy, It must provide application products and it must sell. Let me emphasise the word "SELL", this means going out into the big wide world and persuading company executives to buy your products.

Barrie Webster
Mercia Software Ltd
Aston Science Park
Love Lane
Birmingham B7 4BJ

APL '90 in Copenhagen

From: Henri Brudzewsky

20th July 1987

At APL '87 in Dallas the ACM accepted the Danish offer to host the APL '90 Conference. I wrote the following introduction to the Danish APL Group for the ACM. It might also be of interest for the readers of VECTOR. The postal address of the APL '90 Group is:

APL '90 Gruppen
Dansk Databehandlingsforening
Kronprinsensgade 14
DK-1114 Copenhagen K.

For any questions, you are welcome to contact me through Sharp mailbox code <HENRI>, or the other members of the organizing committee, Gitte Christensen <GITTE> and Kim Andreason <KIMA>. The group also has the common mail box code <APL90IG>.

The History and the Organization of the Danish APL Group

In 1968 Dr. Kenneth Iverson demonstrated APL at an international summer school at the Technical University near Copenhagen. The following year enthusiastic Danish "witnesses" of that demonstration started the first commercial service bureaux outside the US. The number of such bureaux soon increased from two to five, thus providing Denmark with the highest APL density in the world.

As a natural consequence of that status Denmark became the host nation of the international APL Congress 73, which is still remembered as one of the best APL conferences ever held. The conference was arranged by an ad hoc group representing the three APL vendors that were active at that time. This was to last another four years until the Danish APL Group was founded.

It is a well-known fact that one APLer can often replace a dozen or more COBOL programmers. Therefore it is not a surprise that an APLer can feel lonely - even with the highest APL density in the world.

Early in 1979 two of the lonely APLers met and decided to do something about their situation. They soon discovered that they were not at all alone: Denmark had scores of lonely APLers.

After several months of discussions on How and What it was decided to create a group of six persons plus a secretary from the Danish Data Processing Association. In that way the new-born APL Group would have access to offices and files. By meetings and other arrangements the Group should work for a more widespread use of APL as well as helping APLers to a better utilization of the language. The Danish APL Group could thus be characterized as a committee making arrangements which are open for whoever might be interested rather than a club with ordinary membership.

A series of afternoon meetings on various subjects soon brought the number of attendees close to a hundred. In September 1979 the 10th anniversary of APL in Denmark was celebrated by holding a two day APL conference in the Tivoli Gardens of Copenhagen. Two hundred and fifty APLers came from most parts of Scandinavia. The following years several other conferences, exhibitions and workshops were held in addition to the small afternoon meetings.

Today the Danish APL Group has grown to eight people and a secretary, but it still works without a chairman. An ad hoc committee of three persons from the APL Group has undertaken the task of organizing the APL 90.

Working under the auspices of the Danish Data Processing Association has been a very positive experience for both parties. Arranging the APL 90 in Copenhagen is therefore looked forward to as a culmination of that good cooperation, and as a way leading to another highlight in the history of APL.

Henri Brudzewsky
Sjølundsparken 23
DK-3150 Hellebaek

Impetus flies the Nest

From: George MacLeod

3rd July 1987

During the last three years a new Personal Computer product called IMPETUS has been developed at Boeing, based on the TABAPL hierarchical concept, but taking advantage of the economies of PCs while still providing the functionality of a system designed for a large mainframe.

IMPETUS is written in STSC's APL*PLUS/PC, with recourse to assembler where necessary to achieve satisfactory performance. It provides a framework for structuring planning and reporting systems which allows both APL novices and experts to use the inherent power of APL without the responsibility for formatting, saving workspaces, maintaining files and so on.

IMPETUS has now completed 15 months of field testing and is about to be launched to a wider market place. Unfortunately IMPETUS no longer fits the Boeing product strategy and as a result Cedric Heddle, who developed and supported both TABAPL and IMPETUS, has left Boeing to set up his own company - APL Impetus Limited, and has acquired the IMPETUS product from BCS.

Cedric Heddle has been developing and supporting IMPETUS, and before that TABAPL, for over ten years, and no doubt his new company will be offering IMPETUS products and services for many years to come.

Cedric may be reached on 0732-885126.

George Macleod
General Manager
Boeing Computer Services (Europe) Ltd
P.O.Box 747, 364 Euston Road
LONDON NW1 3BQ

LLOYD SAVAGE LTD

**are pleased to announce that due to expansion,
we will be moving to new premises as from**

FRIDAY, 10TH JULY 1987

Our new address will be

**Wallis House
27 Broad Street
WOKINGHAM
Berkshire
RG11 1AU**

Our new telephone number will be

Wokingham (0734) 771266

APL-Ication

The Practical State-of-the-Art APL Conference

University of Kent
Canterbury, England
28-30 September 1988

The British APL Association is pleased to announce this three day conference to be held in the beautiful Cathedral City of Canterbury in September 1988.

APL-Ication is designed to be complementary to APL88, being held in Sydney, Australia from 1-5 February and will be an exciting, outward looking conference of interest to all APL practitioners - both newcomers to APL and those with more experience in the use of the language. We extend an invitation to other APL club members, particularly those in Europe, to become involved in **APL-Ication** by submitting papers, offering presentations and by attending the conference.

Canterbury

The City's roots can be traced back to a prehistoric iron age settlement, but the story of Canterbury really starts after the Roman invasion of AD 43. Then known as Durovernum Cantiacorum, it was a trading centre on the main route between London and Europe. After successive occupation by the Vikings and the Danes, the Normans arrived and it was they who began building the Cathedral in 1070. In 1170 Archbishop Thomas Becket was murdered in the Cathedral. Pilgrims from all over Britain and Europe were soon travelling to Becket's shrine, a typical pilgrimage being recorded in Chaucer's *Canterbury Tales*. Despite the later attentions of Henry VIII and Cromwell the Cathedral survives to this day as the 'Mother Church of English Christianity', and is famed for its collection of stained glass windows.

University of Kent

The conference will be located at the University which overlooks the Cathedral City of Canterbury - a famous City, situated in the heart of the Kent, the county known as the 'Garden of England'. The University campus, opened in 1965, offers outstanding facilities for staging conferences, with on-site accommodation and social amenities. The University can be reached conveniently by road and rail. Canterbury is close to the motorway network affording rapid access to all parts of the UK and Europe via Dover, which is close by. The rail link to London also provides access from all parts of the country and from Heathrow and Gatwick airports.

APL-Ication

The conference has been designed to give a careful balance between the presentations given by invited speakers and contributed papers, with the accent on the practical application of APL in a variety of business disciplines. The conference will also look critically at the world of computing to see what lessons we might learn and how APL can be integrated into non-APL environments.

APL-Ication will have an international flavour, with contributions from overseas, particularly our APL colleagues in Europe. Anyone interested in presenting a paper should follow the instructions for submission given in the following Call for Papers.

Although there will be no formal exhibition, hospitality suites will be available for consultancies and vendors to meet delegates and to display their products, and time will be available in the lecture theatres for presentations of software and hardware for the APL user.

Further details of **APL-Ication** will be given in the next issue of *Vector*.

APL-Ication Conference

University of Kent
Canterbury, England
28-30 September 1988

Call for Papers

APL-Ication is a three-day conference being organised by the British APL Association, devoted to the topic of the **Application of APL in Business**. Who uses APL? Why do they use APL? What does the practitioner need to know to apply APL effectively? What changes to APL would make it more useful?

In addition to contributed papers, there will be a number of invited speakers from both within and without the APL world who will present papers and lead tutorial and discussion sessions. These speakers will detail experiences gained in non-APL as well as APL fields, beneficial to the APL professional.

Sessions currently planned include:

APL and Software Engineering.

Case Studies. Why was APL used? What benefits accrued? Will APL be used in similar situations again? What changes to APL would increase its value to such applications?

Teaching APL. What education is relevant to those using APL as a problem solving tool for business? How do we teach it?

APL and the Human Computer Interface.

APL and other systems. The use of APL to augment 'standard' packages and databases. Why in a multi-language system does APL always seem to be the one that bends?

Development and productivity tools for the APL professional.

Other programming languages, particularly functional programming and object oriented languages. What are they, and what are the implications for APL?

APL-Ication will be an exciting, outward looking conference of interest to all APL users at all levels. The event will be highly participative. No talk will exceed half an hour, and all sessions will be concluded by a panel discussion involving the presenters. 'Live' presentations to accompany papers will be encouraged by the provision of a wide range of computing and audio-visual equipment. The University of Kent has excellent facilities for both speakers and delegates.

Papers are welcome from outside the UK, particularly from Europe, as it is our aim to make APL-Ication a truly international event. We would like to share the experiences of using APL gained in the UK with our overseas colleagues and learn how they put APL to effective use in other countries.

It is important that we adhere strictly to a timetable for the submission of papers, leading to publication in the conference proceedings. This timetable has been carefully organised in order that work on your submission to APL-Ication will not interfere with your visit to APL88 next February.

Submission of Papers

Full draft papers, for consideration by a panel of referees, must be submitted by 15 January 1988. Four copies of the paper, which is to be in English and should not exceed 6000 words, must be submitted, together with:

a title, an abstract and a list of keywords for subject identification and classification,

a photograph and a short autobiography, and

a full description of the 'live' presentation and the equipment required.

Notification of the referees' decisions will be sent out on 1 March 1988. Authors of papers must provide final copies of their papers suitable for photo reproduction by 29 April 1988.

All papers accepted and received by the final submission deadline will be published in the Conference Proceedings. Other papers will be considered for publication in Vector.

It is intended that prizes will be awarded for the 'best' papers and presentations given at the conference.

Important Dates

Submission deadline	15 January 1988
Acceptance deadline	1 March 1988
Final Version	29 April 1988

For further details on the submission of papers, or if you would like to be on our mailing list to receive information about the conference, please contact:

BISL Conference Department (APL-Ication)
The British Computer Society
13 Mansfield Street
London W1M 0BP
England
UK

Telephone 01 637 0471

APL-Ication Organising Committee

Chairman	Dick Bowman
Treasurer/Administration	Mel Chapman
Publicity	Philip Goacher
Programme	Christine McCree
Technical	David Parker

Info Center/1

*an IBM licensed program that helps business professionals
perform their daily tasks quickly and productively*

Info Center/1 provides an integrated, multifunction information center environment compatible with predecessor products such as ADRS II and APLDI II. A full-screen interface, with prompts and extensive help facility, provides easy access to the following powerful general business functions, as well as providing the full power of APL:

Query System

The Query System provides a simple, effective way to interactively access, analyze, manipulate, and report information stored in files of up to several hundred megabytes.

Reporting System

Provides an organization with a single, comprehensive system for generating and maintaining reports. Standard calculations can be defined and stored for future use. Calculations can be made with predefined functions and with APL.

Data Entry and Validation

This tool allows information center personnel to tailor panels for users to display, update, and enter data in column format.

Financial Planning System

The Financial Planning System provides a set of 60 modeling routines that work with the Reporting System and address periodic data. Some examples are:

- Financial analyses and plans
- Statistical analyses and projections
- What if analyses and modeling
- Project evaluations and risk analyses.

Business Graphics

The Business Graphics facility is a particularly powerful yet flexible tool for interactively producing the following types of charts: line graphs, surface charts, histograms, pie charts, scatter plots, bar charts, stacked bar charts.

Technical Data

Info Center/1 is an IBM Licensed Program, Program Number 5668-897.

The program runs under CMS and TSO together with the following IBM programs or their equivalents: APL2 or VS APL, Application Prototype Environment, GDDM (Graphical Data Display Manager). Some examples of terminals supported are: IBM 3277, 3279, 3270 PC/G and GX.



Membership Projects Scheme

After APL86, the BAA committee decided to channel some of the proceeds of the conference back to the membership of the Association. In particular, it was decided to establish a 'Membership Projects' award of up to £5,000 in any given year. The awards are for those projects which, in the view of the BAA Committee, will do the most to further the aims of the BAA, viz:

- Promote the use of APL
- Help people to use APL well
- Contribute to the development of APL and international standards
- Increase the membership of the BAA

The scheme was announced in Vector and the announcement implied that the scheme was only for the year ending June 1987. In fact the BAA Committee has decided to keep the scheme going at least until the next AGM in 1988. We have decided to issue this reminder about the Projects Scheme to prompt further applications for this year's award.

The award is for a total sum of up to £5,000 in any given year for the project or projects which, in the opinion of the BAA Committee, are the most deserving. The choice of project is entirely up to the applicant, but examples might include:

- Writing/Publishing a new APL book
- Development of new APL software
- Development of the APL language itself
- APL-based research

The award can be used to finance time off work, the purchase or rental of equipment, the preparation of material for publication etc. The awards will be made at the AGM of the BAA and selection will be made by members of a BAA subcommittee and ratified by the full committee. The rules are simple:

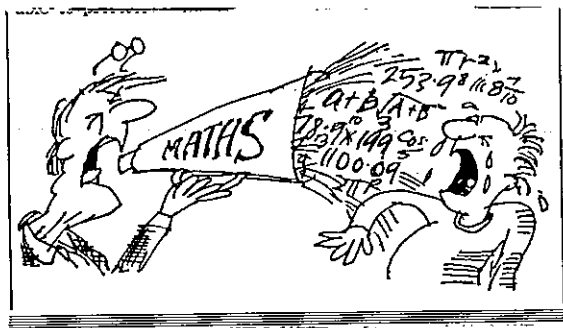
1. Projects must be proposed and carried out by paid-up members of the BAA.
2. A sum of up to £5,000 is available for the projects deemed most suitable by the BAA committee.
3. The decision of the BAA committee in choosing suitable projects is final.
4. Members of the BAA committee are not eligible for this award

5. A contract governing the terms and methods of payment will be drawn up between the BAA and the recipient of the award.
6. If the project has some commercial worth, the award will be treated as venture capital funding and the BAA will expect some suitable return on the investment.
7. The awards will be made to the project or projects which, in the opinion of the BAA Committee, are the most original, best further the aims of the BAA and will not be carried out in the absence of this funding.
8. Submissions for project awards should be drawn up to include some budget, timescale and objectives.

The progress of the project will be monitored by the BAA committee and in many cases the funding will be released in stages as objectives are met.

Proposals for project awards should be made to the Projects Officer.

THE GUARDIAN
Saturday February 27 1988



News from Sustaining Members

Compiled by Cathy Dargue

Mercia Software Ltd

We have now completed two major development projects which extend the capabilities of STATGRAPHICS. A major U.K. oil company commissioned us to develop extensions to the Quality Control routine in STATGRAPHICS in order that the Q.C. methods conform with the new standards that the Ford Motor Company have issued to their suppliers. One of the world's largest chemical companies have for some time used a highly sophisticated CUSUM time series technique. This has now been fully integrated into STATGRAPHICS as a custom modification.

LOGOL, which is our new finished goods control system, has now been installed on a trial basis in two well-known companies. R.G. Brown, the author of the system, will be paying a visit to the U.K. shortly to hold a week's course on Materials Management, and also for discussions with Mercia about future developments to the system.

APL People Ltd

Following the successful completion of a very large financial system for one of the major banks, we now plan to focus more on developing banking and treasury systems for clients, whilst still maintaining our existing consultancy services.

The APL Group's headquarters down in Bath is getting ready for another move - the present offices having become too cramped for all of us following the addition of Chris Cunnane, who has taken over from Jill Moss as Office Manager in Bath. Jill will now be concentrating on the employment agency side of the business, which is still expanding at an encouraging rate.

The new Sheffield office is now up and running, and with two very experienced salesmen working from it, sales of our engineering software packages in the Midlands have really taken off.

The work that APL Tran-Plan is carrying out on the now legendary Canary Wharf development in London's Docklands continues apace. There was one nasty moment when we thought the whole thing had been wound up when the original developer failed to get commitment on the funding of the project. However, literally at the eleventh hour, a new developer was found - who happens to be the biggest single real-estate owner in New York - and now it's all systems go!

Dyadic

Dyadic has announced further details of its new version of Dyalog APL for Intel 80386-based computers.

Dyalog APL/386 is designed to run on a wide range of 80386-based PC AT-compatible personal computers, including those manufactured by Compaq, Wyse, Zenith, Olivetti, ITT and Texas Instruments. It will also run on IBM PC AT-compatible machines which have been 'turbocharged' with a 386 accelerator card.

Dyalog APL/386 is a full implementation of Dyalog APL, but one that is specially configured for personal computers; and it includes support for standard PC keyboards, monitors and printers.

Not only is it a full second-generation APL with nested arrays, but Dyalog APL/386 supports very large objects and workspaces, up to a theoretical limit of 4 gigabytes!

Dyalog APL/386 uses 32-bit native 80386 code, and therefore takes full advantage of the 386 processor. This means that performance of a 386-based PC running Dyalog APL/386 is comparable to far more expensive Motorola 68020-based APL systems. In some cases it is actually faster! Detailed benchmarks are available from Dyadic on request.

Dyalog APL/386 runs under SCO XENIX 386 System V Operating System which is available direct from Dyadic. In addition to supporting Dyalog APL, XENIX 386 provides a full 32-bit multi-user, multitasking environment; and unlike DOS, XENIX 386 releases the full potential of the Intel 80386 processor.

Dyalog APL/386 includes an interface to SCO CGI, a powerful device-independent graphics development system, which is packaged with Xenix. It also includes a workspace containing business graphics functions for producing graphs, histograms, pie-charts, titles and legends. The system supports standard PC displays, dot matrix printers, and other popular graphics devices.

Two important system extensions, SCO Multiview and SCO VP/ix, will soon be available and will be supported by Dyalog APL/386. Multiview is a combined multitasking windowing environment and user-friendly interface to Xenix. SCO VP/ix permits multiple DOS and XENIX applications to be run concurrently from the system console and on attached ASCII terminals. This means that all the benefits of speed, size and functionality of Dyalog APL/386 can be obtained without sacrificing the ability to run DOS applications.

In addition to Dyalog APL/386 and Xenix, Dyadic can also supply a 386 'Turbo' card. Final pricing had not been established at the time of going to press, but can now be obtained from Dyadic on request.

Dyalog APL/386 is the first of a new range of Dyalog APL implementations for personal computers. Other implementations for the IBM Personal System/2 under Xenix, AIX and OS/2 are also planned.

MicroAPL Ltd

MicroAPL is heavily committed to making the APL language more popular. As part of this drive, MicroAPL is making **DRAMATIC PRICE REDUCTIONS** on APL.68000 for the Apple Macintosh, Atari ST, and Commodore Amiga, with effect from 23rd September 1987. The price for all three implementations has been slashed to just £99.95 including VAT. This move, timed to coincide with the PCW Show, where MicroAPL had two stands, has been made in response to market research, which showed that many home computer users who do not know APL might be tempted to try the language if the price was around £100. The price reduction is part of an overall campaign aimed at non-APL users and involving press reviews, advertisements and exhibition appearances in Europe and the USA, and it follows MicroAPL's announcement in May of free run-time licences for APL.68000 on small machines.

Following Atari's launch of the new Mega ST, the price of high-performance APL has dropped dramatically. A full Mega ST system with a massive 4 Megabytes of RAM, 20 Megabytes of hard disc, a high-quality monochrome monitor, and APL.68000, costs well under £2000, and can even run the Mirage multi-user, multi-tasking operating system. This is the ideal APL machine for education, teaching, research, and even commercial use where IBM compatibility is not essential. Workspace size is nearly 4MB, and there are no object-size restrictions!

For its existing corporate APL customers, MicroAPL is pleased to announce an upgrade to APL.68000 for the IBM PC family, running under 68000 and 68020 coprocessors.

The new release includes very much improved full-screen function and variable editing, faster floating-point operations, an interface to the Halo graphics library, and improved STSC emulation. The 68020 version with 4 MB of memory is proving particularly popular for running VSAPL applications which are too big for APL*PLUS/PC.

For Spectrum and Aurora users, MicroAPL now has available a wide range of extra general-purpose software, including the Omnis database software, LISP, Prolog, and specialised vertical applications.

APL Impetus Ltd

Although APL Impetus Ltd was formed only in July this year, when it was decided that 'Impetus' no longer fitted into Boeing's product strategy, the product has been used by customers for 18 months and the concept goes back to 1976 when TABAPL, its mainframe timesharing predecessor, was first marketed.

When the Impetus development was started in 1984, the challenge was to provide all the functionality of the mainframe product in a machine with only 640K memory. The successful result is based on APL*PLUS/PC, enhanced by 30 or so assembler routines for such crucial activities as formatting, scrolling and panning, and displaying without snow on colour and shaded monitors.

Demonstration diskettes are available for consultants and users who would like to know more about this alternative, APL-based, approach to solving those complex modelling problems which are beyond the scope of spreadsheets.

Cocking and Drury Ltd

Quality control is an important part of software product development, and Cocking and Drury are pleased to be heavily involved in the Beta-testing of new releases of APL*PLUS PC and Statgraphics. As STSC's sole UK distributor, Cocking and Drury have agreed new terms for Academic Site Licences, from which their academic customers will soon benefit. The recent upsurge in APL*PLUS PC sales is seen by the company as an encouraging sign for the future of APL.

On the mainframe side, Cocking and Drury is currently developing two major new Auxiliary Processors for use with their APL*PLUS mainframe product; one is an interface to DB/2 and the other is for access to Oracle.

The course business continues to expand; the company's System Design course has been on offer for some years, and although very favourably received it has not enjoyed great popularity until recently. In the last five months Cocking and Drury have given five in-house System Design courses. Overseas courses are becoming more frequent, and the availability of portable lightweight hardware makes courses outside the London area very attractive.

Although Cocking and Drury are happy to work on short projects, most of their consultancy revenue now comes from large projects; in 1978 about 70% of the revenue came from jobs between £2,000 and £5,000 in size, whereas in 1986 over 30% came from projects exceeding the £100,000 mark. In the same year only 3% came from the £2-5,000 range.

Cocking and Drury's recent hirings include Kevin Ryall and Sylvia Kahl, both of whom have a couple of years APL experience; Kevin on mainframes and micros, and Sylvia mainly on PCs. Christian Jensen also joins as a graduate trainee. Christian has moved from Denmark to continue his APL career here in the UK. Harry Casely-Hayford has come to Cocking and Drury for his technical training year, from North Staffordshire Polytechnic.



APL CONSULTANCY

- * Debugging
- * Support and maintenance
- * Feasibility studies
- * Interfaces to mainframe systems
- * Conversions
- * Major developments

Our highly experienced consultants are capable of tackling any APL problem large or small.

For further details call Valerie Lusmore on 0225 62602

APL People Ltd
17 Barton Street, Bath BA1 1HQ
Tel: 0225 62602



UNLEASH THE POWER OF YOUR IBM PS/2 MODEL 80, COMPAQ 386, OR COMPATIBLE WITH DYALOG APL/386

Now you can develop serious large-scale APL applications on a PC
because Dyalog APL/386 is FAST (many times faster than APL*PLUS/PC)
UNRESTRICTED (no practical limits on workspace and object size)

and gives you

▼
Nested arrays, defined operators, etc.

▼
Multiple concurrent APL sessions with 'hot key' facility

▼
Powerful CGI graphics interface

▼
Concurrent UNIX/DOS support

▼
Multi-user environment

▼
► ► From only £995 ◀ ◀

Dyalog APL/386 runs under Xenix on the IBM PS/2 Model 80, and on a wide range of 386 PC-AT compatible systems including Compaq, Wyse, Zenith, Olivetti, ITT XTRA and TI. It also supports certain 386 'turbo' cards including the Orchid 386 and Intel INboard.

Evaluation kit (including Xenix) available

For further information, contact
Sales Department
Dyadic Systems Limited
Park House, The High Street, Alton,
Hampshire, GU34 1EN, United Kingdom
Telephone: (0420) 87024. Telex: 858811

IBM and IBM Personal System/2 are registered trademarks of International Business Machines Corporation
APL*PLUS is a registered trademark of STSC, Inc.
UNIX is a trademark of AT&T Bell Laboratories
Xenix is a trademark of Microsoft Corporation

The Education Vector

by Norman Thomson

The first APL clear workspace, called CLEANSPEACE, came to life in November 1966, and today in APL2, you can still:

```
)LOAD 1 CLEANSPEACE
```

to which the reply is

```
SAVED 1966-11-27 23:53:59
```

21 years later, as APL comes of age, it is appropriate to look back and view it in the context of the education/technology lag whereby today's development becomes after a lapse of time enshrined in tomorrow's curriculum. 21 years ago APL was an off-beat program harboured within the confines of IBM's Yorktown Research Laboratory. Today, as was demonstrated at the Winchester course in July, APL systems twice as versatile can be carried around on a floppy disc and immediately brought to life on a sundry assortment of computers anywhere in the world.

The Winchester course is adequately reported elsewhere in these pages - as a tutor it stimulated recollections of my own first introduction to APL some 16 years ago, when in order to conduct a course with on-line demonstrations, the entire computing service of a major University had to be suspended during the periods when lectures were taking place. APL itself resided on a disc file about the size of a small dustbin, and the University computer required a system IPL to bring even the single terminal to life. The idea of practical sessions was quite absurd.

As the wheels of technology turn, the phases of development will undoubtedly be repeated as the educational part of the cycle comes round. In particular, one recalls the difficulties inherent in APL's special requirements - keyboards, golfballs, EPROMs and so on - and these or similar hurdles will be surmounted anew as I-APL brings the language into the arena of the home and school micro. Already we see some of the first excitement of APL being relived by the teachers and students for whom it has now become affordable, and this will undoubtedly ensure that these hurdles will be overcome, and that APL will become a feature without which a school or college micro is quite unacceptable.

There is nevertheless a danger that the excitement remains within a closed group of enthusiasts, and it is still as vital as ever for members of the Association to be outward-looking and bring others into the fold. Keep in mind the "Please Tell a Teacher" slogan used recently in a VECTOR insert. A mention of APL, however trivial, can be crucially important, since human nature contrives that on first hearing about a new idea, the

name is mentally filed but nothing is done. It is the second mention from an independent source that triggers action.

At the last Committee meeting Val Lusmore undertook to compile a list of "mentors" spread around the country who would be willing to act as advice support centres for teachers, and perhaps make visits to introduce APL to schools and colleges. Would members who are prepared to have their names added to this list please contact Val at APL People, 17 Barton Street, Bath BA1 1HQ. Act now, while it is on your mind. You determine your own level of commitment, and you could find such contacts highly satisfying.

A final news item comes from the City University, where a 3-hour Lab session on APL is incorporated into the M.Sc. course on Information Engineering. It is an open-ended activity in which there are no formal lectures, but students explore simple examples with the help of an instruction manual. It is good to hear of still more courses which have an APL content.

A Report on the Winchester Summer Course on APL

by Pauline Hawkes

'APL is the language for mathematicians'. 'If you have to communicate with computers, choose the language you use with care'. 'Why go on using steam engines when the electric motor is to hand?' Remarks like these, and being married to a dedicated APL user, made me want to know more. When I learnt that an APL course for teachers was to be run at King Alfred's College, Winchester, I grasped the opportunity gratefully. The course was sponsored by the British APL Association with the intention of helping to spread the use of the language, which is still almost unknown among teachers, even after 20 years, to large numbers of young people in schools, colleges and at home.

This will be made very much easier with the release of I-APL this year. This is a public domain interpreter which should run on a wide variety of small microcomputers of the kind typically found in schools and in the home, such as the BBC, Atari, Spectrum, etc. The only cost to the user will be the price of a disc and some documentation. You will be able to make as many copies as you like. However, there is still the task of convincing teachers.

Few people in schools and colleges have even heard of APL, and the reaction of most of those who have is 'Oh, yes, it is a very mathematical language. Too many symbols for us, we need something more practical that can be used in business studies or science, etc.'.

Well, it is excellent for doing mathematics and this should be encouraged. However, it has a much wider application, and is used particularly for important decision support systems in most of our major companies, where an odd symbol never shows its face. This kind of use should also be brought home to teachers and pupils, but using a rather different approach through practical case studies using supplied software.

Thus it was that I found myself, one of fourteen people from the world of education, in delightful surroundings which I would have loved to explore. However, the intensity of the four-day course, which went on late into the evenings, permitted little respite. It was reminiscent of cramming for finals. Fourteen proved to be a reasonable number in terms of getting to know each other and the level of supervision the course tutors were able to offer. A larger group would certainly have had severe hardware problems. Despite the small number there was an amazing variety of teachers from public and comprehensive schools, a tertiary college, sixth form college, college of higher education, a university and polytechnics. Also there were two people from county education advisory services, one in mathematics and one in computing. Geographically we came from as far afield as Newcastle and Swansea as well as the home counties. Mostly we were mathematicians with only limited computing experience, or computer specialists with limited mathematical training. The few well versed in both disciplines survived the best and astounded the rest by how much could be achieved in such a short space of time with such a powerful tool as APL.



The team of tutors, led by Norman Thomson, consisted of John Durran and Anthony Camacho, supported by his wife Sylvia. The course was primarily aimed at the use of APL in teaching mathematics, although some felt they would have liked more emphasis on uses in teaching other subjects. The presence of guest speakers Paul Barnetson, Matt Taub and Mike Kingston from IBM was very welcome, and it was particularly interesting to hear about applications in business and engineering, such as analysing customer files, costing the construction of a housing development, and the problems of magnetic coatings on tapes and discs. A versatile language indeed! The lecturers were excellent and their enthusiasm infectious. The support they gave each other was admirable: team teaching at its best.

Perhaps some of us were as sceptical as our colleagues when we first arrived. However, after a memorable four days we shall never be quite the same again. At times we felt a bit overwhelmed by the intensity of the course, the power and breadth of a language which can achieve so much so concisely and with such elegance. An important factor for us was to discover that APL has an intellectual attraction, and it is fun. What better recommendation for its use in education? Why struggle with the one-step-at-a-time complexities of BASIC, PASCAL, COBOL etc., which are usually matters of boring, trivial detail, when the power, logic and eloquence of APL is at hand? APL has sparked off a new interest and enthusiasm for computing in those of us who had previously found computing dull.

Lectures and two excellent videos were interspersed with exercise sessions, although the latter proved too short for most of us. We will have more sympathy for our own students in future when they fail to complete their assignments on time. The exercises needed to be more carefully graded. It is difficult to cope with unusual new symbols and concepts while trying to solve fairly hard mathematical problems. We needed a lot of simple problems at first to boost our confidence in assimilating the APL vocabulary and grammar. The overall effect was a little too much detail too fast for most of us to master confidently. It is clear that some of us should have studied more thoroughly the tutorial notes sent to us beforehand; however this had not always been possible. We felt also that it would have been useful to have some discussion about uses in the classroom.

The course achieved its main purpose of giving us a broad feel of APL in a short space of time and encouraging us to go away and learn more. The documentation supplied was excellent and will keep us going for quite some time. In varying degrees we were all converted to APL, and while I doubt if many of us would rush off and immediately start teaching a course ourselves, most of us now feel inclined to encourage the introduction of APL into our own educational environments. One young lady will be able to do so using an old but working IBM 5110, donated by the Decision Analysis Unit at LSE, which she won for her establishment by an eloquent plea on their behalf. The rest of us eagerly look forward to getting our I-APL interpreter. The only real criticism concerned the hardware.

The variety of machines and especially the keyboards, some incorrectly labelled with APL symbols, caused quite a bit of difficulty, while one machine per person would have been ideal.

The advent of GCSE makes this an ideal time to introduce APL into schools. The investigative approach encouraged in these new examinations is ideally suited to the use of APL for experimenting with mathematical ideas and for the ease with which statistical data can be explored. One course member, a computer scientist, was a little worried that APL would encourage bad habits, i.e. programming at the keyboard and the 'try it and see' approach. Mathematicians, on the other hand, would see this as one of the attractions! It is quite simply the best way to do mathematics on a computer, and is a potential motivator in the classroom as well as enabling self-taught high achievers to go a long way. The statistics student can explore in detail real data instead of performing routine calculations on artificially simplified data. A big advantage is that APL has a very low threshold, which should enable even quite young children to do something interesting. And yet it also has a very high ceiling which provides a challenge to the highest intellect.

In conclusion, I would like to express the grateful thanks of the participants to the BAA, those behind the development of I-APL, and above all the course lecturers for all the hard work that went into introducing us to APL in a style and at a price we could afford. I hope that similar courses will be run at other locations around the country, perhaps with the formation of local interest groups. If the new 'freeware' interpreter can be made available in significant numbers, and with the right promotion, I expect that, as with the BASIC expansion, the children will soon leave the teachers far behind. We wish the project every success.

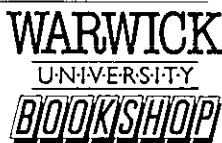
Pauline Hawkes

Swansea Tertiary College

APL BOOKLIST
(In author order)

Title	Price £
Sharp APL Reference Manual, P Berry	10.50
Star Map, P Berry & J Thorstetsen	6.00
APL and Insight, P Berry and G Bartoli	4.50
APL 86 Tutorials, A Camacho	(members £9.30) 12.00
A Source Book in APL, A Falkoff & K Iverson	10.00
 FinnAPL Idiom Library	 11.20
Application Systems in APL, Gibson Levine Metzger	30.00
APL:An Interactive Approach, Gilman & Rose	26.50
Solutions to Algebra, J Iverson	3.00
A Dictionary of APL, K Iverson	2.50
 A Concise Dictionary of APL, K Iverson	 2.00
Algebra: an Algorithmic Treatment, K Iverson	22.50
APL in Exposition, K Iverson	3.00
Applied Mathematics for Programmers, K Iverson	8.00
Elementary Analysis, K Iverson	8.00
 Introduction to APL for Scientists & Engineers, K Iverson	 3.00
Introducing APL to Teachers, K Iverson	3.00
Mathematics and Programming, K Iverson	8.00
APL Toolkit (CIPS APL SIG), R Levine	4.50
Reliable Software Through Composite Design, Myers	15.00
 APL:An Introduction, H Peelle	 POA
APL in Practice, Rose/STSC	40.00
Sharp APL Users Meeting Procs 1982 Vol 2	8.50
APL:Design Handbook for Commercial Systems, A. Smith	13.10
Resistive Circuit Theory, R Spence	20.00
 Whizzbangs Volume II, R Sykes	 14.50
Whizzbangs Volume I, R Sykes	14.50
An APL Notebook, Barrie Wetherill (when available)	1.90
APL Idiom List (Yale University)	2.00
APL 86 Conference Procs, D Ziemann	(members £13.30) 17.30
 APL Business Technology '83 Proceedings	 11.20
APL Quote-quadruple the Early Years	32.00
APL Trivia Cards (per set)	4.50
Special APL 86 Wallet Offer (see Vector 3.2 page 92)	22.50

Please order direct from



Arts Centre, University of Warwick, Coventry CV4 7AL, Tel 0203-523388

Access, American Express and Barclaycard accepted. Order by telephone.

Postage at cost on credit card orders.

APL Product Guide

Compiled by Cathy Dargue

VECTOR's exclusive APL Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We do depend on the alacrity of suppliers to keep us informed about their products so that we can update the Guide for each issue of VECTOR. Any suppliers who are not included in the Guide should contact me to get their free entry - see address below.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage.

The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages. Where no UK distributor has yet been appointed, the vendor should indicate whether this is imminent or whether approaches for representation by existing companies are welcomed.

For convenience to readers, the product list has been divided into the following groups:

- * Complete APL Systems (Hardware & Software)
- * APL Timesharing Services
- * APL Interpreters
- * APL Visual Display Units
- * APL character set printers
- * APL-based packages
- * APL Consultancy
- * APL Training Courses
- * Other services
- * Vendor addresses

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

Note: 'poa' indicates 'price on application'.

All contributions to the APL Product Guide should be sent to:

Cathy Dargue,
60 Downhall Ley,
BUNTINGFORD,
Herts SG9 9TL.
Tel: 0707-325161 ext 2418

COMPLETE APL SYSTEMS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Analogic	The APL Machine	≈60,000+	AP500 array processor, 4 Mb data memory, 80 Mb disk drive.
Dyadic	APL Coprocessor	3,500+	32-bit coprocessor board for IBM PC. NS32000 cpu with FPP, up to 4Mb RAM, 16Mb virtual memory. Software includes Unix V.2, Dyalog APL, graphics support, DOS interface. Provides multi-user Unix/DOS environment.
	IBM 6150	15,000+	Multi-user Dyalog APL system with Fast 32-bit RISC processor, FPP, up to 8Mb RAM, 210Mb Disk, 16 users. Interface to SQL, graphics and APL support for standard IBM peripherals.
	Altos 3068	25,000+	Multi-user Dyalog APL system with MC68020 cpu & MC68881 FPP. Also features a LAN which supports IBM PCs as Dyalog APL terminals.
	Sun 3	15,000+	Multi-user Dyalog APL systems which can be configured as a network of workstations and/or a traditional time-sharing cpu. With its 25MHZ 68020 cpu, the Sun 3/200 is the fastest APL microcomputer on the market.
Gen. Software	Myriade	poa	TI computer APL & APL operating system
Inner Product	IBM PC	2,000-6,000	IBM PCs supplied for turnkey applications
M.B.T.	MBT Series 10	poa	UNIX/68010 based multi-user APL system
	TORCH	poa	68000/Z80 multiprocessor
MetaTechnics	-	poa	Details on application -- IBM PC compatible
MicroAPL	Aurora	20,000+	Multi-user APL computer using 68020 CPU. Std. configuration 2Mb RAM, 16 RS232 ports, 68 Mb hard disc, 720K diskette
	SPECTRUM	7,000+	Expandable multi-user APL computer using Motorola 68000. Std. configuration 1 Mb RAM, 12/36 Mb disc, 12 ports.
	Atari 1040ST	799	1 Mb Mono/Colour System, includes 1 Mb disc drive & mains transformer built into Console.

APL TIMESHARING SERVICES

COMPANY	PRODUCT	PRICES(£)	DETAILS
Mercia	APL*PLUS	poa	STSC's Mainframe Service -- MAILBOX etc.
I.P. Sharp	SHARP APL	poa	International Network application systems and public databases.
Uniware	APL*PLUS	call	STSC's mainframe service

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL Software	Dyalog APL	1,000-8,000	See Dyadic Systems entry
Cocking/Drury	APL*PLUS/PC Rel 7	475	STSC's full featured APL for IBM PC, PC/AT and PS/2. Now with virtual workspace, quad-MF, GSS graphics, Network and VGA support.
	Upgrade 6 to 7	145	
	Upgrade 5 to 7	260	Extension upgrades to release 7.
	Run-time	poa	Closed version of APL*PLUS/PC which prevents user exposure to APL.
	APL*PLUS UNX	poa	STSC's 2nd generation APL for IBM PC/AT, DEC, AT&T and other Unix computers.
	APL*PLUS VMS	poa	Integrated 2nd generation APL for DEC m/cs under VMS
Dyadic	Dyalog APL	795-10,000	2nd gen. APL for UNIX systems, e.g. IBM 6150, Sun, Vax, NCR, HP9000, AT&T, Altos, Apollo, Whitechapel, Sperry, etc.
Gen. Software	APL*MYRIADE	poa	Runs on Texas Instruments TI990 range.

IBM UK	IBM PC APL	poa	Event-handling & APs for full-screen I/O disks, diskettes, asynch. comms.
Inner Product	VIZ::APL	250-350	8-bit Zilog Z-80 CP/M
	APL*PLUS/PC	600	See under Cocking & Drury
M.B.T.	Dyalog APL	poa	See Dyadic Systems entry
	MBTAPL	poa	Enhanced Dyalog APL for MBT hardware.
	VIZ::APL	poa	Customized for TORCH hardware
Mercia	APL*PLUS/PC Ref 6	450	STSC's full-feature APL for IBM PC, and compatibles. No 64K object size limit.
	Upgrades 2,3 & 4-6	225	
	Upgrades 5 to 6	130	
	APL*PLUS/UNIX	poa	Interpreter for UNIX systems: WICAT, CADMUS, CALLAN, FORTUNE 32:16, HP, 9000/500, OLIVETTI 3B2, SUN etc.
	APL*PLUS/MAC	poa	APL for the Macintosh. Big workspaces, big objects, mouse, icons etc
MetaTechnics	APL*PLUS Ref 6	475	Discount on quantity.
MicroAPL	APL.68000	1,000	a Full implementation with component lcs, error trapping etc. for SPECTRUM, HP300, SUN, NCR etc.
	QL/APL (keyword)	87	Full keyword APL for QL with many extra features.
	QL/APL (APL chars)	87	VSAPL compatible APL for QL with many extra features.
	APL.68000 Apple Macintosh	257	
	APL.68000 Commodore Amiga200		Full APL interpreters with support for windows, mouse, graphics etc.
	APL.68000 for Atari ST	170	
	APL.68000 for IBM-PC	995	
	APL*PLUS/PC -- REL 6	450	
Portable	PortAPL	≈195	IBM PC Software
		≈275	Macintosh
		≈2,995	DEC VAX
I.P. Sharp	Sharp APL/PCX	2,575	For IBM XT/AT
		1,000+	For IBM mainframes
	Sharp APL/PC	325	For IBM PC or PC/XT
Uniware	APL*PLUS/PC	495	STSC's full feature APL for IBM PC/XT/AT, Compaq, Olivetti
	Release 5 update	call	Extension upgrade from release 5
	Release 4 update	call	Extension upgrade from release 4
	Release 3 update	call	Extension upgrade from release 3
	Run-Time	call	Closed version of APL*PLUS/PC which prevents user exposure to APL
	APL*PLUS/UNIX	call	STSC's full feature APL for UNIX based computers.
	PortaAPL	280	PORTABLE SOFTWARE's APL for APPLE MACINTOSH.
		2,995	PORTABLE SOFTWARE's APL for the DEC VAX.

APL VISUAL DISPLAY UNITS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Dyadic	Lynwood j300	1,560	Monochrome ANSI 3.64 APL vdu, 15-inch high quality screen, Tek graphics, local macro keys.
	Lynwood j500	2,295	Colour ANSI 3.64 APL vdu, 15-inch high quality screen, Tek graphics, local macro keys.
	IBM 3163	791	Low-cost Monochrome APL vdu. Supports downloaded Dyalog APL font.
	IBM 3164	1,093	Low-cost Colour APL vdu. Supports downloaded Dyalog APL font.
Farnell	Tandberg TDV 2221	995	Ergonomic design APL terminal, 50-19200 baud, 1518if5 anti-reex screen, low profile keyboard
	Tandberg TDV 2271	1,195	Combined APL/ANSI ergonomic terminal as above.
Gen. Software	Mellordata	400	Second-hand Elite 3045A
M.B.T.	various		Contact MBT for details
Meta Technics	IBM EGA compatible	299	Emulates EGA & Hercules, Half Card
MicroAPL	Insight VDT-1	795	Inexpensive APL VDU
	Insight GDT-1	1,450	With monochrome graphics
	Concept201	1,295	APL VDU with 8 page memory
	Concept 201G	1,650	Graphics VDU
Shandell	HDS2010	1,215	ANSI3.64 DEC VT52/100/220 compatible. 15" tilt/swivel screen, low profile keyboard 8 page memory, windows, viewports, 80/132 columns, full overstrike, 2 or 3 comms. ports, 55 PF keys. NVM storage.
	HDS2010G/GX	1495+	As above plus Tektronix 4014. Retrographics VT640/D0540 and Visual 500 compatible. 1024 x 390 or 1024 x 780 resolution.
Tektronix	4114B	13,500+	19" D V S.T.:Graphics: 3120 x 4096 displayable; Intelligent: up to 800K memory; APL keyboard (option 4E)
	4125	21,550+	19" 2D colour graphics; Workstation (1280 x 1024); Intelligent: up to 800K memory; APL keyboard (mod AP)
	4128	26,822+	As 4125 plus 3D wireframe

APL PRINTERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Datatrade	Datasouth DS180+	1,295	180 cps matrix printer with 4K buffer, 9 x 7 dot matrix and APL option.
	Datasouth DS220	1,695	Letter quality; graphics capability, APL option (both available with IBM Twinex or Coax interface).
Dyadic	IBM 4201 Proprinter	poa	100, 200, 40(nlq) cps, matrix printer, with graphics. Supports downloaded Dyalog APL font.
	Toshiba P351	poa	24 pin high-quality matrix printer 100 cps letter quality, 192 cps draft.
Inner Product	Epson FX80	500	Soft char. set, 160 cps, 80 column
	Anadex 9620	1,150	200 cps, 132 col, tractor feed
	Siemens PT88	620	180 cps, 80 col, silent
	TGC Starwriter	1,180	40 cps., letter quality
M.B.T.	Facit 4565	poa	40 cps letter-quality
	Facit 4510/11/12	poa	Matrix printers
MetaTechnics	Quen-data	295	Low-cost APL Daisy-wheel printer

MicroAPL	Datasouth DS180+	1,295	See Datatrade entry
	Philips GP300	1,924	Matrix printer with letter & draft quality and APL
	Qume Letterpro20	549	APL/ASCII Daisy-wheel printer

APL PACKAGES

COMPANY	PRODUCT	PRICES (£)	DETAILS
APL-385	APL-385	50(PC), 125(mf)	including ...
	FSM-385		Screen development
	DRAW-385		Screen design
	DB-385		Relational W.S.
	GEN-385		Miscellaneous Utilities
APL Software Ltd (mainframe)	AFM/AP	11,035	Interprocess Software for VM/CMS & MVS/TSO
	- Keyed Access	2,650	Component File Management System (VSAPL/APL2)
	- Interactive Link	1,325	
	- Mail Exchange	2,650	
	CALL/AP	4,030	Non-APL program execution (VSAPL/APL2)
	APLPRINT	2,205	Output to high speed line printer or 328x devices (VSAPL/APL2)
	ENHANCED FORMAT	2,205	Extends Format operator to full "Quad-FMT" status (VSAPL/APL2)
	ISP	750	Input and Output Stack Processors for manipulating terminal I/O
	OSP	2,205	with facilities for Error Trapping (VSAPL)
	DISPLAY CAPTURE	poa	Allows terminal output to be collected and held for retrieval by an APL function (APL2)
	UCF	poa	User Communication Facility for data transfer between users (APL2)
	RDS	poa	Relation Data Base System
	PANEL	poa	Fullscreen management system
	PFS	poa	Program File System - APL Systems development aid
	IPLS	poa	Project Management System
	REGGPAK	poa	Regression Analysis Package
(microcomputer)	POWERTOOLS	295	Assembler written replacement function for commonly used CPU-consuming APL functions, includes a Forms Processor.
	REGGPAK	poa	Regression Analysis Package
	RDS	990	Relational Database System
Beta-plan	BETA-FONT	poa	Multiple font PC character generator. Dealers required for non-Scandinavian countries.
APL IMPETUS	Impetus	poa	Hierarchical Planning System
Butel	Merlin	5,000	Mainframe APL spreadsheet runs under VM/CMS, TSO, VSPC
	Merlin/PC	poa	Version for APL*PLUS/PC
Cocking/Drury (for VSAPL)	SHAREFILE & enh'ments	30,000	Component files, quad- functions & nested arrays for IBM VSAPL under VM/CMS & MVS/TSO
	SHAREFILE only	15,000	
	ENHANCEMENTS only	17,000	
	COMPILER	30,000	First APL compiler. Available with APL*PLUS enhancements and Sharefile under VM/CMS & MVS/TSO
	FILEPRINT	8,000	Print APL component files
	FILESORT	8,000	Sort APL component files
	FILECONVERT	8,000	Convert non-APL files to APL files
	FILEMANAGER	8,000	Extends APL primitives to database management
	TOOLS + UTILITIES	8,000	APL Software development tools

	DATAPORT	poa	Powerful Information Centre spreadsheet incorporating data exchange between APL and FOCUS, IFPS, SAS, APL/DI, ADRSII, LOTUS123, VISICALC, MULTIPLAN, DIF files
(for APL2)	SHAREFILE/AP	15,000	STSC's sharefile for APL2
	FMT	poa	Full featured FMT for APL2
	WSDOC	5,000	
	FILEMANAGER	8,000	
(microcomputer)	STATGRAPHICS Rel 2.6	645	Integrated Statistics and graphics on IBM PC, AT, PS/2 and compatibles
	Update 2 to 2.6	140	
	Update 1 to 2.6	330	
	APL*PLUS PC Tools	325	Utilities including: RAM disk, full screen data entry, menu input, report generation, file documentor, exception handling and games.
	IRMA Module	120	327x IRMA support.
	Fin & Stat. Library	350	Financial & statistical routines
	SPREADSHEET MGR	195	APL-based spreadsheet for APL*PLUS/PC. Cell arithmetic; transfers to ASCII, LOTUS
	APL Debugger	95	Debugging tool for APL*PLUS PC
	UNITAB	250	Spreadsheet for APL*PLUS PC
E&S	PROTOPAK		Packages for prototyping management information systems - consisting of: PC & mainframe modules
	RMS		Relational databases.
	AMS		Multi-dimensional arrays
	RAMS		Combined RMS & AMS.
	BMS		Dynamic Financial modelling & forecasting
	FMS		Full-screen handler for APL*PLUS/PC. (AP124-based)
	CMS		Communications package.
	SOS	poa	Scheduled ordering and stock control.
FASTCODE	FASTCODE, MONITOR	239	Assembler written monitor for APL applications in APL*PLUS/PC
Gen. Software	PROPS	500+	Spreadsheet system for Product and/or Project Planning.
H.M.W.	INPUT	poa	Matrix manipulation package for data entry & report generation
	PRINTPAK	poa	Block printing for VM/CMS
	VIEWPAK	poa	AP124 Protocol emulator for IBM/PC
Holtech	CASH	3,500-10,000	Accounting package & hotel management system on MicroAPL SPECTRUM & SAGE CPUs.
Inner Product	Viewcom	150	Control Viewdata from APL
	APL/dBASE II	150	Interface APL with dBase II
	APL/dBASE III	150	Interface APL with dBase III
	APL/LOTUS	150	Interface APL with Lotus
	APL/WORDSTAR	150	Interface APL with Wordstar
	APL/MULTIPLAN	150	Interface APL with spreadsheet
	CEMAS	3,500	EEC monetary and agrimonetary analysis.
M.B.T.	RHOMBUS	poa	Integrated Office System
	HASLEMERE	poa	Hotel Accounting System

Mercia	STATGRAPHICS 21	585	Integrated stat. graphic system for PCs.
	Upgrade to Release 21	175	
	EXEC*U*STAT	395	Easy to use Statistics for management.
	APL*PLUS tools		
	- VOL 1	225	IBM PC Utilities:IRMA3270 comms, full screen, RAM Disk report generator
	- VOL 2	125	File documentation, screen editing, Exception handling.
	FINANCIAL AND STATISTICAL LIB.	325	Financial and Statistical analysis
	APL Spreadsheet	195	APL spreadsheet - links to popular spreadsheet software.
	EXECUCALC	4,000	Mainframe Spreadsheet with VisiCalc and Lotus 1-2-3 functionality; requires VSAPL under TSO or VM.
	EXECUPLOT	3,200	Mainframe Graphics display system with VisiPlot functionality; requires VSAPL under TSO or VM and GDDM.
	MICROSPAN	250	Comprehensive APL tutor
	LOGOL	poa	Logistics management system for PC, Forecasting, Inventory Control, Scheduling, Distribution, etc
MetaTechnics	MetaScreen	99	Full-screen handler for APL*PLUS/PC, based on VSAPL AP124
	MetaPack	495	Comprehensive utilities package for APL*PLUS/PC. Includes: MetaScreen, MetaWS, Browse, Toolbox, Numeric Editor.
	APL-IEEE488	99	Controls IEEE488/GPIB Bus from APL*PLUS/PC.
	PLOT/PC	99	2D & 3D Graphics package. Includes interactive diagram Editors.
	Browse	99	Scrolling of DOS files, large APL variables.
	ADAPTA OLS	poa	Production & purchasing scheduling for process manufacturing.
MicroAPL	ADAPTA MSP	poa	Job-shop loading & scheduling for multi-stage production.
	MicroTASK	250	Product development aids
	MicroFILE	250	File utilities and database
	MicroPLOT	250	Graphics for HP plotters etc
	MicroLINK	250	General device communications
	MicroEDIT	250	Full screen APL editor
	MicroFORM	250	Full screen forms design
	MicroSPAN	250	Comprehensive APL tutor
	MicroGRID	poa	Ethernet & other networking
	APLCALC	400	APL spreadsheet system
	MicroPLOT/PC	250	For APL*PLUS/PC product
	MicroSPAN/PC	250	For APL*PLUS/PC product
	PC TOOLS Vol 1	295	
	STATGRAPHICS Rel 1	495	
Parallax	STATGRAPHICS Rel 2	535	
	ExecuCalc	£5,000	Mainframe-based electronic spreadsheet for VM/CMS & MVS/TSO with links to micro products.
I.P. Sharp	ExecuPlot	£5,000	Mainframe-based colour graphics with micro links.
	ACT	poa	Actuarial system
	APS	poa	Financial Modelling
	BOXJENKINS	poa	Forecasting technique
	CONSOL	poa	Financial Consolidation
	COURSE	poa	APL Instruction
	EASY	poa	Econometric Modelling
	FASTNET	poa	Project Management
	GLOBAL LIMITS	poa	Exposure management for banks
	MABRA	poa	Record maintenance/reporting
	MAGIC	poa	Time series analysis/reporting
	MAGICSTORE	poa	N-dimensional database system
	MAILBOX	poa	Electronic Mail

	MICROCOM	poa	Mainframe to micro link
	SAGA	poa	General graphics, most devices
	SIFT	poa	Forecasting system
	SNAP	poa	Project management
	SUPERPLOT	poa	Business graphics
	VIEWPOINT	poa	4GL - Info centre product
	XTABS	poa	Survey Analysis
Sugar Mill	Stat 1	#129.95	Statistical toolbox, menu driven
Uniware (mainframe)	STSC's ENHANCEMENTS	10,715	Quad-functions & nested arrays for IBM VSAPL under VM/CMS and MVS/TSO
	STSC's SHAREFILE	10,715	Component files for IBM VSAPL under VM/CMS and MVS/TSO and for IBM APL2
	PROGRAMMER TOOLS & UTILITIES	5,715	
	FILEPRINT	5,715	
	FILESORT	5,715	
	FILECONVERT	5,715	
	FILEMANAGER (EMMA)	5,715	STSC's database package.
	APL*PLUS COMPILER	21,430	First APL compiler. Complements APL*PLUS enhancements and Sharefile under VM/CMS and MVS/TSO.
	EXECUCALC	3,995	Mainframe spreadsheet compatible with VISICALC and part of LOTUS 1-2-3 under VSAPL (VM or TSO).
(microcomputer)	STATGRAPHICS	725	Statistics & Graphics for PCs.
	STATGRAPHICS FCA	140	An add-on module to STATGRAPHICS: Factorial Correspondence Analysis.
	APL*PLUS/PC TOOLS - VOL1	325	Incl. 327 x IRMA support, RAM disk, full screen data entry, menu input, report generation, games.
	- VOL2	125	Incl. File documentor, screen editor, exception handling.
	SPREADSHEET MNGR	250	APL spreadsheet with built-in ASCII, LOTUS and SYMPHONY interfaces.
	APL*PLUS/PC FIN. & STAT.LIBRARY	350	Collection of financial and statistical utilities.
	POCKET APL	140	Smaller version of APL*PLUS/PC.
	UNIASM	275	Collection of assembler routines for APL*PLUS/PC users.
	UNITAB	240	APL*PLUS/PC spreadsheet-like data entry and validation system.
	The APL DEBUGGER	105	First released APL*PLUS/PC debugger.
	OVERLAYS	250	Fast assembler routines to handle overlays in APL*PLUS/PC.
	R:BRIDGE	380	Interface between APL*PLUS/PC & R BASE 5000.
	DMA	380	A version of EMMA (APL database manager) for APL*PLUS/PC users.
	APL2C	295	Interface between APL*PLUS/PC and DATALIGHT C language.
	ADAPTA/DLS	33,333	Production & purchasing scheduling for process manufacturing.

APL CONSULTANCY

(prices quoted are per day unless otherwise marked)

COMPANY	PRODUCT	PRICES (£)	DETAILS
APL Consultants	Consultancy	poa	Project management financial applications relational databases. Difficult problems solved. Management consultancy. Links to non-APL systems. From consultant level to managing consultant. Documentation a speciality.

APL Software Technology	Consultancy	poa	Technical & business systems, micros, networking & communications a speciality.
Buckland Management Systems	Consultancy	poa	Business and Technical systems in commerce and industry. Designing, programming and implementing applications of O.R and statistics.
Camecho	Consultancy	poa	Specialising in programming & manual writing.
Chapman	Consultancy	150-300	24-hour programmer: APL, C, assembler, graphics; PC, mini, mainframe, network.
Cocking/Drury	Consultancy	120-150	Junior consultant
		140-200	Consultant
		185-300	Senior consultant
		275-400	Managing consultant
Peter Cyriax	Consultancy	100-150	Junior Consultant
		120-200	Consultant
		160-300	Senior Consultant
Delphi	Consultancy	poa	Specialising in management reporting systems and APL on microcomputers.
Dyadic	Consultancy	poa	APL system design, consultancy, programming & training for Dyalog APL, VSAPL, APL*PLUS, IPSA APL etc.
E & S	Consultancy	150-250	System prototyping: all types of information system.
FASTCODE	Consultancy	poa	Specialise in improving performance of APL applications on micros & mainframes.
Gen. Software	Consultancy	100+	
H.M.W.	Consultancy	100-250	System design consultancy, programming.
Inner Product	Consultancy	200	On-site micro-mainframe APL, PC/DOS & Assembler
Lloyd Savage	Consultancy	poa	Decision support, particularly specialising in Sales & Marketing systems.
M.B.T.	Consultancy	poa	
Mercia	Consultancy	poa	APL*PLUS & VSAPL consultancy.
MetaTechnics	Consultancy	poa	Management Information & Production, Engineering, APL - C/Assembler custom programming
MicroAPL	Consultancy	poa	Technical & applications consultancy.
M.T.I.	Consultancy	poa	Specialise in Maintenance and development of existing APL systems
Parallax	Consultancy	£750	Introductory APL, APL for End-user & Advanced Topics in APL
QB On-Line	Consultancy	200	Specialising in Banking, Financial & Planning Systems.
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL
I.P. Sharp	Consultancy	poa	Consultancy & support service world-wide.
Uniware	Consultancy	call	Junior to managing consultancy in APL.

OTHER PRODUCTS

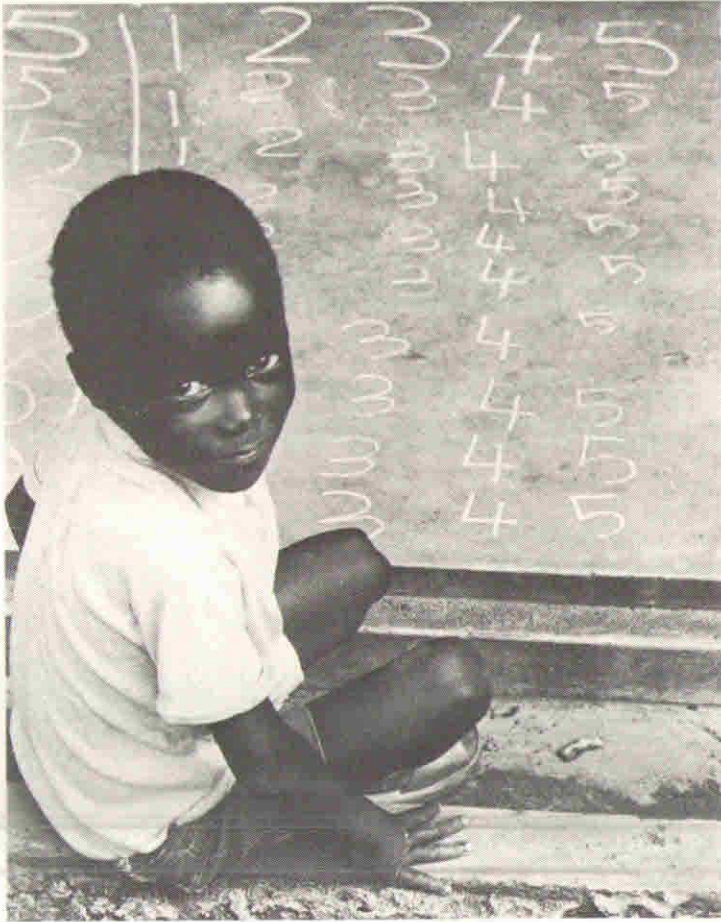
COMPANY	PRODUCT	PRICES (£)	DETAILS
APL People	Employment Agency	poa	Permanent employees placed at all levels. Contractors supplied for short/long-term projects, supervised.
Mercia	ALLCARD	495+	Memory management unit, allowing 952K under DOS - extra 312K APL*PLUS/PC workspace.
	MULTI-APL	195+	Multi-task / Multi-user / Network APL*PLUS/PC with file locking, etc.
I.P. Sharp	Productivity Tools	poa	Utilities for systems, operations, administration & analysts; auxiliary processors, comms software, international network.
	Databases	poa	Financial, aviation, energy and socioeconomic.

VENDOR ADDRESSES

COMPANY	CONTACT	ADDRESS & TELEPHONE No.
Analogic Corporation	Denise Favorat	8 Centennial Drive, Centennial Industrial Park, Peabody, Mass. U.S.A. 01961 Tel: 617-246-0300
APL 385	Adrian Smith	Brook House, Gilling East, York. Tel: 04393-385
APL Consulting	Jill Moss	17 Barton Street, Bath, Avon BA1 1HQ Tel: 0225-62602
APL Impetus Ltd	Cedric Heddle	Rusper, Sandy Lane, Ivy Hatch, SEVENOAKS, Kent TN15 0PD Tel: 0732-885126
APL People	Valerie Lusmore	17 Barton Street, Bath, Avon. Tel: 0225-62602
APL Software Ltd	Philip Goacher	27 Downs Way, Epsom, Surrey KT18 5LU Tel: 03727-21282
		17 Barton Street, Bath, Avon BA1 1HQ Tel: 0225-62602
APL Software Technology	John Hagger	14 Rosewood Avenue, Alveston, Bristol BS12 2PP Tel: 0454415737
	Per Hultin	46 Vicarage Road, South Benfleet, Essex SS7 1PB Tel: 0374550501
Beta-plan APS	Kim Andreasen	Stengrade 75, DK-3000 Helsingør, Denmark. Tel: 45 2 21 48 48
Buckland Management Systems	John Buckland	Westwood, 9 Grove Road, Camberley, Surrey, GU15 2DN Tel: 0276 684327
Butel Technology Ltd.	Mike Munro	Butel House, 3 Great West Rd., London W4 5QJ Tel: 01-995-1433
Anthony Camacho		2 Blenheim Road, St. Albans, Herts AL1 4NR. Tel: St. Albans (0727) 60130
Paul Chapman		18, Trevelyan Road, London, SW17 9LN Tel: 01-767 4254
Cocking & Drury Ltd.	Romilly Cocking	180 Tottenham Court Road, LONDON, W1P 9LE Tel: 01436 9481
		155 Friar Street Reading RG1 1HE Tel: 0734-588835
Datatrade Ltd.	Tony Checkself	38 Billing Road, Northampton, NN1 5DQ Tel: 0604-22289
Delphi Consultation	David Crossley	Church Green House, Stanford-in-the-Vale, Oxon SN7 8LQ. Tel: 03677-384
Dyadic Systems Ltd.	Peter Donnelly	Park House, The High Street, Alton, Hampshire. Tel: 0420-87024
E & S Associates	Frank Evans	19 Homesdale Road, Orpington, Kent BR5 1JS. Tel: 0689-24741
Farnell International Instruments Ltd.	R. Fairbairn	Jubilee House, Sandbeck Way, Wetherby, W. Yorks. Tel: 0937-61961
	Roger Attard	Davenport House, Bowers Way, Harpenden, Herts. Tel: 05827-69071
FASTCODE	Andrew Dickey	P.O. Box 281, Croton-on-Hudson, New York 10520, U.S.A. Tel: (914) 271-3200
General Software Ltd.	M.E. Martin	22 Russell Road, Northolt, Middx. UB5 4QS. Tel: 01-864 9537

H.M.W. Programming Consultants Ltd.	Ken Jackson	142 Feltham Hill Rd, Ashford, Middx. TW15 1HN. Tel: 07842-41232
IBM UK Ltd	Chris Sell	PO Box 32, Alencon Link, Basingstoke, Hants. RG21 1EJ. Tel: 0256-56144
Inner Product Ltd.	Dominic Murphy	Eagle House, 73 Clapham Common Southside, London SW4 9DG. Tel: 01-673 3354
Lloyd Savage Ltd	Philip Johnson	Cambridge House, Oxford Road, Uxbridge. Middx. UB8 2UD. Tel: 0895-59826
Mercia Software Ltd.	Gareth Brentnall	Aston Science Park, Love Lane, Birmingham B7 4BJ. Tel: 021-359 5096
MetaTechnics Systems		John Stenbridge Unit 216, 62 Tritton Road, London, SE21 8DE. Tel: 01-670 7959
MicroAPL Ltd.	Bernadette Leverton	South Bank Technopark, 90 London Road, LONDON SE1 6LN Tel: 01-922 8866
Modern Business Technology Ltd. (MBT)	Michael Branson	P.O. Box 87, Guildford, Surrey GU4 8BB Tel: 0486823956
M.T.I.	Ray Cannon	7 Pine Wood, Sunbury-on-Thames, Middx. TW16 6SH Tel: 09327 80848
Parallax Systems Inc.	Kevin Weaver	60 West 9th Street, New York, New York 10011, U.S.A. Tel: 212-475-4001
Peter Cyriax Systems	Peter Cyriax	213 Goldhurst Terrace, London NW6 3ER Tel: 01-624 7013 (Answerphone) 0880-377963 (Mobile)
Portable Software	Richard Smith	60 Aberdeen Ave, Cambridge, Mass. U.S.A. 02138. Tel: 617-547-2918
QB On-Line Systems	Philip Bulmer	5 Surrey House, Portsmouth Rd Camberley, Surrey, GU15 1LB. Tel: 0276-20789
The Rochester Group	Robert Pullman	164 Pinnacle Rd., Rochester NY 14620 Tel: 716-461-3169
Shandell Systems Ltd.	Maurice Shanahan	12 High Street, Chalfont St. Giles, Bucks HP8 4QA. Tel: 02407-2027
I.P.S.A. Ltd.	David Weatherby	10 Dean Farrar Street, London SW1. Tel: 01-222 7033
Sugar Mill Software Corp.	Lawrence H. Nitz	1180 Kika Place, Kailua, Hawaii 96734 Tel: (808) 261-7536
Tektronix UK Ltd.	Paul Morgan	Fourth Avenue, Globe Park, Marlow, Bucks SL7 1YD. Tel: 06284-6000
Uniware	Eric Lescasse	15 Rue Erlanger, 75016 Paris, France Tel: (1) 45-27-20-61 Telex: 648348F UNIWARE

With apologies to the 'Save the Children Fund'



He will never forget you
for showing him 10 5p15

Book Review: APL - Advanced Techniques and Utilities

by Gary A Bergquist

reviewed by Dick Bowman

Chapter Titles: 1 Limbering Up; 2 Branching and Looping; 3 Computer Efficiency; 4 Positioning Character Data; 5 Sorting and Searching; 6 Selecting; 7 Frequency Counts, Accumulations and Cross-tabulations; 8 Writing User-friendly Interactive Functions; 9 Manipulating Dates; 10 Writing Reports; 11 System Development Procedure; 12 Programming Standards; 13 Workspace Design and Documentation; 14 File Design and Utilities; 15 Boolean Techniques; 16 Irregular Arrays; 17 Curve Fitting; 18 Financial Utilities; 19 Exception Handling.

I really want to feel positive about this book; the world's been waiting for one with this title, but the contents make me feel uneasy. In my perception APL needs two educational texts which it doesn't yet have; the first is an introductory text which is as contemporary as those of Gilman/Rose and Pakin/Polivka were when they were published, the second is the book that takes over where the first stops and solves the problem so many newcomers express of 'now I know what all the symbols do, how do I make something useful?'.

Many have tried, and in essence I perceive the text at hand as being an attempt to supply my second need; it functions at a level closer to the code than Smith's 'Design Handbook' and is less philosophical than the Gibson/Metzger/Levine IPSA82 tome. It's not a case of choose one from three, they complement one another.

I really do like the way it starts, with a set of exercises for the reader; it conjures up a vision of the future in which we install APL terminals in all bookshops and only permit books to be purchased by those mentally equipped to read them. At least it establishes whether the reader and Mr Bergquist are on the same mental plane.

Let's be absolutely positive about this; anyone who reads this book, attempts the exercises therein, and follows the advice given about development procedures is going to produce APL solutions which are a whole league in advance of those produced by someone who doesn't. So why am I so uneasy about an unqualified recommendation?

At a general level I continually found myself thinking that the author was addressing questions which the APLer fresh from an introductory course should have formulated answers to already. That these questions should either be the stuff of an introductory course, or else that the solutions should form part of well-stocked public libraries. The question I find hard to generate an answer to myself being why it's so difficult to inculcate a instinctive urge to go and look in the public libraries on the part of the newcomer? Perhaps to be as fair as I can, a novice reader of Mr Bergquist's book would

benefit from the insight being given into developing this sort of utility - maybe the book is more subtle than me (admittedly this isn't too difficult).

More specifically, two chapters make me distinctly uneasy; Chapter 8 on 'user-friendly interactive functions' and Chapter 16 on 'irregular arrays'.

The first is based exclusively around stop-start dialogues of the 2741 genre; and I don't see where this is a major technique any more. Can't we agree on an approach to dialogue amongst ourselves so that a system gets developed in two stages... a) as an open system in the classic 'functions take data from their arguments and return results' mould; which serves to get the processing running, before... b) we wrap these proven functions up in utility dialogue for current hardware like IBM327x's and PC's. Agreed that the IBM327x is one of the lousiest man-machine interfaces yet developed; in the mainframe world everybody else uses one and if we want our APL applications taken seriously we really ought to try and make them have the same sort of features as are demanded of other applications. I haven't yet seen a really good set of utilities either for the 327x or for the PC using STSC's `□KITCHENSINK` approach - both of these must surely be overdue.

As for the second - my personal feeling is that old-generation APL is old-hat and that making your oldAPL imitate newAPL isn't a relevant skill for 90% of APLers. Either you have generalised arrays (in which case use them), or you don't (in which case live with what you have). Given the historic progress in hardware power I would anticipate desktop nesting becoming the norm in a couple of years for all but the most underprivileged users. Agreed there are problems, but I really would like to begin seeing books about APL which treat nested arrays as something more fundamental than a trip to the leper colony. Do it right from the very first day. Again, to be fair to Mr Bergquist - his excursions into APL2/NARS/IPSA territory appear to be thorough, he favours no single participant.

So there you have it, to me this seems to be a somewhat expensive book for its contents; on the other hand I would expect most of it to find a ready market if the word 'Advanced' is sufficiently discounted. By another token - if you find a function in it that saves the time of writing that function yourself you end up ahead.

Available from:

Zark Incorporated
53 Shenipsit Street
Vernon
Connecticut 06066

USA Price: 44.95 USD
(plus 15.00 USD for optional APL*PLUS/PC-compatible functions diskette).
Paperbound, 488 pages, 200 + utility functions described/listed/on disk.

Hardware Review: the NEC V20 Processor

by Adrian Smith

Background

The NEC V20 is a straight swop for the standard 8086 CPU chip that comes with your PC. It costs around £18.00, and is claimed to speed up applications by an average of 10%. In particular it improves screen-access times, and anything involving graphics should go up to 50% faster.

Having shelled out my £18 to the imaginatively titled Ctrl-Alt-Deli (waits apprehensively for the machine to re-boot!!) I fired off my usual set of APL benchmarks. One side issue to note is the drop in speed on using APL-V.6 as opposed to APL-V.5 ... for interest I also timed some simple array operations on both versions; you may draw your own conclusions!

Benchmark Results

Firstly my standard collection from VECTOR 4.1. As previously noted, the simple 1000 looped FN 1000 seems to be the best indicator of overall speed:

APL Expression	Type	Vn.6	Vn.5	V20(5)
T←DTS ⋄ +/V+V ⋄ DD DTS-T A INT		390	380	350
T←DTS ⋄ +/V[C] ⋄ DD DTS-T A INT		60	60	50
T←DTS ⋄ V+.xV ⋄ DD DTS-T A RL		490	440	380
T←DTS ⋄ +/V*2 ⋄ DD DTS-T A RL		4220	4120	3680
T←DTS ⋄ X←+/M ⋄ DD DTS-T A RL		710	720	660
T←DTS ⋄ V+.x,M ⋄ DD DTS-T A RL		2360	2360	2140
T←DTS ⋄ +/S='T' ⋄ DD DTS-T A CH ...		4010	3790	3630
T←DTS ⋄ 'FAT'←S ⋄ DD DTS-T A CH ...		60	60	60
T←DTS ⋄ X←S←'FAT' ⋄ DD DTS-T A CH ...		5710	2860	2520
T←DTS ⋄ X←B×B^~B ⋄ DD DTS-T A BL....		5710	4950	4840
T←DTS ⋄ ^/B ⋄ DD DTS-T A BL....		160	140	110
T←DTS ⋄ X←Y\B ⋄ DD DTS-T A BL....		320	280	220
T←DTS ⋄ FN 10 ⋄ DD DTS-T A FN....		550	390	330
T←DTS ⋄ FN 100 ⋄ DD DTS-T A FN....		4080	3300	3100
T←DTS ⋄ FN 1000 ⋄ DD DTS-T A FN....		47.7s	34.6s	30.3s

... and then the timings for some simple array operations to illustrate the precipice at the 32K boundary:

+ / S X ' T ' :

pS	V.5	V.6
--	---	---
1000	380	440
2000	770	820
5000	1920	1970
10000	3790	4010
20000	7630	7910
30000	11370	11860
40000	---	22.9s
50000	---	27.4s
60000	---	34.4s
100000	---	WS FULL IN 195K

+ / V + V :

pV	V.5	V.6
--	---	---
1000	380	440
2000	730	770
5000	5170	1920
10000	9610	9670
15000	19270	18460
20000	11370	11860
25000	24500	22900
30000	28800	27400
40000	---	45900

DWA 300K APPROX

Conclusion

The V-20 is good value for money in price-performance terms when compared with any of the other 'go-faster' boards, all of which come in well above £500. I could claim to see the difference in $\square$ WPUT and graphics tests, but the improvement is otherwise too marginal to show up in day to day use.

NEC V20 is available from:

Ctrl-Alt-Deli, 44 Brownbaker Court, Milton Keynes.
Cost: £20 approx.

Dyalog APL on the IBM 6150

reviewed by Adrian Smith

Acknowledgements

I must first of all thank Dyadic Systems Ltd for the loan of their APL, and IBM UK Ltd for the loan of a 6150 on which to run it. This machine also came with a copy of the Oracle database to allow me to try out the interface to SQL for data query and update.

I would also like to thank Sun Microsystems for setting up a trial of Dyalog APL on a Sun 3/140 at their Stockport office.

Introduction

This was a system trial to which I had been very much looking forward. I well remember enjoying Dyalog's 'Mexican Snacks' at APL86, and thinking then that their APL had got so many things right, and that the 6150 was a superbly crafted machine. I had been interested in SQL, and particularly in the Oracle implementation of it, for some time; I had also been itching to get my hands on a genuine Unix box! Consequently it was with a real sense of anticipation that I wheeled in the considerable bulk of a (slightly used-looking) IBM 6150 and powered up.

Unix and the 6150

I had known for some time that Unix was regarded within the company as a very severe barrier to the acceptance of any system. In short, if it ran under VMS we would consider it; if it ran under Unix we would consider all the alternatives first! This impression was strongly contradicted by many of my university contacts, and indeed by the occasional DEC representative when taken quietly to one side.

All I can really report is that the basic setting up was straightforward, but that I did have some help from a Unix buff who made it all look very easy. I found the DOS shell quite helpful to start with, but soon dispensed with it in favour of the tiny subset of Unix commands I needed to get by.

I wish I had a really clear crystal ball on this one; judging from the current literature many other people are in the same position. There are lots of super products under Unix, but no-one want to be first in with an operating system which may never really escape from the bounds of academia, and is known to give experts the kind of freedom that DP departments dread!

What is true without a shadow of a doubt is that the fortunes of Dyalog are currently hitched firmly to the Unix star. They will prosper in universities and in companies who have taken on board the 'workstation' concept. In VMS shops like ours, it will have a long hard slog to gain entry, let alone acceptance in the mainstream of application development.

Dyalog: Documentation and Installation

I really ought to find a live user of Dyalog to confirm my feelings on this. The manuals look and feel really excellent; they also come with the neatest set of keyboard stickers I have seen, and Dyalog have thoughtfully included a key-puller so that you get the little blighters on straight! What bothers me is that I had the very devil of a job finding my way round the documentation. There are just too many thumb indexed separators, and the sections may have made sense to the system author; they certainly did not make intuitive sense to me!

In short this has the look and feel of a manual which has grown up with the APL, and has had new features slotted in rather ad hoc. It really would benefit from a proper restructuring with far fewer sections and better organisation within them.

Installation was superb. You simply bung in the disks, type Install and follow instructions! I only wish more PC packages were as easy.

Dyalog - the Environment

This seemed to me to fall roughly midway between the typical mainframe session manager, and the environment on APL*PLUS/PC. I had always to keep reminding myself that I was running on a terminal, and that although this machine looked like an overgrown PC/AT it was actually a fully fledged minicomputer, capable of supporting another 7 users like me.

It compared very well with (say) the TSO session manager, and the sort of 'cut and paste' on prior lines was helpful and easy to use. I suspect it also compared well with the typical Vax environment, although my contact with Vax's has to date been rather limited. One significant beef was the use of the Unix 'vi' editor as the full-screen editor for APL code. I'm afraid I quickly gave up on this intractable beast in favour of dear old del. Again it could just be my total lack of Unix exposure, but I found this one of the worst APL editors I have ever met!

Incidentally, the Del-editor had one very handy feature: simply typing `VFNAME` gave you an immediate display of the function. Please could some of the other purveyors of APL take note!

The contrast with APL*PLUS/PC was much less in Dyalog's favour here. I suppose it was inevitable that the PC, with its memory-mapped screen and instant response, would win on feel. I would have to say that users moving up to this much more advanced machine would face a considerable reduction in the feel and power of the user-interface.

Dyalog - the APL

As I said at the beginning, there are many, many good things about this APL. It is close to being a complete implementation of the APL2 standard, complete with defined operators and most of the enhanced functionality. In a week of fairly solid testing, I crashed it once on an APL system error ... the fault was clearly mine in inventing a ridiculous piece of code!

Rather than dwelling on all the good things, let me just air a few reservations; I'm sure Dyalog (or maybe some of their users) will take me to task on these ... but here goes!

- The error-trapping behaviour seemed much too complex for its own good. I assume this comes from IPSA originally, but whatever the source, you can tie yourself in some horrendous knots very easily! Please can we have something simple and useable like `LEX` and `ERROR`, rather than a heap of obscure `FNS`!
- I felt that there were often too many alternative ways of getting to the same end result. For example you can `CR`, `VCR`, and `NR` (nested representation) functions. Much of the original beauty of APL lay in the orthogonality of the primitive functions; Dyalog have very much taken the `KITCHENSINK` approach to system functions, which leads to functional overlap, confused users, and an overloaded manual!
- the same is true of external variables. These are a wonderful concept, but they exist alongside a conventional (and very effective) component file system. In my view, they only serve to clutter the documentation, lock users firmly into the Dyalog environment, and add an unnecessary extra choice into the system design process. Sorry!
- the final one relates to the 'composition' of any dyadic operation with assignment, to give compound functions such as 'increment'.

```

A←0
A←+1
A
1

```

In my looper benchmark, I had the classic line of code:

```
Lp:→(N<CT+CT+1)+End
```

Naturally, I thought I would just improve this a bit by incrementing CT, rather than adding then assigning:

```
Lp:→(N<CT++1)↑End
```

... would anyone like to tell me why this doesn't work? Basically what happens is that this (very C-like) expression actually behaves in a fairly APL-like way. The counter is indeed incremented, but the result of the '++' function is the right argument, i.e. it always returns 1. Oh dear!!

Enough nitpicking! To wrap up this section, I liked this APL very much, but felt that it was still being treated rather as a private research vehicle by its authors. As a result it has lots of interesting and exciting ideas (which you are unlikely to come across elsewhere), but because these ideas have never faced the hurdle of an international standards body there are some things in there which are distinctly screwball. Caveat emptor, as they say!

Graphics

I was provided with an IBM 6154 monitor, which showed the supplied graphics demos off to good advantage. The speed was acceptable (say 10 to 20sec for a reasonably complex chart), and the imaging quality admirable. The APL code of the demo functions was clear and readable, but I'm afraid I did not have the time to explore this side of the system as fully as I would have liked.

Linking with SQL

This was really good! The examples were clear and easy to follow, and the overall efficiency rivalled that of the native Oracle products. In fact I suspect that with a bit of thought you could write a full-screen update in APL which would knock spots of SQL*FORMS!

Here are a few examples, to illustrate how you set up a simple query, and how the data is structured in the workspace. Please note that I was using the (default) matrix representation. If you can accept you data as nested vectors, everything runs about twice as fast, and the overhead on the array sizes is minimal.

Dyalog APL Session Manager

Mon Apr 13 08:55:52 1987

./APL.ACTIVE.WS saved Fri Apr 10 16:31:35 1987

)LOAD ACDSQL

./ACDSQL saved Mon Apr 13 08:52:54 1987

VSQV

▽ R←SQ QRY;RC;DATA;HD

[1] A Run SQL Query and return formatted result

[2] R←'**** NO JOY FROM QUERY ****'

[3] RC←PREP'XX'QRY A Get ready

[4] RC←OPEN'XX' A Set Cursor

[5] RC DATA←FETCH'XX' ⚡ →(0=1+RC)⚡Err

[6] A

[7] RC HD←DESC'XX' ⚡ HD←HD[1;] A Pick up titles ...

[8] HD←HD,[0.5](p''HD)p''-'' A ... and rule off

[9] DATA←HD;DATA A Tag on to data

[10] R←(⚡DATA)⚡'= ' ⚡ →0 A Finish off

[11] Err←R←R,' RC = ',⚡RC

▽

QQ

SELECT EMPNO,RDESC,DAYS FROM abs,rs

WHERE abs.RCODE = rs.RCODE

AND EMPNO = 7839

AND DAYS > 10

SQ QQ

EMPNO RDESC DAYS

7839 Bad Back 11

7839 Bad Back 12

7839 Bad Back 11

7839 Flu 11

7839 Flu 12

7839 Flu 12

7839 Wife Ill 12

7839 Wife Ill 11

7839 Wife Ill 12

=====

```

OPEN 'XX'
0 0 0 0 0

```

```

RES*FETCH 'XX'

```

```

pRES

```

```

2

```

```

RES
0 0 0 0 0 10 ACCOUNTING NEW YORK
                20 RESEARCH DALLAS
                30 SALES CHICAGO
                40 OPERATIONS BOSTON

```

```

RC DATA*RES

```

```

pDATA

```

```

4 3

```

DISPLAY DATA

10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

```

+/DATA[;1]

```

```

100

```

```

DATA[;2]

```

```

ACCOUNTING RESEARCH SALES OPERATIONS

```

```

+DATA[;2]

```

```

ACCOUNTING

```

```

RESEARCH

```

```

SALES

```

```

OPERATIONS

```

```

p+DATA[;2]

```

```

4 10

```

```

p+DATA[;2]

```

```

41

```

Timings, and Impressions of the Sun 3/140

No, I'm not going to go through that benchmark table again! However it did turn over some interesting quirks in the interpreter, e.g.

```
'FAT'ES      ... 15 sec for 100 executions
'FAT CAT'ES   ... 14 sec for 100 executions!
```

The times that matter are for simple function execution, which gave the results:

```
PC/AT :      13sec (APL*PLUS/PC)
../386:       7sec
6150  :       6sec (Dyalog APL)
Sun   :       3.6s
```

In other words, as a single-user system the (old) 6150 is roughly comparable with a Compaq/386. The Sun is about half as fast again. Running these against a heavy second session (a 40 second SQL query was set running) increased the clock time by almost exactly 50% across the board.

The Sun was in some senses a disappointment. You can do all sorts of visually exciting things (e.g. spinning a globe in one window while APL runs in another); the snag is that APL totally ignores all the mice, icons, pull-downs and so on. Essentially it gives you a simple VT220 session in a 24 by 80 window; after seeing MicroAPL's Mac interface, I found this a real waste of opportunity.

It does, however, go like a bat out of hell!!

Summary

I liked this APL very much indeed. I liked the 6150 very much indeed. However I must add that there are some big question marks hanging over the future of Unix (in the commercial sense), and that Dyalog seem to me to have been too willing to throw every available good idea into the same pot.

A super research vehicle, and ideal in an academic environment, but I would have a hard time getting one into the company. I hope for Dyalog's sake that this is not generally true.



SimCorp Ltd.

CONSULTANTS to £25,000 pa.
ANALYST PROGRAMMERS from £12,000 pa.

SimCorp Limited is a newly established UK subsidiary of a Danish Consultancy and Software House. SimCorp A/S was formed in 1971 and is one of Europe's largest APL specialists with over 70 employees providing a range of services in consultancy and software development to Banks, Building Societies, Insurance Companies and Industrial Clients in most of Western Europe. Our rapidly growing workload in the UK means that we need to recruit additional staff at all technical levels to augment our services in:

consultancy for mainframe, mini and micro-computer systems,

system studies, systems analysis and design,

project management,

programming, implementation, training etc.,

You should have self confidence, initiative, the ability to work productively with minimal supervision, and general experience in DP and practical experience in solving other people's computer problems; you must be prepared to work at our Client's premises from time to time.

Experience of **APL** and/or **Treasury Operations** and education to Degree Level would be an advantage, however we are also concerned to find the right personalities to compliment our team and we will provide training as required.

In the first instance, please send your CV to, or obtain an application form from:

Miss S Parsons, SimCorp Limited,
120, Guildford Street, Chertsey, Surrey KT16 9AH.
Telephone: Chertsey (09328) 67311

RECENT MEETINGS

This section of VECTOR is intended to document the seminars delivered at recent meetings of the Association, particularly for the benefit of those members based away from London who often find it hard to attend. It also covers other selected events which are likely to be of interest to the wider APL community.

We are dependant on the willingness of speakers to provide us with a written version of their talk, and we would remind them that "a picture is worth a thousand words". Copies of slides and transparencies will enhance their talk.

The Activities Officer (see inside back cover) will respond enthusiastically to offers from individuals who wish to contribute seminars and supporting papers.

Windows and Pop-up Menus in Application Design

by Adrian Smith

Background

This paper was presented to the BAA in November 1987. An expanded (updated) version was also given at APL88 in Sydney. For a more complete writeup (including listings of all the windows functions) please see the conference proceedings.

Windows and pop-ups are rapidly becoming de rigeur in the design of PC applications. Unfortunately the majority of APLers (myself included) have come to the PC with a long history of mainframe design (typically AP124/6 based). The initial temptation is simply to transfer this experience to the PC, using some kind of AP126 'look-alike' as the basis for update and menu selection.

Of course such systems can be made to work; the problem is that they make almost no use of the PC's real strengths, and typically they run rather slowly in comparison with a mainframe version of the same thing.

In order to better understand the potential of window-based systems I started work on a set of window utilities (written in APL*PLUS/PC) with no real idea of the final specification these would need. Of course my limited experience of using WIMPish applications gave me a starting point, but in general I only added functions/facilities as they became necessary to particular applications. The notes and examples which follow are a guided tour of the genesis of these routines.

In the Beginning ...

This all started with a prototype enquiry system through which shop-floor supervision could query a database of plant information (stocks, ingredient usages and so on). The brief was for something "so simple even the cat could use it"; ideally data could be accessed with little other than a 'joystick' interface supporting 'up,down,left,right,fire' commands. The initial design evolved towards a set of screens, each showing a simplified map of part of the plant.

To enquire about any part of the plant the user simply pointed to it with the cursor and hit <ENTER>. An appropriate query would then be run, and the answer would 'pop up' on top of the diagram.

To obtain more detail, one could use <PgDn> to 'explode' the diagram down to the next level of detail, and so on

Taking only the windows aspect of this, the basic requirements were for a utility (winHput say) which would take as arguments the required position (row,col) and a character matrix (no bigger than the screen). It would then frame it up, set a sensible background colour and overlay it on whatever lay underneath. A corresponding function (winHundo say) would restore things as they were:

```
CUPOS WINAPUT CHARMAT    ... output data at cursor
WINAUNDO                 ... take it away
```

This simple spec came close to covering all the requirements of the first prototype system. However two enhancements which soon proved essential were:

- some check on the requested position. If it turned out to be too low down, or too far right, the window could be moved to fit on the screen.
- up/down scrolling of long arrays. This is handled in the obvious way with the cursor arrows, PgUp, PgDn, Home and End. <Enter> behaves like PgDn.

I have yet to tackle left-right scrolling of wide data, it is a straightforward extension (probably using Tab and Back-tab); so far I haven't needed it!

One further enhancement completed this phase of the evolution. Once you allow things to scroll you find you need to freeze the heading line(s) at the top. This led to the syntax:

```
(ROW,COL,FREEZE) WINAPUT MAT
```

... which gave me all I needed to create a full prototype of the proposed information system.

Phase-2: Pop-up Data Entry

Those of you with really long memories may just be able to recall the days before full-screen, when we wrote our input functions using quote-quad, and had little utilities like:

```
DATA+VALIN 'Please enter your age:- '
+{ASK 'Do you really want to quit? ')+EXIT
```

I suddenly realised that I had been going through all sorts of hoops to get this effect in full-screen systems, when all I wanted was a little input window for old-style teletype mode entry:

```
2 3 WINAOPEN 8 34      .... OPEN A WINDOW 8 BY 34 AT POSN 2,3
NAME+BAREIN 'Who are you? '
WINAUNDO              .... GET RID OF IT AGAIN
```

It is strange how professional this looks in comparison with the same thing run from the normal APL session! The change of background colour and the smart frame make all the difference.

The windows happily stack up and unpick themselves with no action at all from the programmer. Rather than specifying the colour of each window, I set up a default cycle of 12 'aesthetically pleasing' foregrounds and backgrounds to be used in turn. So far no-one has complained, and it does speed things up if you don't have to worry about the colour when you set up the window.

Phase-3: The Expert System Model

Rule-induction is an aspect of expert-system development which looks particularly promising from an APL angle. Accordingly two of us set out to explore the possibilities of developing rules efficiently from sets of examples, and I used this development to complete my collection of windows utilities (at least for the moment).

The main workhorse function of this system is winHpick, which takes a character matrix (any number of rows) and sets up a slider bar which the user can move with the arrow keys. As before, PgUp, PgDn, Home and End will jump around, and in addition there is the option of hitting the initial letter of the option for an immediate selection. <Esc> quits and returns zero, <Enter> returns the index of the selected row.

```
POS+2 3 WINΔPICK MAT      .... WINDOW AT ROW+2, COL+3
```

As before, you often need to freeze a couple of lines of heading (which the scroll bar is excluded from):

```
POS+5 5 2 WINΔPICK HD,[1] MAT ... lock top 2 lines
```

Typically a 'pick' window should vanish automatically as soon as a selection is made. If you want it to hang around (as in the example above) until you winHundo it, you need another flag:

```
POS+5 5 0 1 WINΔPICK MAT      ... leave it!
```

Finally, it is very handy to pass a default selection (e.g. to select the disk drive to run on you would pass the current drive as default):

```
ΔDRV←(2 3 0 0 ,ΔDRV) WINΔPICK ΔDRIVELIST
```

In practice you would probably check for a zero result (Escape) before re-assigning the variable!

Of course when you have a whole stack of windows it is very pretty to loop round `WINAUNDO` to take them off one by one; it is much faster to restore the whole initial 'backcloth' screen in one shot, hence `WINACLOSE` which kills all outstanding windows in one go.

Summary of Windows Routines

The basic application requirements have so far been met by the following functions:

<code>WINAPUT</code>	... data to screen
<code>WINAPICK</code>	... from an array
<code>WINAOPEN</code>	... an input window
<code>WINAUNDO</code>	... the latest window
<code>WINACLOSE</code>	... all windows

A certain amount of 'intelligence' is built in to all the routines, for example: they will all run with no left argument (default position); they choose the colour cycle depending on whether you are on colour or mono; they treat repeated 'puts' to the same spot as a simple refresh rather than a request to stack another window. Similarly you can 'pull' a window to the top of the heap just by refreshing its contents.

Rather to my surprise, all this runs very adequately fast when coded in plain old APL. Scroll bars keep up with the keyboard repeat (unless you are pushing down a very large amount of data), and the pop-ups really do! I had visions of coding the critical bits in 'C', but as long as you have a tolerably fast PC you will have no problems with execution speed.

Summary of Benefits to the Developer

Simplicity and economy. I feel much more comfortable without the excess baggage of a heap of FSM functions and associated formats. The whole of the windows library is only just over 100 lines of APL (not including comments) and uses very little working storage while running (say 10K on average).

Portability of ideas. As more and more applications take on board the GEM-type interfaces, users get to expect this style from all their systems. Already APL.68000 has good hooks into the window manager on the Atari and Amiga; we should soon be able to make limited use of windows on GDDM Version 2.

Implicit validation. With a 'pick' window, you know that all you can possibly get back is zero, or a number in the right range. All that "Invalid day ... please re-enter" code can be totally eliminated!

More flexible update. Lotus-123 is quite happy to let you put more in a cell than will fit the column width. Particularly with nested arrays it is a nonsense to fix a maximum value for (say) a name at the width of a screen column. With a window-based editing system the user can point to the row to be changed, open an input window, and enter a name of any length in 'teletype' mode. The fact that only the first (say 10) characters show on the screen doesn't matter at all.

Fun. Experience to date (especially with the shop-floor system) suggests that this is not a trivial point! A system which is colourful, friendly and fast will get used; one which is dull, boring and slow (which FSM tend to be on the PC) will not.

Footnote

At the end of the talk I handed out the best part of a box of floppies, containing the windows routines. To my great surprise, the majority of these homed within the week, and I think the last straggler made it home at the next BAA meeting!

Many of the functions returned with suggestions and improvements. Thank you all for returning the originals so promptly; I will get an updated set of routines on the software exchange as soon as I am sure that the functions have fully stabilised!

Please select from ...	
A: left-hand floppy disk	query
B: right-hand	
»C: Current directory of hard disk	eries
M: \MELANG directory	ry library
<Enter> to select; <Esc> to quit	
DOC	... To list a query on paper
BACKUP	... make a DOS file to backup all queries
DROP	... remove a query from file
»DRIVE	... Select disk drive to run from
TALK	... sign on to VAX as a terminal
RAW	... use raw SQL input to VAX
LOGOUT	... sign off VAX session
Menu	... to exit to application selection
<Enter> to select; <Esc> to quit	

SQL Query Editor

Please Select from	
=====	
Drive C: selected	te selected query
<Enter> to continue	ary of all queries
LIBEDIT	... update / sort query library
LIST	... list any query
DOC	... To list a query on paper
BACKUP	... make a DOS file to backup all queries
DROP	... remove a query from file
»DRIVE	... Select disk drive to run from
TALK	... sign on to VAX as a terminal
RAW	... use raw SQL input to VAX
LOGOUT	... sign off VAX session
Menu	... to exit to application selection
<Enter> to select; <Esc> to quit	

SQL Query Editor

Please Select from	
=====	
»EDIT	... update selected query
LIB	... library of all queries
LIBEDIT	... update / sort query library
LIST	... list any query
DOC	... To list a query on paper
BACKUP	... make a DOS file to backup all queries
DROP	... remove a query from file
DRIVE	... Select disk drive to run from
TALK	... sign on to VAX as a terminal
RAW	... use raw SQL input to VAX
LOGOUT	... sign off VAX session
Menu	... to exit to application selection
<Enter> to select; <Esc> to quit	

SQL Query Editor

PLANDAY :day's plans	
QD :Query time and date	
REFC1 :rfc1	
REFC2 :rfc2	uery
REFC3 :rfc3	
»REFINERS:all refiners info	eries
REFS10 :rfs10	ry library
REFS11 :rfs11	
REFS12 :rfs12	
REFS8 :rfs8	n paper
REFS9 :rfs9	o backup all queries
REGR1 :regring for 35 & 37	om file
REGR2 :regrind for 38	
REGR3 :regrind for 36 & 84	to run from
REGRINDS:all regrind tank info	a terminal
REGRSUM :sum of all rg types	to VAX
REGT1 :35/37 for diag	on
REGT2 :38 diag	
REGT3 :36/84 diag	tion selection
(Enter) to select; (Esc) to quit	

SQL Query Editor

<pre> »(REFINERS)----- Select REF_DATA_STATUS REF_ID To_char(REF_LAST_UPDATE, REF_CURRENT REF_CURR_RATE REF_CHOC_TYPE from REFINER Order by REF_CHOC_TYPE, REF_ID ===== format ===== Refiner Information @ Refiner Current Rate (Parameters)----- </pre>	<pre> (REFINERS)----- Add ... a new line (push line down) Delete ... the line at the cursor Undo ... put back deleted line(s) Copy ... copy line, or selected block Select ... select line(s) for copy/delete Break ... break into 2 lines at cursor Zap ... erase to end of line Width ... set page width (min 20) Join ... to following line Kill ... clear entire query Go into ... to try a new query Run ... to run unformatted query »X'ecute ... to run formatted query Input ... to set parameter substitutions Table ... get column names into query Line ... sort out line-drawing chars Print ... hardcopy to PC printer Vax ... ship query down to Vax Fetch ... contents of another query End ... and save changes Quit ... and abandon changes (Enter) to select; (Esc) to quit </pre>
--	---

<pre> »(REFINERS)----- Select REF_DATA_STATUS REF_ID To_char(REF_LAST_UPDATE, 'HH24:MI') REF_CURRENT REF_CURR_RATE REF_CHOC_TYPE from REFINER Order by REF_CHOC_TYPE, REF_ID ===== format ===== Refiner Information @3 Refiner Current Rate Choc (Parameters)----- </pre>	<pre> C1, C2, C3, C4, C5, C6 </pre>																																								
<pre> Refiner Information @3 Refiner Current Rate Choc </pre>	<table border="1"> <thead> <tr> <th colspan="4">Refiner Information 14:43</th> </tr> <tr> <th>Refiner</th> <th>Current</th> <th>Rate</th> <th>Choc</th> </tr> </thead> <tbody> <tr><td>C01</td><td>0</td><td>0</td><td></td></tr> <tr><td>C02</td><td>0</td><td>0</td><td></td></tr> <tr><td>C03</td><td>0</td><td>0</td><td>38N</td></tr> <tr><td>S08</td><td>0</td><td>0</td><td>38N</td></tr> <tr><td>S09</td><td>133</td><td>.1</td><td>38N</td></tr> <tr><td>S10</td><td>135</td><td>.04</td><td>38N</td></tr> <tr><td>S11</td><td>0</td><td>0</td><td>38N</td></tr> <tr><td>S12</td><td>0</td><td>0</td><td>38N</td></tr> </tbody> </table>	Refiner Information 14:43				Refiner	Current	Rate	Choc	C01	0	0		C02	0	0		C03	0	0	38N	S08	0	0	38N	S09	133	.1	38N	S10	135	.04	38N	S11	0	0	38N	S12	0	0	38N
Refiner Information 14:43																																									
Refiner	Current	Rate	Choc																																						
C01	0	0																																							
C02	0	0																																							
C03	0	0	38N																																						
S08	0	0	38N																																						
S09	133	.1	38N																																						
S10	135	.04	38N																																						
S11	0	0	38N																																						
S12	0	0	38N																																						

SQL Query Editor

GENERAL ARTICLES

This section of VECTOR is oriented towards readers who may neither know APL, nor be interested in learning it. However we hope you are curious about how, under the right conditions, such impressive results can emerge so quickly from APL programmers.

Finished Goods Inventory Control

by Barrie Webster

The One-man Show

Dave Watson is in charge of my local off licence. He keeps track of several hundred different wines, beers, and soft drinks. Much of his day is spent re-stocking shelves and talking with customers. He also finds time to spend a lot of time talking with suppliers, going to wine-tasting parties, and reading the various speciality papers.

Peter Mason runs the large book store in the next town. He doesn't buy from publishers himself, but has a number of experienced people, each one responsible for a department such as medical books, or children's books, or business. Penny MacDonald deals with fiction and knows where a book is on the shelves, what is selling well, and what is going out.

Jean Wolstenholme sells upholstery and drapery fabric. She has a stock of several hundred different bolts of cloth. She spends a lot of time every day working with customers to help them choose the right material, and to estimate the length they need. She also reads decorating magazines, talks to salesmen, orders material and related supplies.

None of these successful businessmen uses a computer for anything. Each one manages several hundred to a few thousand different items. In talking with customers each one is fully aware of what is available from the suppliers. In talking to suppliers, each one is fully aware of what has been moving off the shelves. The decisions about how much of what to buy are instantaneously related to knowledge about what is available from suppliers.

The Large Corporation

Now consider any large corporation. The job there is too large for one person to cope with. For one thing the business is spread out all over the country, or all over the world. You just can't stand in one place and see all the inventory. Hence it is necessary to specialise the job so that it can be parcelled out to different people.

The Purchasing organisation can buy raw materials on good terms from many suppliers, and knows a great deal about efficient means of manufacturing, and holding to quality standards. The Production Department knows a great deal about how to get things made.

Because of the separate organisations, Production doesn't know a great deal about supply. The informal and instantaneous communication about supply and demand, that the High Street shopkeeper enjoys, requires considerable buffering in a large organisation. For many years inventory was a useful buffer between demand and supply. Inventory takes working capital (and space), and has a tendency to become obsolete.

There needs to be a better way of sharing supply and demand information, so that the corporation can make informed operating decisions as easily as the shopkeeper does. A computerised implementation of the concepts of MRP-II should unify supply and demand decisions.

Using MRP-II to Share Information

Every day or so, current stock and requirements at every location are automatically processed. This generates shipping requirements on the sources of supply (vendors or plants), so that current priorities of work can be adjusted to match current needs in the marketplace. The same information is used to decide how much of which products to send to each destination, so as to replenish stocks in the most efficient way.

Perhaps once a week, the information about stock status and requirements generates timed streams of notional orders (out to the planning horizon), for long-range visibility at the sources of supply (either plants or vendors).

In many businesses it isn't necessary to revise the forecasts more often than once a month. The results of all calculations are immediately available to other calculations, so that it isn't necessary to co-ordinate weekly production planning with monthly forecast revisions.

Whenever standard costs change, usually at the annual budgeting process, economic lot quantities and safety stocks can be revised.

With a single database in constant communication with order entry, manufacturing control, accounting, marketing and warehouse operations, the demand manager can see the impact of changes on availability from the supply side. The supply manager is working to meet a current, accurate and complete picture of what is really required in the field.

And it is all done in APL!

A major application product has been developed by R.G. Brown, one of the leading authorities in this field. Only one computer language was thought to be appropriate for this type of sophisticated decision support system - APL. This PC implementation is based on APL*PLUS/PC.

MetaAPL: an APL Preprocessor

by Glen Tickner and Norval R Oswald

Studies conducted by IBM in the 1960's showed that APL, when compared with traditional languages, reduced the time required by programmers to develop systems. However, the use of APL has for the large part been restricted to small applications and applications for which the development time is necessarily short. In particular, APL has remained a language poorly suited to large applications which must be maintained over an appreciable length of time.

The Performance Problem

The major factor prohibiting the use of APL in the development of large systems is its inefficiency when compared with compiled languages. For smaller applications performance was not a pressing problem, since the APL interpreter can efficiently handle most operations where repetitive execution is not required [3].

The APL world has made a serious commitment toward solving the problem of efficiency; first, with the addition of efficient substring search and table lookup functions, either built into the language or provided as add-on functional interfaces to external programs. An example of the former would be STSC APL's substring search function, quad SS, while the search functions available through INTERPROCESS's CALL/AP or AFM systems provide examples of the latter. Secondly, with the advent of APL2, a new family of efficient (albeit external) subroutines has become possible, leading to a variety of new offerings from APL vendors.

The third area of progress in improving the efficiency of APL may prove the most significant. Thanks to STSC, an APL Compiler has at last become practical and commercially available. At least two additional (or more properly, "translators") have recently been developed.[2,3] These events may at last remove efficiency as a barrier to the use of APL in large applications.

Maintainability as a Barrier to the Use of APL

Before it can achieve acceptance as a development tool for large systems APL must improve its reputation for maintainability. Most of those familiar with APL have heard the story of the APL programmer whose code must be discarded when he leaves the company. With maintenance costs typically running at 50% of costs over the software development life cycle,[1] APL will not broaden its acceptance as a language for developing large applications if what it gives in programmer productivity is later taken back in maintenance costs.

Part of the reason APL code is difficult to modify and errors are difficult to diagnose lies in the powerful and condensed primitives. The primitives, alone or in combination, can form constructs whose meaning is extremely difficult to decipher. Because this problem is so intrinsic to the structure of the language, it may not have a solution other than the development of staff fluent in the use and interpretation of APL primitives.

Other factors affecting APL's maintainability are more tractable. These factors can be classified under one of two headings:

1. features of the language which make APL less transparent
2. the absence of structured conditions

"Transparency" refers to the absence of hidden effects and the ease with which accurate function or program cross-reference information can be generated. Some of the opaque features of APL:

1. The APL convention (entirely backwards from our point of view) that variables are assumed to be global unless they are explicitly designated as local in the function header. The typical consequence of the latter, unless the programmer is vigilant, are unintended globals, or "leaky" variables. Leaky variables may lead to side-effects and excessive space utilization. These pathologies are most likely in large systems developed by more than one programmer, where they are most difficult to correct.
2. The capability of making multiple assignments on one line, encouraging "statement-gluing", long lines of virtually unreadable APL code. The most severe offenders in this area may be the experienced programmers who have developed the most familiarity with APL primitives.
3. APL contains primitives for the conversion of character strings into variables, variables into functions, and vice versa. These hidden assignments make cross-referencing a difficult accomplishment for both the programmer and automated tools.

All of the above three problems can be overcome with proper programmer vigilance. For example, the programmer can adopt the convention of making explicit references to all global objects in the comments section at the top of each function. However, the programmer need not do so, and this fact has particular importance when multiple programmers contribute to the same application.

Formal Structures

Similarly, it is quite possible to write structured code in APL (or with any language that depends upon branching to resolve conditions). In practice, APL conditions rarely correspond to Dijkstra's eight allowable structures (for several examples, see Martin [7]).

The language's lack of structure has long been a source of embarrassment to supporters who feel that APL's unique character make it superior to more traditional languages. In the early 1970's, following Dijkstra's criticisms of APL, several attempts were made to add control structures to the language. Three separate approaches have been taken:

1. The Functional Approach

The simplest approach has been to define APL control functions such as "IF" and "DOWHILE" (see Lim and Lewis [6] and Mason [8]). This solution is also the least satisfactory, for two reasons. First, the procedure is inefficient, since it adds unnecessary function calls. Secondly, nothing would prevent or discourage a programmer from using a raw branch, since the GOTO remains an integral part of his programming environment.

2. A-Pre-Compiler

A significant improvement over the functional approach is to provide a structured control language which can be later compiled into APL code. The best-known example of an APL pre-compiler is APLGOL, written to take advantage of both APL's powerful operators and ALGOL's control structure.[5]

Pre-compiled APL code carries no penalty in terms of either performance or space, since the functions which establish the control structures normally would not reside in the workspace at run-time. Furthermore, the source code can be indented to highlight the structure of code blocks. With some imagination, checks can be added to the compiler to bring to APL some of the code verification attributes of languages such as PASCAL.

Mandatory features of any APL pre-compiler would be utilities for the printing of source code and the recompilation of object code. These features would help ameliorate the problem that the interpreted functions would not be the same as the functions as they were originally developed.[4]

3. Integration with the Interpreter

There has been one documented case of integration of high-level control structures into an APL interpreter.[4] The modifications took little programming effort, code written with control structures such as *IF* and *DO WHILE* could run in conjunction with code using the raw APL branch, and the implementation actually improved performance.

Clearly, the best option from a technical viewpoint is integration with an APL interpreter. However, for reasons of portability and compatibility with other APLs and releases of APL, an interpreter with improved control structures is not likely to become available, except through one of the major vendors. And, as the last fifteen years have demonstrated, the major vendors are unlikely to risk such fundamental modifications to the language syntax. The best alternative at this time would appear to be some form of pre-processor.

The advantages of structured languages have been well documented. Structured code is easier to read and therefore easier to maintain and modify. Because structured code has fewer possible paths it is less likely to contain undetected errors.

Why have more formal control structures not become an accepted part of APL? In part, because the APL operators are so well structured. Conditional logic has always been recognized by APL programmers as something to be avoided, particularly as it often exacts a heavy cost in machine resources.

The development of APL compilers are likely to change the APL world's resistance to structured controls. As larger APL systems become feasible the requirement for conditional execution will become of increasing significance. At the same time, the advent of compilers may change APL programming style, since they remove the penalty associated with APL branching.

META\APL

Code maintainability and reliability are both central concerns of ACT Computer Services of Edmonton, Alberta. ACT began using VS/APL in 1983 and now has an APL programming team of from 5 to 8 programmers. The company's APL programming group develops and maintains several large database and management information systems.

ACT's APL programming is responsible for the following databases and supporting software:

1. Two time series databases:
 - a) CANSIM, the Government of Canada's public database. ACT's CANSIM holdings include approximately 63 megabytes of data.
 - b) ASIST, the Government of Alberta's public database. 42 megabytes.
2. Two "multi-dimensional" databases stored in ALASS (A Large Array Storage System), which uses keyed and component files to simulate multi-dimensional APL arrays:
 - a) MDASIST, the Government of Alberta's public database for multi-dimensional data. 64 megabytes.
 - b) EXPORTS, the Government of Alberta's holdings of exports data for Canada and Alberta, 34 megabytes.
3. One relational database system storing records of doctor and hospital visits, practitioner and patient addresses, and billing and diagnostic code descriptions (approximately 200 megabytes).

The APL software which provides the user interface to these databases includes between 60 and 70,000 lines of decommented code.

ACT decided on the need for tools to improve the quality and maintainability of its systems written in APL. To this purpose, ACT developed META\APL, a pre-processor with the following features:

1. A structured programming language.
2. A decompiler.
3. Automatic documentation during compilation. Documentation includes the programmer name, time of the compile, a box region listing global references, and a classification of the META\APL type.
4. Function diagnostics.

The current version of META\APL is written in APL and occupies about 40K of memory.

META\APL is invoked as a function editor, so that the programmer would enter the command

`MEDIT 'fname'`

where 'fname' is the name of the function to be edited. The syntax is the same whether 'fname' is a new function or an existing function that is being decompiled.

If an existing function is to be edited, META\APL creates a canonical representation and enters edit mode using the available editor (in this case, the full-screen VS/APL XEDITOR). META\APL converts the functional representation into a character variable in order to permit indentation of code blocks. META\APL indents the blocks automatically when a function is decompiled.

The version of META\APL currently in use at ACT is an elementary prototype. Despite its simplicity and some rather strict limitations imposed by the current version, all APL programmers have been required for the last two years to do all their development work in META\APL.

META\APL Structured Conditions

META\APL's syntax consists of two blocked conditions and two line conditions, as shown in figure 1 below.

<u>META\APL</u> <u>SOURCE CODE</u>	<u>OBJECT CODE</u>
<pre> <WHEN n condition1 function1 function2 WHEN n condition2 function3 => </pre>	<pre> ΔWB: →(condition1)ρΔWE function1 function2 →(condition2)ρΔWE function3 →ΔWB ΔWE: </pre>
<pre> CASES <CASES n caseselect case1: statement1 case2: statement2 case3: statement3 => </pre>	<pre> + caseselect →ΔC case1: statement1 →ΔC case2: statement2 →ΔC case3: statement3 ΔC: </pre>
<pre> IF n condition n s1 n s2 </pre>	<pre> →(condition)ΦΔ2,Δ1 Δ1:s1 →Δ3 Δ2:s2 Δ3: </pre>
<pre> QUIT n condition n endroutine statement1 statement2 </pre>	<pre> →(condition)+ΔQ1 endroutine →ΔQ ΔQ1: statement1 statement2 ΔQ: </pre>

Figure 1. Source and Object Code for META\APL Conditions

The following example shows how global, local, and semi-global objects would be entered into a new META\APL function:

```

      V RESULT+FUNCTION ARG;SEMI1;SEMI2::GLOBAL1;GLOBAL2
[1]  ATTIL
[2]  A Unformatted comment region.
[3]  ATTIL
[4]  LOCAL1+FN1 ARG[1]
[5]  LOCAL2+FN2 ARG[2]
[6]  GLOBAL1+FN3 SEMI1
[7]  GLOBAL2+FN4 SEMI2
[8]  RESULT+GLOBAL1 FN5 GLOBAL2
      V

```

Figure 2. Newly-entered META\APL source code.

The semi-global variables, *semi1* and *semi2*, are assigned as globals in the subfunctions *fn1*, *fn2*, or *fn3*. Local variables (*local1* and *local2*) will be localized in the function header. The compiled function will appear as shown below:

```

      V RESULT+FUNCTION ARG;LOCAL1;LOCAL2;SEMI1;SEMI2
[1]  RM+APL[1]SEMI1;SEMI2::GLOBAL1;GLOBAL2
[2]  R1987 3 16 0 49 24
[3]  ATTIL ( Name ) 87/MAR/16(MON) 0:49 AM -----
[4]  A -----
[5]  A Automatic Copyright Region
[6]  A -----
[7]  A
[8]  A Formatted comment region (words are left and right justified).
[9]  A
[10] ATTIL-----
[11] A FN1 FN3 FN5 GLOBAL2
[12] A FN2 FN4 GLOBAL1
[14] ATTIL-----
[15] LOCAL1+FN1 ARG[1]
[16] LOCAL2+FN2 ARG[2]
[17] GLOBAL1+FN3 SEMI1
[18] GLOBAL2+FN4 SEMI2
[19] RESULT+GLOBAL1 FN5 GLOBAL2
      V

```

Figure 3. Compiled META\APL function, showing comment region.

Note the automatic generation of date and programmer identifier information (programmer identification is stored in a global variable, as is the copyright notice).

Since local objects are automatically entered into the function header the programmer does not need to waste time entering insignificant local objects, such as loop counters, into the function header. However, META\APL requires the programmer to be vigilant in keeping track of global objects by forcing him to list them in the function header. This is as it should be, since it is far more important that the programmer remain conscious of global function outputs. Note the globals list automatically generated below the copyright region.

Function Diagnostics

An APL pre-compiler provides an excellent vehicle for the integration of function diagnostics. META\APL performs the following checks during compilation:

1. META\APL syntax check

The pre-compiler verifies that the source program has been written in correct META\APL syntax (closed blocks, no branches, etc.).

2. APL syntax check

Checks against unbalanced brackets, parentheses, or quotes or non-unique function headers.

3. Multiple assignment check

To ensure that META\APL code is fully restartable and to eliminate "statement-gluing", a check is run against multiple assignments. This check can be bypassed so as to permit such necessary constructs as

```
'var+fn cond2' [BEA 'var+fn cond1'
```

Several additional diagnostics can be performed after compilation:

1. prefix LEAKYNAMES n1

Prints out the names of all global objects called by functions in the name list, n1, not prefixed with the characters provided in prefix.

For LEAKYNAMES to be effective, consistent naming conventions must be adopted for all global objects (a good idea in any case) so that the function will not output long lists of objects intended to be global.

2. prefix UNUSEDNAMES nl

Prints out all objects called by functions in the name list, nl, that have been localized or input as arguments but never used.

prefix can be used to exclude semi-globals from the unused name search.

3. prefix UNASSIGNEDLOCALS nl

Prints out all objects called by functions in the name list, nl, that have been localized but never assigned. Like LEAKYNAMES, this function can be used to trap typographical errors in the naming of variables.

META\APL Enhancements

In response to requests from the APL programming group at ACT Computer Services, META\APL is currently undergoing modification. The new version will be written in C to allow for the addition of new features without a penalty in performance. The new version is being developed for a variety of micro and mainframe APLs. A version for STSC APL*Plus running on a Macintosh personal computer will be available at or shortly after the time of this conference.

The following enhancements will be incorporated into the new release:

1. The restriction of one condition block per function will be removed. Block structures can be nested or entered sequentially.
2. IF and QUIT forms will be expanded to a block structure so that multiple-line DO blocks are permissible.
3. Global and semi-global variable lists will be entered in a block region. The declaration block will have the same format as condition blocks.
4. META\APL error messages will be improved to give better information regarding errors in the source code.

The current version of META\APL is available on a beta-test basis from ACT Computer Services of Edmonton, Alberta.

REFERENCES

1. Boehm, "The High Cost of Software", *Practical Strategies for Developing Large Software Systems*. Addison-Wesley, 1975.
2. Ching, Wai-Mee, "Program analysis and code generation in an APL/370 compiler", *IBM Journal of Research and Development*. November 1986: Vol. 30, no. 6, p. 594.
3. Driscoll, Graham C. Jr. and Donald L. Orth, "Compiling APL: The Yorktown APL Translator", *IBM Journal of Research and Development*. November 1986: Vol. 30, no. 6, pp 583-593.
4. Harris, Larry R., "A Logical Control Structure for APL", *APL Congress 73*, P. Gjerlov, H.J. Helms and Johs Nielsen, eds. North-Holland, 1973.
5. Kelley, R.A., "APLGOL, an Experimental Structured Programming Language", *IBM Journal of Research and Development*. January 1973: Vol. 17, no. 1, p. 69.
6. Lim, A.L. and G.R. Lewis, "Toward Structured Programs in APL", *The Computer Journal*. August 1974: Vol. 18, no. 2, p. 140.
7. Martin, B.R. "Concepts of Structure in APL", *APL80; International Conference on APL*, Gijsbert Van Der Linden, ed. North-Holland, 1980.
8. Mason, J.A., "Some User-Defined Control Functions for More Readable APL Programs", *ACM SIGPLAN Notices*. 1975: Vol. 10, no. 8, p. 11.

Expcheck: an Intelligent Diagnostic System in APL

by Mike Williams

Introduction

Much discussion has been recently devoted to the question of whether or not APL can be used effectively for programming within an artificial intelligence environment. The major arguments appear to centre on how one can mimic the various facilities that are available in fourth-generation languages within the APL environment, for example the use of nested arrays in APL2 to mimic the procedural structure of LISP. However, a more serious question that should be considered is how to construct a suitable rule-base, that can be easily referred to from within the APL environment.

In the context of a project (funded by the Alvey directorate) to develop a general system for experimental design, a diagnostic system for spotting problems with proposed designs has been constructed. Written in APL, ExpCheck uses file management to control the rule-base that assists in the detection. The way in which the system works goes some way towards solving the problems mentioned.

Background

The traditional method for developing rule bases for intelligent systems is by discussing with an expert how decisions are reached in their specialist area. An alternative to this is to ask what questions would be asked if a problem was suspected, or if someone wanted to check a solution to a problem. Such checklists for experimental design have been in existence for some time, but for the purposes of this project 'Statistical Checklist 1: Experimental Design' by J.N.R. Jeffers was selected.

In order to try and gain further insight into the philosophy of designing an experiment, it was decided to try and build a diagnostic system with this checklist as the basis. For the sake of simplicity, the assumption was made that the answer to each question was either 'Yes' or 'No'. Starting from this point, a network of paths between the questions can be built up, including an indication of when problems are detected. This semantic net forms the structure of the system, while the questions, and the paths linking them, form the decision-making rules required.

Structure

Each question in the checklist is represented by a relatively straightforward APL function consisting of the following parts:

- 1) The question itself, expressed in a (hopefully) understandable form.
- 2) The action to be taken, dependent on the final answer given. A global variable, N, denotes which question to ask next. The action will either be to change this, or to give a diagnosis of the problem if one is suspected.
- 3) Access to various help facilities. These are explained later.

Example 1 shows what one of the questions looks like in APL form. The global variable ROUTE is used as a store of the sequence of questions asked on the design being checked.

```

QUEST12;X
-----
[1] BLOCKBLANK 12 A Print 12 blank lines.

[2] A Question part 1 !

[3] L6: Can all of the combinations be achieved in practice ?
[4] BLOCKBLANK 12
[5] X;1+0
[6] →(X='Y','B','W','R','H','E')/L1,L4,L5,L6,L8,L10
[7] →L2
[8] L1: 'Are all of these combinations realistic in the context of your
[9] 'experiment ?' A Question part 2 !
[10] X;1+0
[11] →(X='Y','B','W','R','H','E')/L3,L4,L5,L6,L8,L10
[12] L2: 'The reasons why this may occur are many and varied. There may
[13] 'be factors you are using which do not have any effect on the
[14] 'results. There may be a large number of levels within
[15] 'each factor. Please could you take some time to think about
[16] 'your treatments. The restart code for this problem is 11.'
[17] n; 'STOP' A Action when answer is no
[18] →0
[19] L4:n;1+1 '2 route A Backtracking
[20] →0
[21] L5: WHY 12 A Why is question 12 asked
[22] →L6
[23] L8: HELPLINE A Print a summary of the commands
[24] →L6
[25] L10: DEFINE A Dictionary access program
[26] →L6
[27] L3:n; '13' A Action when the answer is yes
    9
  
```

Example 1: Question Function

There are seventy questions in total, divided into eight groups, representing the eight stages of setting up a design. Each group of questions is stored in a component file. A

separate function is used to maintain control over each section of the checklist, as well as giving a brief explanation of the things been considered. At the end of the section, the option is given of stopping to consider the results so far instead of carrying on.

Example 2 shows the control function for Section 6, where the randomisation procedure is checked.

```

LIST6;X;Y
-----
[1]  BLOCKBLANK 12

[2]  A Introductory explanations

[3]  'SECTION 6 - Randomisation'
[4]  '=====
[5]  'The randomisation of allocation of treatments to plots/units is'
[6]  'considered in these questions.'
[7]  'There are 6 possible questions to answer in this section.'
[8]  'Press any key to continue.'
[9]  BLOCKBLANK 8
[10] Y;INKEY A Accepts the code for the key depressed
[11] 'R' USE 'SECTION6.APL' A Open the relevant component file
[12] L1:→((%N)>52)/L2 A Have we run out of questions?
[13] CLS A Clear the screen presently showing
[14] route;route,' ',n
[15] %X;QFX GET n;(%N)-46 A Get and exc the relevant question function
[16] Y;QEX X A Remove the question from the active workspace
[17] →(^/n%0123456789)/L1
[18] L2:Y;RELEASE 'SECTION6.APL' A Close the component file
[19] →(^/n%STOP)/L3 A Has a problem been detected?
[20] '*****End of Section 6*****'
[21] 'Do you wish to suspend the run now?(Y/N)'
[22] →((1+Q)='Y')/L3
[23] flag;0
[24] →0
[25] L3:flag;1
  7

```

Example 2: Sub-control Function

The eight control functions are themselves stored in a component file, which is accessed as necessary by a master control function. This function forms part of the control workspace, appropriately named EXPCHECK. This workspace contains management functions, general explanations of the system and the global variables used to act as the fact base for the system.

Help Facilities

A number of simple help facilities are available to the user. It may be that a user would want to change the answer to the previous question. This is possible because the route

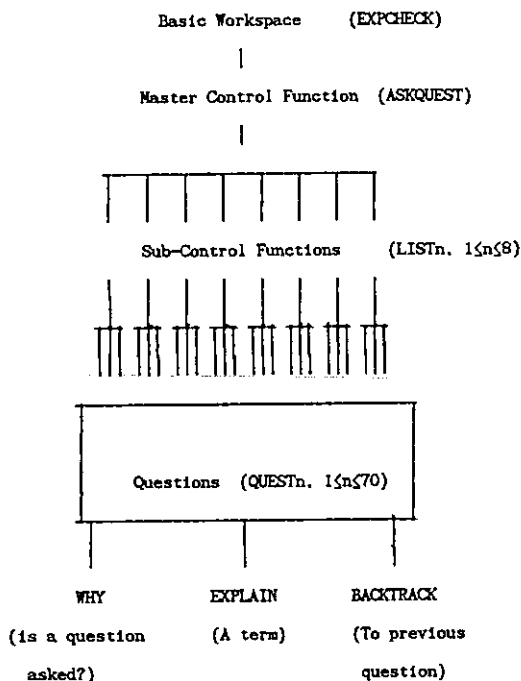
taken through the checklist is stored, and backtracking to the previous question is accomplished by executing:

$n \leftarrow 1 \uparrow 2 \uparrow \text{route}$ A Backtracking

... thus giving N the value of the last question asked.

A simple dictionary of terms is included with the system, and this is easily accessed by answering E(xplain) to a question in the list. Similarly, a component file with explanations of the reasons for asking each question is included, and this can very easily be called up.

Figure 1 shows the overall structure of ExpCheck in a very clear and simple manner.



The results of any run through the system, the questions asked and facts gathered, are stored in a reference file for recall at a later date, or in a future run. The name of the file is used as a reference point.

Dialect Used

ExpCheck was originally conceived and written in IBM APL Version 2, running on an IBM AT with 640K of memory available. It makes use of a number of file management and utility functions. More recently, an APL*PLUS version has been prepared, and this appears to work reasonably well on IBM compatibles as well.

In both cases, the required workspaces and files fit onto one 5 1/2 inch floppy disk, and consist of:

EXPCHECK.APL	Master Workspace
EXPQUEST.APL/ASF	Sub-Control Functions
EXPDEFS .APL/ASF	Dictionary
EXPWHY .APL/ASF	Explanations
SECTION1.APL/ASF	┐
SECTION2.APL/ASF	
SECTION3.APL/ASF	
SECTION4.APL/ASF	----- Questions to be asked.
SECTION5.APL/ASF	
SECTION6.APL/ASF	
SECTION7.APL/ASF	
SECTION8.APL/ASF	┘

APL/ASF indicates the use of IBM APL V.2 or APL*PLUS respectively. The system is currently at a development stage, and it is hoped that local trials can begin in the near future.

Answers to our Questions?

Can we build a working expert system in an APL environment? The answer appears to be yes. By making each rule an APL function, and storing them in a component file, we have a flexible and powerful rule base from which to work. In addition, because the

system works with such basic components, debugging and updating can be very easily accomplished. New rules can be added on, and a record of unused rules is very easy to maintain as a global variable.

Storage of any facts is also straightforward. ExpCheck always stores the results of a run for future reference, and gives a printed report of the determined facts. As the facts are in the active master workspace, updating can be accomplished with a simple line of code.

There are still some questions that remain to be answered on this area, but ExpCheck and the lessons learnt from deriving the system provide a basis on which to continue striving for those answers.

Mike Williams

Department of Statistics,
University of Newcastle upon Tyne,
NEWCASTLE NE1 7RU

EAST MIDLANDS
CONSULTANCY
now offering
CONTRACT WORK
in
A P L

Dick Gray
Associates

Horseshoe House
Sproxtton
Melton Mowbray
Leicestershire

☎ Grantham 0476/868483

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Photo Caption Competition



A



B



C



D

Prize Competition Result: APL86 Photo Caption

by Dave Ziemann

Apologies to those of you who have been patiently awaiting the result of this competition; for one reason or another it got held over from one issue of Vector to the next, to the next, ...etc.

The four photographs labelled A, B, C and D were taken at APL86 and readers were invited to supply suitable captions. Sam Sexton suggested that photograph A should read:

A: "You certainly need these rose-tinted glasses for looking at APL2!"

Sam's entry was submitted on an I P Sharp Associates compliment slip. Iain Hunneybell thought that photo C should be captioned with:

C: "Speak no evil, Hear no evil, See no evil."

Gareth Brentnall submitted the following three entries

A: "I didn't think these new-fangled graphics would help me work out what dyadic transpose did on a 3-D array!"

C: "God, is it encode or decode? I hope he doesn't ask me."

D: "I knew rotate extended to tables, but chairs?!"

Unfortunately, there were no printable entries for photograph B. Sam and Iain each receive fourteen pounds, and Gareth wins a star prize of twenty-two pounds.

Hacker's Corner

by Adrian Smith

Here is something for all you PC freaks out there. You may remember that DOS has this thing called a keyboard buffer. APL generally ignores this (unless you run with E=2), so anything you put there in the APL session stays around until you sign off.

The buffer is but 15 characters long, so you can't do a great deal with it, but one obvious possibility is to fire up a batch file on exit from your APL system. The little function listed below will insert up to 15 chars in the buffer, replacing anything that already happens to be there. As far as I know it will work on anything PC compatible and is quite harmless in all other respects.

SHOVE TXT;XX

[1] A Shove text into DOS keyboard buffer.
[2] A Maximum length is 15 chars

[3] TXT←,TXT ⋄ TXT←(15⌊TXT)⌈TXT
[4] TXT←,TXT,(1.5)⌈⋄
[5] XX←(1+DAV⌊TXT)⌈POKE '041E'

[6] A Set head and tail of buffer ...

[7] XX←30 ⌈POKE '041A'
[8] XX←(30+⌊TXT)⌈POKE '041C'
⋄

e.g. SHOVE 'MENU' ⋄ DSA←'OFF' A Exit, then run MENU.BAT

WAS+BYTES ⌈POKE LOC;SEG;DELX;N

[1] A Poke to Hex location LOC for N bytes
[2] A Offset defaults to low memory

[3] DELX←'ODM' ⋄ →Exit
[4] LOC←⌊LOC ⋄ DSEG←0 ⋄ BYTES←,BYTES ⋄ LEN←⌊BYTES

[5] A Check for offset in form 'F000:1234' and set DSEG ...

[6] →(1:⌊LOC)⌈Base
[7] SEG←4⌈LOC ⋄ LOC←5⌈LOC ⋄ DSEG←16⌈1+⌈'0123456789ABCDEF'⌈SEG
[8] Base←LOC+16⌈1+⌈'0123456789ABCDEF'⌈LOC
[9] WAS+BYTES ⌈POKE 1+LOC+⌈LEN
[10] Exit;DSEG←0
⋄

Technical Correspondence

How to Unlock Locked Functions in APL2

From: John Sullivan

27th May 1987

Now that David Piper has 'blown the gaff' on the APL2 system command)CHECK SYSTEM, I feel I can rush into print with the following useful item. You may, for some reason or other, have a locked function that you want to unlock, and nowhere in the APL manuals can you find a way of doing this. The method is as follows:

In the APL workspace, enter the following

```
)CHECK SYSTEM SYSDEBUG(128)
)OUT filename function-names
```

Then you edit the output file (which will be APLTF.filename in TSO, unless your system has changed the defaults or you have enclosed the name in quotes), changing all occurrences of 1 1 1 1 □FX to 0 0 0 0 □FX. You can do this without leaving the workspace if you are using ISPF. The final step is to enter

```
)IN filename
```

and your functions will then be unlocked in the workspace.

Note that if you don't issue the)CHECK command, the)OUT command will not write the locked functions to the output file.

P.S. With reference to Joseph de Kerf's reply in the latest VECTOR to my article on Fast Fibbing, yes I did know about omitting the second term in the formula (it appears in other sources, not just the one cited. A good analyst should be able to spot that the second term can be omitted without reference to the sources, in this case), but in my hurry to prepare the article, which was written from memory, I completely overlooked it. I realise that this is not good enough, and so I tender my humblest apologies. Mea culpa!

John Sullivan
Business Development Division
Natonal Westminster Bank plc
41 Lothbury
LONDON EC2P 2BP

More about APL2

From: Simon Barker

Following on from Dave Piper's letters regarding the undocumented APL2 features, here are some more oddities/features/bugs that have come to light.

The Atomic Function (⌈AF)

Primarily, this system function has two uses: given a simple text array it will return the indices into the atomic vector for each element; and when given an array of integers within the range 0 to 255 will use these as indices into the atomic vector to return text. In both these cases an index of origin of zero is used which is local to the system function.

This is not the end of the story however, because ⌈AF will return a result for ANY positive integer. When an integer exceeds 255 a special character is returned which is not a member of the atomic vector (typically, IBM chose to represent this as an Omega symbol). This can be regarded as a character encoding of the given number, with a storage requirement of 4 bytes. Thus, when ⌈AF is given this symbol as its argument the original number encoded is returned.

As you may know, APL2 will store integers in a single byte if the values lie in the range 0 to 255. The only way I have found to force storage in this way is by using ⌈AF. All that is required is to use ⌈AF on the output of ⌈AF used on the data you wish to compact. Beware though, any number representation other than positive integer or boolean gives a domain error.

The)CHECK Command

Here are some parameters that can be passed to the)CHECK system command.

)CHECK SYSTEM SMAPL (OFF)

This will switch the Session Manager off mid-session, saving screen data if the log size is set greater than zero.

)CHECK SYSTEM SMAPL (ON)

This switches the Session Manager back on, with no corruption of the session log if the log size is set above zero.

)CHECK TRACE STMT

This will display each line of any function or operator executed, next to a CPU usage figure indicating how much CPU time has elapsed prior to the execution of the line.

)CHECK TRACE EXEC

This command displays information for any line of APL that is executed, whether in a function, operator, or entered directly. As well as showing the information indicated above, special two-character prefixes are used to indicate how APL2 is processing the expression. These indicate such things as recognition of idioms that are coded as special routines in the APL2 Executor.

Other facilities available include various types of workspace dumps, listings of the names of the assembler modules that APL2 is comprised of (with save dates, problem reports, etc.), and other trace options. A full explanation of these commands is available in the IBM manual "APL2 Diagnostic Reference", order number SY26-3932-1.

Notes

I am currently using APL2 release 2, running under CMS, at my workplace. Prior to this I used VSAPL under CICS and CMS.

Keep up the good work on VECTOR - it's much better than Quote Quad, and at a cheaper price.

Simon Barker
55 Conisborough Crescent
Catford
LONDON SE6 2SP

Idioms and Oddities**Removal of all rows which are duplicated**

Removal of duplicate (multiply?) rows from a character matrix (leaving 1 copy) is a much-used process with several idiomatic synonyms. I tend to stick to the 1 1 transpose version, e.g. FINNAPL No.208 (et al), from which the following is derived.

I recently found a need to do a similar thing, except that I wanted to scrap all the entries in duplicated rows (i.e. no copies retained). One feels that this must be published somewhere, but I have a limited set of APL references, and scanning the FINNAPL

permuted index under 'duplicate', 'rows' etc. got me nowhere - although I always have a nasty feeling that with 600+ entries I might have missed something.

Hence the suggestion listed below, which at least served my purpose at the time. Although I slightly regret the embedded assignment, I think it makes it a little clearer than repeating the assigned part, and perhaps it makes it more efficient.

```
IDiom
(~+/<\Y-<\Y*X^.=RX)/X+A2
```

```
A2
SURELY
THERE
MUST
SAME JUNK
BE
A
SAME JUNK
SAME JUNK
BETTER
WAY!
```

```
⊣IDiom
SURELY
THERE
MUST
BE
A
BETTER
WAY!
```

THIS IS ONLY TO DEMONSTRATE THE EDGE
CONDITION (ALL ROWS ARE DUPLICATES),
AND ORIGIN INDEPENDENCE.

```
⊣3
1 2 3
```

```
A2
THIS
THIS
THIS
THAT
THAT
```

```
⊣IDiom
0 4
IDIO+0
⊣3
0 1 2
⊣IDiom
0 4
```

A peculiarity in Quote Quad ...

Using VS APL under VM/CP I had, tucked well down in a 30-odd line function, two lines of fairly conventional form:

```
D←SINK←'MESSAGE'
→((('YN'=1↑(ρSINK)+D),1)/YES,NO,OTHER
```

but kept getting the message catenated onto the name of a local variable. After looking in the wrong place for a while, I decided to think instead, and soon found towards the top of the function I had accidentally typed:

```
..... 1+0p0+VARNAME .....
```

where the Quote-quad assignment should not have been there at all! Well, that was alright; after deleting the two unwanted symbols it behaved properly. Clearly, because of the 'nought reshape', the name was stacked away and appeared on the next Quote-quad call.

However, what I found of interest was the behaviour of TRACE in this situation. With the error present, and TRACE embracing the pair of lines mentioned, the workspace seemed to be locked into an inescapable situation, and the 'message' kept re-appearing whatever I did. Normal interrupts (PA1 and PA2) seemed to be of no help, and switching the monitor off (naughty boy!) did no good; the logon gave automatic re-connect into the same loop - with the 'heiroglyphic' character set, of course.

Up until recently I have successfully managed to avoid VS APL, but the mortgage repayments need looking after, so I was stuck with it. I will also echo other people's comments about IBM manuals, so I didn't bother with those. Well, a little more trial and error showed me the only way out I could find, which was a weak interrupt (PA1) into CP, then a direct CP logoff - and of course you cannot save anything. Interestingly, with the error (and TRACE still on) the same loop re-appeared, but this time the strong interrupt (PA2) worked as expected; one could clear the stack and save the work.

Can anyone tell me whether this problem is specific to VS APL, or do similar things happen with other APL implementations?

Dr Peter Branson

Oaklands Cottage
Wray Common
Reigate,
Surrey RH2 0LE



H.M.W. PROGRAMMING CONSULTANTS LTD

Why not discover more about

- ★ **Consultancy/Support Service**
- ★ **APL on the IBM PC**
- ★ **VM/CMS Packages**

Ring Ken Jackson at

H.M.W. PROGRAMMING CONSULTANTS LTD

142 FELTHAM HILL ROAD,
ASHFORD, MIDDLESEX TW15 1HN
Telephone: Ashford 41232

Workspace Management - or, Re-emphasising Idioms

by Dr Peter Branson

The idiom which I call 'del-all-dupes' (see Technical Letters on page 96) was invented (or, more likely, re-invented) for a particular application. But then, as often seems to happen, I found another couple of uses for it within the space of a few days, both of which arose when playing with some workspace management functions. (Although many commercial APL packages have a good set of WS management functions bundled with them, there are times when you may need to generate your own, as I was doing at the time.)

The first example arose when enlarging a simple function to list the WS functions, but excluding the listing function itself, and one or two others. Instead of the more traditional ways of removing selected names from a table, it seemed that an interesting variant would be simply to add the unwanted names and then use the above idiom to get rid of them. This is done in LISTFNS, which in this instance is set up to give just a screen display. (CHAIN1 is a simple function for vertical catenation and ALPHSORT will be self-explanatory.)

```
LISTFNS;M;W;Y
-----

[1]      A Display (almost) all functions in WS with OCR

[2]      W←1+ρM←DNL 3

[3]      A First remove unwanted names by adding them
[4]      A and using the DEL-ALL-DUPES idiom ...

[5]      M←M CHAIN1 'LISTFNS' CHAIN1 'LISTVARS' CHAIN1 'ALPHSORT'
[6]      M←(←+/<<Y<←Y←M^,=BM)/M
[7]      M←ALPHSORT M

[8]      A Now loop through list ...

[9]      Lp: '
[10]     '
[11]     →(0=1+ρM)/End
[12]     UCR,(1,W)←M
[13]     →Lp Δ M← 1 0 +M
[14] End: '** DONE **'

V
```

The other case was the somewhat more difficult task of identifying only the global names in a function. Along with other sources, Chapter 10 of Adrian Smith's book "APL - A Design Handbook for Commercial Systems" provides a sound basis for generating WS management functions, but I have always thought it was a slight pity that his function ΔSTRIP (p.128), which aims to cut out most unwanted names etc., doesn't get rid of labels

as well. Of course, as he comments, if you are surveying a complete WS the 'and dot equals' against `⍵NL 2` and `3` will do most of the work - but this doesn't cover the case where the same label name is used both locally and globally (and I have been caught by that before!). Also, I like to have a stand-alone 'stripper' to look at individual functions without doing the whole workspace each time. As well as labels, I sometimes like to knock out QUAD names (as here), although there can be a case for keeping the function modifiers (`⍵FX`, etc.).

My extended version of `ΔSTRIP` is listed below. Firstly, the check in line 9 avoids an INDEX ERROR if the name is mis-typed. Then QUAD and the colon are included in `ALPH`, and removal of QUAD names in line 38 is straightforward since the quad symbol will always appear first. Labels are a bit more tricky, though. We need the colon to identify them (and it can be anywhere on the line except first), and we have to get rid of the name, both with and without the colon attached. The way I have chosen is shown in lines 26 onwards: genuine duplicates must be reduced to 1 copy first (otherwise they get lost later!) and, as usual, some extra blanks must be included here to avoid names coalescing. Then by replacing the colons with blanks, and tidying up, the 'del-all-dupes' idiom in line 32 (which, to re-emphasise, removes all entries of a duplicated name) can finish the job (well, almost - see below). It is also necessary to move the deletion of numerics and header names lower down, in case they are associated with labels; and I have taken the liberty of putting in a couple of extra local variables, which I find helps to make the switching between vectors and matrices a bit easier to follow.

Adrian Smith notes on p.128 of his book that this function contains a "...remarkable selection of the more obscure idioms...". Perhaps I have added to this obscurity, but I think it does demonstrate an important point in APL, which, although well-known to many, is always worth re-emphasising: idioms provide another level of modularity in APL (below functions).

A good idiom is origin-independent (unless specifically stated otherwise), handles null conditions correctly, and produces a well-formed output from a well-formed input. Thus, using idioms, one can often proceed through large chunks of a quite complex task without having to keep track of branches, loops, labels, and a large collection of variable names, (obscure or otherwise). One also has a slightly greater sense of security that the end result should not fall over in unusual conditions - although this must not be taken for granted.

Which brings me back to the 'almost' remark above. There are several things the extended function still won't do. Firstly, there is the rather sneaky case of a label set, and not used, but at least it is picked out. More important, though, are any implicit global references which appear (wholly or partly) within quotes. I have already referred in passing to `⍵FX` etc., and simple cases of this kind are not too difficult to deal with automatically. It is slightly harder when techniques such as boolean mapping onto tables

of function names have been used, but the one I really don't like much is a direct execute of a global name which is assembled by catenating characters and local variables. I don't know any 'clean' way of dealing with things like that, except to highlight them for visual examination, and I would be interested to hear how others have tackled these problem areas.

R←ASTRIPX FN;HEAD;ALPH;V;M

```
[1]  A Chop FN to non-localised names
[2]  A
[3]  A Adrian Smith's fn (P128) extended to also remove
[4]  A Dnames and labels. ** P.B.B.
[5]  A (** indicates additions **)
[6]  A
[7]  A ** check that FN is in WS **
[8]  →(1≠M/M^,=(1+PM+QNL 3)+FN)/Fail
[9]  A ** add '0' (and Δ) to ALPH **
[10] ALPH←'0','ABCDEFGHIJKLMNPOQRSTUVWXYZΔ'
[11] ALPH+ALPH,'0123456789abcdefghijklmnopqrstuvwxyz'
[12] FN←QCR FN
[13] A Header checklist for later
[14] HEAD←';'←ΔTAB FN[1;]
[15] M←1 0 +FN
[16] A Cut out comment lines, ravel ...
[17] V←,((~MV,='A')≠M),' '
[18] A Take out if inside quotes ...
[19] V←(~(V='''')≠\V='''')/V
[20] A Blank for non-alph ... kill mult blanks
[21] V[(~V=ALPH)/,ρV]←' '
[22] V←(~(' '=V)^' '=10V)/V
[23] A Rest of names as table ...
[24] M←' ' ΔTAB V
[25] A ** Cut out genuine dups(leave 1 copy); ravel **
[26] V←,((1 1 &\M^.=QM)≠M),' '
[27] A ** ':' → blank for labels **
[28] V[( ':'=V)/,ρV]←' '
[29] A ** remove dupl. blanks; reform table **
[30] M←' ' ΔTAB V←(~(' '=V)^' '=10V)/V
[31] A ** Cut out all copies of dups (labels) **
[32] M←(~+/\R-<\R+M^.=QM)≠M
[33] A Cut out if all numeric ...
[34] M←(~^/M^='0123456789')≠M
[35] A Remove names in header ...
[36] M←(~V/M^.=Q((1+PM)+HEAD),1+PM)+HEAD)≠M
[37] A ** take out if DNAME **
[38] M←(( '0'≠M)[;1])≠M
[39] →0,OPR+ALPHSORT M
[40] Fail:'INVALID FN NAME'
[41] A ** Index Error if FN is locked, or 1-line null **
```

**** TEST <ASTRIPX> ON ITSELF ****

ASTRIPX'ASTRIPX'
ALPHSORT
ΔTAB

A (ED) ALPH WAS DAV[65+164] ... DOES NOT WORK ON PC!!

```
LISTSTRIP;M;W;Y;M1
```

```
-----
[1]  A Display global names in (almost) all WS fns
[2]  W←~1+ρM←DNL 3
[3]  A Remove unwanted names using DEL-ALL-DUPES
[4]  M←M CHAIN1 'LISTFNS' CHAIN1 'LISTVARS' CHAIN1 'ALPHSORT'
[5]  M←M CHAIN1 'LISTSTRIP' CHAIN1 'ΔSTRIPX'
[6]  M←{~^/<\Y-<\Y+M^,=QM)}/M
[7]  A Remove blank rows for neatness ..
[8]  M←ALPHSORT M←(Mv,≠' ')}/M
[9]  A Now loop through the list ...
[10] Lp: ' '
[11]   →(0=1+ρM)/End
[12]   M1←ΔSTRIPX Y←,(1,W)}/M
[13]   Q←Y CHAIN1(Wρ'~')CHAIN1 M1
[14]   →Lp Δ M←1 0 }M
[15] End: '** DONE **'
      ▽
```

As a further thought, I haven't looked at Roger Harper's "Workspace Lister" in the BAA Public Domain Software Library, but in any case this appears from the description to be specific to APL*PLUS/PC. Does anyone have a good set of workspace management tools, for a reasonably complete sub-set of APL (ISO-APL?), which they could pass on to the Software Library?

FOOTNOTE: at the last minute, I have found a little time to look at IBM's VS APL WS management functions in UTILITY 1. However, after copying NAMEGP to another WS, I found that NAMETYPES would no longer pick up labels. Also, NAMEREPLACE has a labelled branch but the label (L2) is not set. Perhaps these (and other?) little quirks are well known to others, but I don't think I'll bother with that set of functions anymore!

Go Pack Your Knapsack - or APL is Better without Loops

by Norman Thomson

The knapsack problem is well known in Applied Mathematics and is capable of succinct APL formulation. It thereby has inherent value in expounding to mathematical audiences the worth of APL in general and of inner products in particular.

Briefly the problem is that you are given a number of items from which you must choose some to pack in your knapsack, subject to a limit on the total weight. Each item has an associated value (relating presumably to its usefulness on the expedition), and the object is to choose the items in a way which maximises this total value.

First define a binary matrix of all possible binary selections with the help of a function ALLS :

(L and R stand for left and right argument in this and subsequent function definitions, and index origin is taken as 1.)

```
ALLS : T←1×/T←Rp2
```

```
ALLS 3
```

```
0 0 0 1 1 1 0
0 1 1 0 0 1 0
1 0 1 0 1 0 1 0
```

Suppose also that the weights and values are defined as global vectors :

```
W←7 4 4 2 3
V←15 8 7 3 3
```

Now select those weight combinations which do not exceed the given weight limit R :

```
U←(R≥W+.×U)/U←ALLSpW
```

From these select the selection which gives maximum value :

```
U[;T←⌈/T←V+.×U]
0 1 1 1 0
```

These steps can be combined in a one-line function which gives all value-maximising selections and also prints the maximum value :

```
KNAPSACK: U[;T INDEX ⌈←⌈/T←V+.×U←R≥W+.×U)/U←ALLS pW]
```

```
INDEX: (L=R)/1pL
```

```
KNAPSACK 10
```

```
18
1 0 0 1 0
1 0 0 0 1
0 1 1 1 0
```

As shown above KNAPSACK not only describes the problem but is also an executable solution. The trouble is that if the number of items is other than a small value, the space requirements are enormous, and this is surely not the way to encourage APL students to program!

To write the recursive section it is useful to have a function which for a given weight limit L and weight vector R returns a matrix whose rows are solution vectors, and returns a single row of all zeros if none of the selections meets this criterion. This latter condition is expressed in the left hand part of $VALID$:

$$VALID : (1 \ 0 \mid pT) + T \leftarrow (L \geq R) \neq ID \mid pR$$

$$ID : (1R) \circ . = 1R$$

It is also necessary to have a function which joins a vector of the single (possibly repeated) lefthand blocks of the above diagram to the matrix right hand block:

$$JOINEACH : ((1 + pR), pL) \mid pL, R$$

$KNAP$ can now be completed with the recursion occurring at line 8.

```

      VZ ← L KNAP R; I; T; X; Y
[1]   → (1 < pR) / L1
[2]   Z ← 1 1 pL ≥ R
[3]   → 0
[4]   L1: I ← I + 1 + pT + L VALID R
[5]   Z ← (1, pR) + T
[6]   L2: → (1 > I - 1) / 0
[7]   X ← T[I; 1]
[8]   Y ← (X + T[I; 1]) JOINEACH (L - R[X]) KNAP X + R
[9]   Z ← Y, [1] Z
[10]  → L2
      V

```

```

      10 KNAP W
1 0 0 1 0
1 0 0 0 1
0 1 1 1 0
0 1 0 1 1
0 1 0 0 1
0 0 1 1 1
0 0 1 0 1
0 0 0 1 1
0 0 0 0 1

```

and the solution is completed with an inner product with V as in $KNAPSACK$.

The set of 9 solutions immediately prior to multiplication by V is certainly an improvement on the 32 of $KNAPSACK$, and so for practical purposes we have a better algorithm.

We can make things better still by arranging that the multiplication by V happens within every step of the loop, i.e. by adding after line 9:

$$Z \leftarrow (X \leftarrow [X + Z + . \times (-1 + pZ) + V]) \neq Z$$

```

      10 KNAP W
1 0 0 1 0

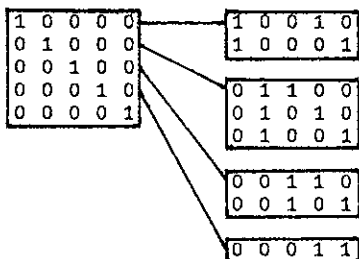
```

We therefore seek other approaches. V and W above are arranged in order of decreasing V/W, a trivial matter to arrange in APL, but one on which the following algorithm depends.

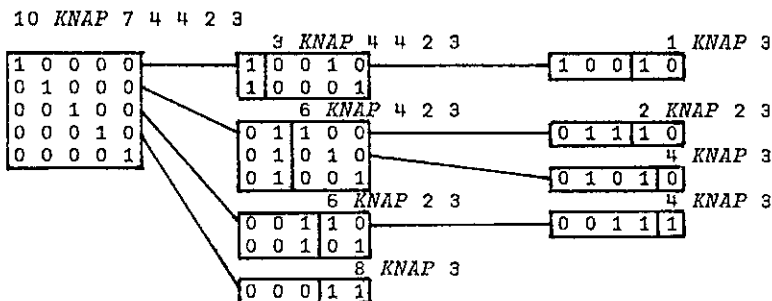
At the first step find (as binary selection vectors) those solutions which involve just one item :

```
1 0 0 0 0
0 1 0 0 0
0 0 1 0 0
0 0 0 1 0
0 0 0 0 1
```

For the first 4 consider solution vectors which include just one more item to the right and remain within the weight limit :



In each case the rightmost block is a subsidiary problem as indicated in the diagram below which also advances the process one step further :



Note that the last line in a block is always a stopper - this is where the V/W ordering comes in. The diagram suggests a recursive solution, and it seems sensible to make the (diminishing) weight vector the right argument R, and the limit weight the left argument L. Clearly the stopping condition occurs when R has just one item, so we can write the first few lines :

```
VZ←L KNAP R
[1] →(1<ρR)/L1
[2] Z←1 1ρL≥R
[3] →0
```

```
1 0 0 0 1
0 1 1 1 0
```

and now the maximum number of extant selection vectors is never more than the number competing for a tie in first place which is 3 with the current data - the algorithm gets better and better as it lengthens!

In its final stage it is what is known as a "branch and bound" solution to the problem, i.e. the V/W ordering ensures that a limit-exceeding partial vector and all its descendants are excluded from further consideration. Branch and bound is the name given to a powerful general technique, or rather family of techniques employed widely in Linear Programming and Operational Research.

APL*PLUS/PC and Assembler (Part 1)

by P Wortham (Uniware)

Abstract

Why use Assembler when we have APL? How can we use Assembler from APL*PLUS/PC? If you use APL*PLUS/PC and if you have a need for better response time, this paper may be of interest.

Introduction

The APL language does not allow us to do things in the best conditions. For instance, a table lookup is a relatively slow operation in APL. The principle is to compare each character from the table with the corresponding character from each row of the matrix. The total number of comparisons is very high. We can reduce this number with a better use of APL, but the fastest APL code will still do too many comparisons, unless it loops and is therefore very slow.

It is easy to write the same function in Assembler. This function will be able to compare a line of the matrix to a line of the table with only one comparison if their first characters differ. This solution will be much faster for big tables.

Another point is that APL needs a table and a matrix with the same number of columns. An Assembler routine can avoid this problem.

An Assembler utility function may be written in less time than necessary to optimize the APL equivalent function, and the result is better.

What may be the drawback of Assembler? One may think that development time is much larger than with APL. In the case of little functions, like a table lookup, we can see that an Assembler routine is harder to write than a bad APL function, but easier than an optimized APL function.

The Assembler routine is harder to maintain than an APL function. Obviously, we will avoid to modify the Assembler code. If we write a utility function in Assembler, this routine will be independent of the workspaces in which it is used.

But be sure to keep a copy of the equivalent APL code, because you may need to transfer your APL workspace to another computer or another APL interpreter, and the Assembler routines will be of no use. In the worst case, you will just have to install your APL code back in the workspace.

So, writing little Assembler routines may decrease the development time and optimize the response time. The maintenance will be about the same. So, why neglect the Assembler solution?

Executing Assembler from APL*PLUS/PC

Now, let us take a look at a more technical aspect.

The APL*PLUS/PC can execute an Assembler written routine with `▢CALL` or `▢INT`. `▢INT` generates a DOS interrupt. So, you can create a resident program, which you will load before APL, and this program will be activated by an appropriate `▢INT`. This is a relatively complex method. On the other hand, you can use `▢INT` for operations that are not directly accessible with APL, like testing the printer state. The available calls to `▢INT` are described in the DOS technical reference and in the technical documentation of the computer you are using.

`▢CALL`, too, allows you to execute an Assembler routine. For instance:

```
▢CALL 203
```

The program, which code is represented by the number 203, is executed. This code means 'return'. So we run a routine that does nothing. We can create an utility function that does nothing:

```

      ▽ NOTHING
[1]  0 0 ρ ▢CALL 203
      ▽

```

This utility function is named NOTHING. It may be copied from a workspace to another one, and is self-sufficient: the program (203) is contained in the function, no disk read is necessary. It is used exactly the same way as APL functions in the workspace. There is no way for the user to know that the program uses Assembler, unless he looks at the program code.

How can we write this Assembler routine that does nothing? Here is its code:

```
code    segment byte           ; defines a segment named "code"
        assume  cs:code        ; the 8088 register CS contains this value
nothing proc    far            ; the procedure's name is "nothing"
        ret                    ; return instruction
        endp                  ; end of the procedure "nothing"
code    ends                ; end of segment "code"
        end                    ; end of assembler code
```

So we first have to create a file, having an .ASM (for instance NOTHING.ASM). This file contains the instructions described above. Of course, the comments may be suppressed.

We have to assemble the program, which we do in the following way (using a macro assembler MASM)

```
MASM NOTHING;
```

This command creates a file named NOTHING.OBJ. We now have to link this file:

```
LINK NOTHING;
```

Now we have a file named NOTHING.EXE. Take care: this file cannot be loaded from DOS. If we wanted to run this routine from DOS, we have to add specific instructions, like an explicit return to DOS. When you type the above commands, you can see an error message given by LINK. In fact, it is only a warning, and if we want to use the routine from APL, we need this warning. The NOTHING.EXE file is not yet useable from APL: we will transform it in binary.

```
EXE2BIN NOTHING
```

We now have a set of 4 files, named NOTHING, with different extensions:

```
NOTHING.ASM
NOTHING.OBJ
NOTHING.EXE
NOTHING.BIN
```

The files NOTHING.OBJ and NOTHING.EXE will not be used for now. We can load APL and read the file named NOTHING.BIN. Once in APL:

```
'NOTHING.BIN'ONTIE ^1      A Open the binary file
CODE←ONREAD ^1 82,ONSIZE ^1 A Read the whole file
```

If you look at the CODE variable, you see a semi-graphic character. It is the 204th character of the atomic vector, or $\square$ AV indexed by 203 in 0 origin. You can now do the following:

```
□CALL CODE
0 0 0 0 0 0 0
```

This instruction does not do anything, but the result is a 7 zeroes vector! In fact, these are the values of 7 registers: AX, BX, CX, DX, BP, SI and DI. The Assembler routine did not modify these values, so we get their initial values back. $\square$ CALL accepts a left argument, which is a scalar or a vector of up to 7 integers. These values are passed to the above mentioned registers and these registers contents are returned by $\square$ CALL at the end of execution. For instance, if we want to initialize the CX register with the value 15, and the others with 0, we can use $\square$ CALL with a left argument of 3 elements:

```
0 0 15 □CALL CODE
0 0 15 0 0 0 0
```

The CX register has kept its initial value.

Now, let us create the APL function 'NOTHING'. Actually, the APL function can contain the binary code in a character format, as the code does not include any special character, as a BEL or FF. In a more general case, this method is impossible, so we have to represent the binary in another format. The floating point format is also unusable, as the least loss

of precision on the less significant digit of a real number would destroy the binary code. The best representation of a binary code in a function is the integer format.

In order to convert the character string into integers, we have to be sure that the number of characters is even. If it is not, we need to add another character to it. `⌈TCNUL` (value 0) is the one we will use. In our case:

```
CODE←163 ⌈BDR CODE,⌈TCNUL
```

CODE now is an integer, its value is 203. `⌈CALL CODE` is still valid. It is now easy to create the APL function 'NOTHING'.

Summary

This is only the beginning. In fact, an Assembler routine will often have to read one or more APL variables, and give, as a result, another APL variable. The first step to access to an APL variable is `⌈STPTR`. This is the subject of the next paper.

Here is an example of an APL function that checks the number of 1 in an APL boolean vector. The result is 1 if there is exactly one 1, otherwise result is 0.

```
▽ R←BOOL1 A
[1] R←1=+/A      A binary scalar or vector
[2] R           Wrong result if A does not contain only 0's and 1's.
[3] R           In that case : R←1=+/1=A
[4] R←'∘(⌈STPTR 'A')⌈CALL ⌈10101 1934 ⌈24282 6 572 4469
    2751 9728 3723 8 ⌈9429 ⌈896 29697 31753 ⌈13054 ⌈18176 1
    20303 3555 440 ⌈3584 30127 17158 ⌈20494 373 13123 ⌈31808
    507 373 ⌈13504
▽
```

The Assembler code contained in this function will be detailed in the next paper, to appear in VECTOR 4.3 (January 1988)

IBM Personal Computer APL/PC Version 2.0

6391329

IBM Personal Computer APL/PC Version 2.0, is a low cost, full function APL interpreter with a high degree of VS APL compatibility. It contains a wealth of auxiliary processors for a wide range of functions and interfaces to external devices.

- Emulates 8087 or 80287 if the co-processor is not present
- RS232 support
- IEEE-488/GPIB support
- Co-operative processing via IBM 3278/9 adapter
- Interface to IBM Macro Assembler and Professional Fortran
- APL2 GRAPHPAK compatible workspace provided
- Can run DOS functions and applications under APL
- Cover Workspaces for auxiliary processors

The interpreter, workspaces and auxiliary processors are supplied on three double-sided diskettes packaged with a comprehensive manual, quick reference card and a keyboard template. The manual includes setup, installation, tutorial and reference sections.

A separate package is supplied, containing a replacement ROM for the IBM Monochrome or Colour display adapters and a ROM puller. A program to load the APL font into the IBM Enhanced Graphics Adaptor is also included.

- Available from Authorised IBM PC Dealers.



IBM (UK) International Products Ltd
West Cross House
2 West Cross Way
Brentford
Middlesex TW8 9DY

The British APL Association Software Library Call for Software

by Dave Ziemann

Purpose

The purpose of the British APL Association Software Library (BAASL) is to provide useful software to the APL community. In order to achieve a high level of service, we need a good supply of quality software.

What kind of software?

We are looking for anything that could be of interest to the APL programmer, user or manager. It might be a tool for migrating APL workspaces between different interpreters, general APL utilities, APL2 functions for simulating component files, a reverse Polish model of APL or even a DOS adventure game. Each of these examples is an actual disk from the library.

There are no restrictions on the type of software, type of APL or the type of machine on which the software is to be run. As long as it is of interest to APLers, then it is worthy for consideration. The programs do not even have to be written in APL, although we do not want to fill the library with software which is easily available elsewhere. The BAASL is primarily concerned with offering 'public domain' software, which should normally include unlocked source code wherever possible. Bona-fide 'shareware' software is acceptable, and we are also happy to carry 'demonstration disks', which advertise commercially available software, provided that this is made quite clear.

If you are considering creating software for the library, read the software submission form first, so that you know what is expected of you. Please share the fruits of your work with others.

Transfer medium

The transfer medium for the BAASL is currently the DOS-format five-and-a-quarter inch floppy diskette. If you can get it onto a floppy disk, then we can make it available to others. If the target machine is not a PC, then appropriate transfer instructions should be included on the disk.

Documentation

The library will not currently handle any medium other than the floppy disk. In particular, we do not undertake to distribute paper documentation of any kind.

All disks must therefore include sufficient user documentation to permit effective use of the software. The donor may include an address where paper documentation can be obtained, possibly at a nominal cost.

Charges

It is not intended that the BAASL make a profit. Our charging policy is designed to cover the cost of materials, postage and handling, although we do make a reduced charge to BAA members in order to encourage membership.

A BAASL software donor must not require payment from the user, although reasonable charges for 'registration' or paper documentation may be advertised on the disk. Disks that demonstrate commercially available products are acceptable, provided that this is made clear.

Liability

A signature is required to declare the donor's right to allow the BAA to copy the software, and to permit anyone to copy and use it. Software is accepted by the BAA on good faith, and we do not vouch for or make any claims regarding donated software.

The BAA cannot be held responsible or liable for any damage, however caused, by the use or misuse of library software, and shall not be liable in any way in the event that submitted material proves to be the subject of third party copyright.

Reviewers

Unfortunately, we do not have the time to study each submitted disk in detail, although we check that disks do roughly what they claim to, and that they appear to work. We would like to publish reviews of library software in VECTOR, and so we welcome offers from all you keen reviewers out there. In fact we will even send your disks free of charge, if you write a review for VECTOR!

In any event, please write to tell us what you think of a particular BAASL disk, and how you found it useful. If you find glaring holes in the library, let us know what is missing and we will try to plug the gap. Better still, write it yourself.

British APL Association Software Library

Full Catalogue

DISK 1 The game of LIFE
SIZE=113K DONOR: Paul Chapman

Hardware required: PC

Software required:

Disk donated: July 1986

README	001	THE FILE YOU ARE READING
LIFE	EXE	Program file
LIFE	DOC	Documentation

GGUN	LIF	CYCLIC30	LIF	FIG11A	LIF	SHM	LIF	
ACORN	LIF	BG1	LIF	FIG11B	LIF	SHS	LIF	these
PUFFER	LIF	BG2	LIF	FIG11C	LIF	S9	LIF	files
SCHICK	LIF	BG3	LIF	FIG11D	LIF	S10	LIF	are
PREGUN	LIF	BG4	LIF	FIG11E	LIF	SHUTTLE	LIF	stored
MSHIPFAC	LIF	FIG15	LIF	FIG11F	LIF	SRING2X2	LIF	LIFE
TRIPWIRE	LIF	FIG13A	LIF	S1	LIF	SRING1X1	LIF	patterns
GRIDGLID	LIF	FIG13B	LIF	S2	LIF	GFRING1	LIF	for
GR1	LIF	FIG14	LIF	S3	LIF	GFRING2	LIF	you
GR1T2	LIF	FIG10A	LIF	S4	LIF	NEWGUN	LIF	to
GR1B	LIF	FIG10B	LIF	S5	LIF	MSHIP1	LIF	explore.
GR1C	LIF	FIG10C	LIF	S6	LIF	MSHIP2	LIF	
CYCLIC1	LIF	FIG10D	LIF	S7	LIF	REFL90	LIF	
CYCLIC2	LIF	FIG10F	LIF	S8	LIF	XLATE4	LIF	
CYCLIC3	LIF	FIG10E	LIF	SHL	LIF			

The large 65532 by 65532 universe is represented as a 'run-coding'. This means that the speed to calculate the next generation is proportional to the number of live cells rather than the universe size.

The program was prototyped using APL*PLUS PC, and then hand-coded in assembler. The development of the software is described in Paul's paper in the APL86 Tutorial Volume.

Extremely good fun, and a must for serious 'LIFE' researchers.

BAA Software Library

DISK 2 Fluctuation analysis for multichannel spectra
SIZE=9K DONOR: Claude Bastian

Hardware required: PC
Software required: APL*PLUS PC
Disk donated: July 1986

README 002 THE FILE YOU ARE READING
WALSH AWS

Further description in the APL86 Conference Proceedings,
on pages 50 to 58.

DISK 3 Free software from APL385
SIZE=282K DONOR: Adrian Smith

Hardware required: PC
Software required: APL*PLUS PC
Disk donated: July 1986

README 003 THE FILE YOU ARE READING
READ ME
FLIST PAS Pascal source code for
FLIST COM DOS file lister
SUPERMAN AWS Expert system shell
PHONES AWS Demo of DB/385 routines
APL385 AWS Screen design and assorted functions
DL AWS Selected Download and char design for FX-80
INIO AWS Straighten out text keyboard
UTIL AWS Useful file utilities and odds and ends
DL PDL Handy set of APL download symbols
HANDOUT PRN Type this to your Epson!
READ ME2 APL-385 license arrangements
CUCKOO AWS Digital cuckoo clock (honest)!

Documentation available for 50 Pounds or 75 US Dollars.

BAA Software Library

DISK 4 Piano
SIZE=4K DONOR: Gerard Langlet

Hardware required: PC
Software required: APL*PLUS PC
Disk donated: July 1986

README 004 THE FILE YOU ARE READING
PIANO AWS

DISK 5 Pianoman
SIZE=355K DONOR: Stuart Yarus *SHAREWARE*

Hardware required: PC
Software required:
Disk donated: July 1986

README 005 THE FILE YOU ARE READING
PLAYRPN COM
PIANOMAN COM
PIANOALT COM
MACROS ARC
TUNES ARC
TUNES2 ARC
COMTUNES ARC
COMTUNES DIR
TUNES DIR
TUNES2 DIR
MACROS DIR
READTHIS IST
PIANOMAN DOC
ARCX COM

Contribution of 25 US Dollars requested.

DISK 6 FRAME - Multidimensional numeric array editor
SIZE=69K DONOR: Robert Pullman

Hardware required: PC
Software required: Sharp APL/PC
Disk donated: July 1986

README 006 THE FILE YOU ARE READING
FRAME--- SAW
FRAMES-- SAF

Full-screen multidimensional numeric spreadsheet, with names for each dimension and column/row/plane etc.

BAA Software Library

DISK 7 Get/Put files for filing data by name
SIZE=21K DONOR: M Kent

Hardware required: PC
Software required: APL*PLUS PC
Disk donated: July 1986

README 007 THE FILE YOU ARE READING
GP AWS Get/Put workspace
GPHOW ASF Documentation (incomplete)

Functions for accessing filed data by item name rather than
component number.

DISK 8 Binary Combinations
SIZE=3K DONOR: Otway Pardee

Hardware required: PC
Software required: APL*PLUS PC
Disk donated: July 1986

README 008 THE FILE YOU ARE READING
COMB AWS

Very fast method for generating the binary image of combinations.

DISK 9 Workspace Lister for APL*PLUS PC
SIZE=16K DONOR: Roger Harper

Hardware required: PC, Epson LQ1500 printer
Software required: APL*PLUS PC

README 009 THE FILE YOU ARE READING
PCTYPE AWS

Pretty-prints variables and functions, with index and pagination.
Assumes LQ1500 printer.

BAA Software Library

DISK 10 TEXT - Typesetting for the IBM PC using APL*PLUS PC
SIZE=343K DONOR: Ian Feldberg

Hardware required: PC
Software required: APL*PLUS PC

README 010 THE FILE YOU ARE READING
FGRK AWS
FENG AWS
FGOTH AWS
FSCPT AWS
FITAL AWS
FMISC AWS
TEXT AWS

Requires BAA PDSL Disk 11 - GRAPHPAK for the IBM PC.
Documented in APL86 conference proceedings.

DISK 11 PCPAK - GRAPHPAK for the IBM PC
SIZE=33K DONOR: Ian Feldberg

Hardware required: PC
Software required: APL*PLUS PC
Disk donated: June 1986

README 011 THE FILE YOU ARE READING
PCPAK AWS Driver
POLAR AWS Polar coordinates
HP7550 AWS HP plotter software

Plot on a PC graphics board or HP7550 plotter.

DISK 12 Functions for Linear Algebra
SIZE=57K DONOR: Garry Helzer

Hardware required: PC
Software required: APL*PLUS PC
Disk donated: July 1986

README 012 THE FILE YOU ARE READING
LINALG AWS

Functions described in Helzer's book Applied Linear Algebra,
published by Little, Brown and Co., Boston 1983.

BAA Software Library

DISK 13 DOS utilities
 SIZE=57K DONOR: Timo Kiravi

Hardware required: PC
 Software required:
 Disk donated: July 1986

README	013	THE FILE YOU ARE READING
BROWSE	COM	View text files
CWEEP	EXE	Mass copy files, etc
D	COM	Formatted disk directory
DOSEEDIT	COM	Command line editor

DISK 14 APL to Reverse Polish Translation
 SIZE=311K DONOR: Richard J Naugle

Hardware required: PC
 Software required: APL*PLUS, IBM, SHARP or PortaAPL
 Disk donated: July 1986

README	014	THE FILE YOU ARE READING
CRCK4	DAT	Checksums for each file on disk
IMPORT--	SAW	Sharp APL ws for importing WSI files
APLDEMO	EXE	PortaAPL demonstration APL. File save disallowed
XLATE	APL	PortaAPL ws of APL to Reverse Polish translation
XLATE	AWS	APL*PLUS ws of APL to Reverse Polish translation
XLATE___	APL	IBM APL Vnl ws of APL to Reverse Polish translation
XLATEDEM	BAT	Batch command for APL to Reverse Polish demonstration
CRCK4	COM	Produces checksums of files
XLATE	WSI	)EXPORT of PortaAPL APL to Reverse Polish workspace
IMPORT__	APL	IBM APL workspace to import WSI workspace
README	PFS	PFS write file of README.1ST (Need PFS Write to use)
TDROM1		Used by APLDEMO when the PC contains an STSC ROM
XLATEDEM	IN	Input commands to PortaAPL for the demonstration
IMPORT	AWS	APL*PLUS workspace to import WSI workspace
README	1ST	ASCII documentation file

An APL model for translating APL statements to RL2 reverse polish expressions. As described in the APL Conference Proceedings. Workspaces supplied in four flavours, as above. A public domain version of PortaAPL that does not allow the saving of files is included.

NB: If you wish to check your disk by running the supplied checksum program CRCK4.COM, you must first erase the files README.014 and CRCK4.DAT from a copy of your disk.

BAA Software Library

DISK 16 F83VEC Demonstration
 SIZE=318K DONOR: Richard Naugle

Hardware required: IBM PC

Software required:

Disk donated: July 1986

README 016 THE FILE YOU ARE READING
 CRCK4 DAT Cyclic checksums for each file
 README PFS PFS WRITE format of README file
 APLDEMO EXE PortaAPL public domain demonstration
 PLOTDEM APL PortaAPL parabola plotting workspace
 PLOTDEM BAT Batch file for PortaAPL parabola plotting ws
 MUSICDEM WSI Workspace Standard Interchange format for below
 MUSICDEM APL PortaAPL Greensleeves playing workspace
 CRCK4 COM Program that produces file checksums
 PLOTDEM WSI Workspace Standard Interchange format for plot demo
 IMPORT-- SAW Workspace used to import .WSI wss into Sharp APL
 IMPORT AWS Workspace used to import .WSI wss into APL*PLUS
 README DEM Instructions for running plot and music demos
 TDROM1 PortaAPL support file for using STSC APL ROM
 F83VEC COM Forth object that extends F83 to use APL symbols
 EXAMPLES 4TH Forth examples that may be run under F83VEC
 DRIVERS 4TH Forth source to support display adaptor with IBM ROM
 MUSICDEM BAT Batch file for PortaAPL Greensleeves playing ws
 README VEC Documentation for F83VEC

This disk demonstrates the object code for an extension of FORTH that uses APL symbols and processes vectors in the APL style.

DISK 17 APL Migrate
 SIZE=268K DONOR: Richard Naugle

Hardware required: IBM PC

Software required: Sharp, IBM, Porta or APL*PLUS APL

Disk donated: February 1987

README.017 THE FILE YOU ARE READING
 README Overall instructions for APL migrate.
 ECO Changes since last version.
 NEC2050.COM Command prints or displays DOS ASCII+APL 1.0 files.
 CRCK4.COM Checksum command.
 CRCK4.DAT Root dir checksums produced by CRCK4 *.* >CRCK4.dat
 PORTA\README Instructions for the PORTA directory.
 PORTA\OUT.APL Migrates PortaAPL workspace to a ASCII+APL 1.0 file.
 PORTA\IN.APL Migrates ASCII+APL 1.0 file to a PortaAPL workspace.
 PORTA\EXPORT.APL Migrates PortaAPL workspace to a WSIS 0 file.
 PORTA\IMPORT.APL Migrates a WSIS 0 file to a PortaAPL workspace.
 PORTA\OUT.WAA ASCII+APL 1.0 file image of PORTA\OUT.APL.
 PORTA\IN.WAA ASCII+APL 1.0 file image of PORTA\IN.APL.
 PORTA\EXPORT.WAA ASCII+APL 1.0 file image of PORTA\EXPORT.APL.
 PORTA\IMPORT.WAA ASCII+APL 1.0 file image of PORTA\IMPORT.APL.

BAA Software Library

PORTA\CRCK4.COM	Checksum command.
PORTA\CRCK4.DAT	Porta directory checksums from CRCK4 *.* >CRCK4.dat
STSC\README	Instructions for the STSC directory.
STSC\OUT.AWS	Migrates STSC APL workspace to a ASCII+APL 1.0 file.
STSC\IN.AWS	Migrates ASCII+APL 1.0 file to a STSC APL workspace.
STSC\EXPORT.AWS	Migrates STSC APL workspace to a WSIS 0 file.
STSC\IMPORT.AWS	Migrates a WSIS 0 file to a STSC APL workspace.
STSC\OUT.WAA	ASCII+APL 1.0 file image of STSC\OUT.AWS.
STSC\IN.WAA	ASCII+APL 1.0 file image of STSC\IN.AWS.
STSC\EXPORT.WAA	ASCII+APL 1.0 file image of STSC\EXPORT.AWS.
STSC\IMPORT.WAA	ASCII+APL 1.0 file image of STSC\IMPORT.AWS.
STSC\CRCK4.COM	Checksum command.
STSC\CRCK4.DAT	STSC directory checksums from CRCK4 *.* >CRCK4.DAT
IBM\README	Instructions for the IBM directory.
IBM\OUT---.AIO	Migrates IBM APL workspace to a ASCII+APL 1.0 file.
IBM\IN---.AIO	Migrates ASCII+APL 1.0 file to a IBM APL workspace.
IBM\EXPORT.AIO	Migrates IBM APL workspace to a WSIS 0 file.
IBM\IMPORT.AIO	Migrates a WSIS 0 file to a IBM APL workspace.
IBM\OUT.WAA	ASCII+APL 1.0 file image of IBM\OUT---.AIO.
IBM\IN.WAA	ASCII+APL 1.0 file image of IBM\IN---.AIO.
IBM\EXPORT.WAA	ASCII+APL 1.0 file image of IBM\EXPORT.AIO.
IBM\IMPORT.WAA	ASCII+APL 1.0 file image of IBM\IMPORT.AIO.
IBM\CRCK4.COM	Checksum command.
IBM\CRCK4.DAT	IBM directory checksums from CRCK4 *.* >CRCK4.DAT
SHARP\README	Instructions for the SHARP directory.
SHARP\OUT____.SAW	Migrates Sharp APL workspace to a ASCII+APL 1.0 file.
SHARP\IN____.SAW	Migrates ASCII+APL 1.0 file to a Sharp APL workspace.
SHARP\EXPORT.SAW	Migrates Sharp APL workspace to a WSIS 0 file.
SHARP\IMPORT.SAW	Migrates a WSIS 0 file to a Sharp APL workspace.
SHARP\OUT.WAA	ASCII+APL 1.0 file image of SHARP\OUT____.SAW.
SHARP\IN.WAA	ASCII+APL 1.0 file image of SHARP\IN____.SAW.
SHARP\EXPORT.WAA	ASCII+APL 1.0 file image of SHARP\EXPORT.SAW.
SHARP\IMPORT.WAA	ASCII+APL 1.0 file image of SHARP\IMPORT.SAW.
SHARP\CRCK4.COM	Checksum command.
SHARP\CRCK4.DAT	Sharp directory checksums from CRCK4 *.* >CRCK4.dat

Contains workspaces to migrate Sharp APL, IBM, PortaAPL and APL+PLUS to DOS files in WSIS 0 and WSIS ASCII+APL 1.0 formats, in both directions. Also includes NEC Spinwriter printing software.

BAA Software Library

DISK 18 FFILES - Component files for APL2
SIZE=48K DONOR: A N Wiggins

Hardware required: PC, MAINFRAME
Software required: APL*PLUS PC, APL2
Disk donated: March 1987

README 018 THE FILE YOU ARE READING
FFILES AWS File utility. Similar to APL*PLUS sharefile.

Uses API21 under APL2 to simulate an APL*PLUS-type component filing system. The functions are stored on this disk in an APL*PLUS PC workspace. Instructions for transferring the software to the mainframe are NOT included. The functions in the workspace are as follows:

FCREATE Creates a new file
FTIE Ties the filename to a unique number
FAPPEND Appends data to the end of a tied file
FDIR Outputs the names of files
FDROP Replaces a component with 10
FERASE Erases the tied file
FLIB Shows the contents of the 'fnums' variable
FNAMES Shows the file names held in 'fnums'
FNUMS Shows the tie numbers currently in 'fnums'
FREAD Reads a specified record in the specified file
FREPLACE Replaces a specified record in a specified file
FUNTIE Unties the specified file
FRECORDS Counts the number of records in the specified file

DISK 19 CAVEDOS - Adventure front-end to DOS
SIZE=83K DONOR: Mark Bassett

Hardware required: PC
Software required: APL*PLUS PC 6.0
Disk donated: March 1987

README 019 THE FILE YOU ARE READING
CAVEDOS AWS APL DOS-adventure game
CAVEDOS DOC Brief guide to play

Now you can really explore your hard disk: but beware of dragons, pirates and dwarves! The more DOS commands you master, the higher your score. Hints, answers or full solution available by sending an S.A.E to the author.

The APL functions are locked.

**British APL Association
Software Library****Software Submission (1)**

Please copy and fill in this form for EACH disk you submit.

Details corresponding to items flagged (*) will NOT be made publically available, but are for our records only. Please Use BLOCK CAPITALS for all items except number 18.

0. Submission date: _____
1. Name of donor: _____ 2. Daytime phone(*): _____
3. FULL address(*): _____

4. Electronic mail addresses(*): _____
5. Short disk title: _____
6. Brief description of disk : _____

7. List target machine: _____
8. List additional software required: _____
9. Indicate special hardware requirements: _____
10. Is user documentation provided on the disk?(Y/N): _____
11. List titles of any paper documentation included with your submission

12. Is this documentation, or any other, available to users upon
application to you?(Y/N): _____ Please give details: _____

13. Does the disk include any form of payment request from users of the
software?(Y/N): _____ Please give details: _____

14. Does your submission constitute a 'demonstration disk' in that it
demonstrates software that is available for purchase?(Y/N): _____
15. If the TARGET machine is NOT a DOS-based PC, does the disk include
instructions for transfer to the target machine?(Y/N): _____

**British APL Association
Software Library**

Software Submission (2)

16. File names and descriptions. This information will be made publically available in the software library catalogue. Please save us some work by including these details on a file named <CAT> (no file extension) on your submitted disk. Please enter these details for all files on the disk, except <CAT> itself. You can affix a print of <CAT> below.

For APL workspaces, please use the space below to document functions

FILENAME.EXT	SHORT DESCRIPTION
--------------	-------------------

[illegible]

17. Use any extra space above for comments you wish to appear in the software library catalogue.
18. Your signature is necessary. It declares your legal right to make the disk freely available for copying and use, and grants the British APL Association a similar right.

Signature: _____

Mail your submission to: The BAA Software Library
c/o Dave Ziemann
Flat 3, 63 Queens Crescent
London NW5 4ES, ENGLAND

British APL Association Software Library

Order Form

To: Dave Ziemann
Flat 3, 63 Queens Crescent
London NW5 4ES, ENGLAND

PLEASE SUPPLY THE DISKS CIRCLED BELOW:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35
36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53
54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71
72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89
90	91	92	93	94	95	96	97	98	99	100							

TOTAL NUMBER OF DISKS ORDERED (please double-check): _____

at 2 Pounds per disk : _____ BAA Member's rate

OR at 3 Pounds per disk : _____ Non-member's rate

I want to join the BAA, and enclose
the 10 Pounds annual membership fee : _____

Postage and handling : _____ 1.00

Add 2 Pounds for orders outside the UK: _____

Total order - remittance included: _____

Your signature: _____

Your name: _____

Full address: _____

Please make cheques payable to the British APL Association.

Payment can be made in US Dollars - pay 1.5 US Dollars for each Pound.

All orders must include payment - allow 30 days for delivery.

BAA membership year runs from 1 May - 30 April. People joining part way through the year will receive appropriate back numbers of Vector.

Software is accepted by the British APL Association on good faith and we do not vouch for or make any claims regarding donated software. The BAA shall not be held responsible for damage caused by the use of library software, and shall not be liable in any way in the event that submitted material proves to be the subject of third party copyright. Your signature above accordingly indemnifies the BAA from all liability.

Index to Advertisers

APL People (half page)	29
Cocking and Drury	2
Dick Gray Associates (half page)	88
Dyadic Systems Ltd	30
HMW Programming Consultants	98
IBM PC	112
IBM Sweden	22
Lloyd Savage	18
MicroAPL	8
Simcorp	60
VECTOR back numbers (half page)	6
APL Booklist	36

All queries regarding advertising in VECTOR should be made to the advertising editor, Cathy Dargue, at the following address:

Cathy Dargue,
60 Downhall Ley,
Buntingford,
Herts SG9 9TL

Tel: 0707-325161

BRITISH APL ASSOCIATION**Membership Application Form**

Please read the membership information in the inside front cover of VECTOR before completing this form. Use photocopies of this form for multiple applications. The membership year runs from 1st May – 30th April.

Name: _____
 Department: _____
 Organisation: _____
 Address Line 1: _____
 Address Line 2: _____
 Address Line 3: _____
 Address Line 4: _____
 Post or zip code: _____
 Country: _____
 Telephone Number: _____

Membership category applied for (tick one):	87/88	
Non-voting student membership (UK only)	£ 5	
UK private membership	£ 10	
Overseas private membership	£ 18	\$ 27
Airmail supplement (not needed for Europe)	£ 8	\$ 12
Corporate membership	£ 85	
Corporate membership Overseas	£140	\$210
Sustaining membership	£360	

For student applicants:

Name of course: _____
 Name and title of supervisor: _____
 Signature of supervisor: _____

PAYMENT

Payment should be enclosed with membership applications in the form of a UK sterling cheque or postal order made payable to "The British APL Association". Corporate or sustaining member applicants should contact the Treasurer in advance if an invoice is required. Please enclose a stamped addressed envelope if you require a receipt.

Send the completed form to the Treasurer at this address:

Mel Chapman, 12 Garden Street, Stafford ST17 4BT, UK.

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by the vote of Association members at the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

1987/88 Committee

Chairman:	Dick Bowman 01-634-7639	CEGB, 85 Park Street, LONDON SE1.
Secretary:	Graham Parkhouse 04383-571281(x2379)	Dept. of Mech. Eng., University of Surrey, GUILDFORD GU2 5XH.
Treasurer:	Mel Chapman 0785-53511	12 Garden Street, STAFFORD ST17 4BT.
Activities:	Phil Goacher 03727-21282	27 Downs Way, EPSOM, Surrey.
Journal Editor:	Adrian Smith 0904-53071	Brook House, Gilling East, YORK.
Education:	Norman Thompson 0962-54433	IBM Mail Point 188, Hursley Park, Winchester Hants, SO21 2JN
Publicity:	Jill Moss 0225-62602	17, Barton Street Bath, Avon
Technical:	David Ziemann 01-493 6172	Cocking & Drury Ltd, 16 Berkeley Street, LONDON, W1X 5AE
Recruitment:	Val Lusmore 0225-62602	17, Barton Street Bath, Avon
Projects:	David Eastwood 01-922 8866	MicroAPL Ltd, South Bank Technopark, 90 London Road, LONDON SE1 6LN

Journal Working Group

Jonathan Barman	0374-588835
Anthony Camacho	0727-60130
Cathy Dargue	0707-32516
Val Lusmore	0225-62602
Adrian Smith	0904-53071
David Ziemann	01-493 6172

Activities Working Group

Phil Goacher	03727-21282
Maurice Jordan	01-562 3090
David Parker	01-834 2333

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for 'A Programming Language' - an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on many timesharing bureaux and on most mainframe, mini and micro computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

APL People
17 Barton Street
BATH, Avon
Tel:0225-62602

APL Software Tech.
14 Rosewood Avenue
Alveston, BRISTOL BS12 2PP
Tel:0454-415737

Cocking & Drury Ltd
180 Tottenham Court Rd
LONDON, W1P 9LE
Tel:01-436 9481

Dyadic Systems Ltd
Park House, The High Street
ALTON, Hants
Tel:0420-87024

HMW Progr. Consultants Ltd
142 Feltham Hill Rd.
ASHFORD
Middlesex TW15 1HN
Tel:07842-41232

APL Impetus Ltd
Rusper, Sandy Lane
Ivy Hatch, SEVENOAKS
Kent TN15 0PD
Tel:0732-885126

Inner Product Ltd
Eagle House
73 Clapham Common Southside
LONDON SW4 9DG
Tel:01-673-3354

Mercia Software Ltd
Astons Science Park
Love Lane
BIRMINGHAM B7 4BJ
Tel:021-359-5096

MetaTechnics Systems Ltd
Unit 216, 62 Triton Road
LONDON SE21 8DE
Tel:01-670-7959

MicroAPL Ltd
South Bank Technopark,
90 London Road,
LONDON SE1 6LN
Tel:01-922 8866

J.P. Sharp Associates
10 Dean Farrar St.
LONDON SW1H 0DX
Tel:01-222-7033

Peter Cyriax Systems
213 Goldhurst Terrace
LONDON NW6 3ER
Tel:01-624-7013
Mobile:0860-377963



The British Computer Society, 13 Mansfield Street, London W1M 0BD.